ssp-doc — Specifications for the Self-Service Platform (SSP) and its Software Dark Factory

This repository is the **source of truth** for everything the factory builds: the multi-cloud self-service provisioning platform (SSP, successor to Taxi), the apps around it (control room, migration), and the factory process itself. Code repositories implement what is written here; when code and spec disagree, the spec wins and the code changes, or a spec-change PR is opened here first.

Decisions in `00-constitution/02-decisions-register.md` come from the design session between Norm, Manoj and Dwayne and are **locked**. Nothing in this repo or in any code repo may deviate from them without a superseding ADR approved by the platform team.

How to read this repo

Folder	What it holds	Read it when
`00-constitution/`	Principles, locked decisions, glossary, definition of done, autonomy tiers	Always, first
`10-product/`	The SSP product: functional spec, domain model and DDL, Go API contract, Temporal workflows, OPA policy, service catalog spec, UI spec, conversational provisioning	Building `ssp-cp`, `ssp-workers`, `ssp-ui`, `ssp-module-*`
`20-apps/`	Control room, migration app, operator agent chat	Building `ssp-control-room`, `ssp-migrate`
`30-infra/`	GKE layout, Temporal install, Terraform worker, Postgres, state buckets, observability	Standing up or changing the platform's own infrastructure
`40-repos/`	Repo map, per-repo structure and conventions, module template	Creating or changing any repo
`50-factory/`	How agents build this: roles, gates, tiers, Cursor rules, task template	Setting up Cursor / CI, writing tasks
`60-adr/`	Architecture decision records (5 categories)	Any design change
`70-acceptance/`	Pointer to the holdout acceptance repo (not here, by design)	Setting up evaluation
`80-runbooks/`	Bootstrap order and operational runbooks	Day 0 and operations
`90-tasks/`	Work items for agents, each with spec references and acceptance criteria	Starting a build task

Requirement IDs

Every normative statement carries an ID of the form `SSP-<AREA>-<nnn>` (for example `SSP-API-012`) and uses MUST / SHOULD / MAY. Tests, policies and scenarios reference these IDs; the traceability gate fails a PR that changes behaviour covered by an ID without a test that cites it.

Repositories

Repo	Role
`ssp-doc`	This repo: specs, ADRs, tasks
`ssp-cp`	Control plane: Go APIs, Temporal workflows, policy engine, catalog loader, assistant gateway
`ssp-workers`	Temporal workers: Terraform runner launcher, post-build integrations, notifications, drift scanner

Repo	Role
`ssp-ui`	User-facing storefront (Next.js + Tailwind + shadcn/ui)
`ssp-module-<service>`	One repo per Terraform service module, e.g. `ssp-module-spanner`; each publishes a catalog bundle
`ssp-control-room`	Operator console and operator agent chat (new repo)
`ssp-migrate`	Migration app: inventory, mapping, one-time state import (new repo)
`ssp-acceptance`	Holdout scenarios and evaluator (new, private; never opened in a builder workspace)

Status

Version 0.1, 2026-10-20. Phase 0 (bootstrap) specs are complete enough to build the first vertical slice: one GCP service (`gcp.spanner-instance` is the worked example; `gcp.gcs-bucket` is the simplest alternative) through all four states, with Temporal and the Terraform worker on the existing GKE cluster.

01 — Principles

These are the non-negotiables. Each later spec is an elaboration of one or more of these.

ID	Principle	Consequence
SSP-PRIN-001	**Self-service only; behave like a hyperscaler.**	No ticket-driven provisioning path exists. Every capability is reachable through UI, API and chat with the same contract. Humans in the platform team operate the platform; they do not provision for tenants.
SSP-PRIN-002	**Four states, nothing else.** Requested → Approved → Expected → Observed.	Every resource the platform knows about is in exactly one state at a time; drift is defined as Expected ≠ Observed.
SSP-PRIN-003	**Approval is guardrail application, not a signature.**	"Approved" means the request plus every guardrail the catalog and policy enforce (CMEK, retention, security requirements). It is produced by code. A human approval or ServiceNow CR is an additional gate only where change management requires it.
SSP-PRIN-004	**No secrets before Expected.**	Requested and Approved never contain secrets. Secrets are injected after approval, only into Expected, and are kept out of Terraform state wherever the provider allows.
SSP-PRIN-005	**One tenant boundary, one identity, one state.**	Each GCP project / AWS account / Azure subscription has exactly one platform service account and one Terraform state file.
SSP-PRIN-006	**Control plane in GCP.**	The platform runs on GCP (GKE, Cloud SQL, GCS). Other clouds are targets, reached by federated identity, never by static keys.
SSP-PRIN-007	**Generate, don't duplicate.**	The ServiceSpec is the single definition of a service. UI forms, API validation, policy inputs, Terraform variables, pricing and docs are derived from it. Adding a service never requires redeploying the storefront.
SSP-PRIN-008	**Everything is a workflow.**	Provisioning, post-build, drift, import and decommission run as Temporal workflows on the platform's own Temporal cluster. There are no cron scripts or hand-run playbooks on the critical path.
SSP-PRIN-009	**Humans own intent; agents own code; gates own merge.**	Specs, ADRs, holdout scenarios and gate definitions are written by humans. Code is written by agents. Automated gates decide what merges, with human approval added by autonomy tier.
SSP-PRIN-010	**Blast radius is designed, not assumed.**	Every identity, bucket, lock and module upgrade has an explicit blast-radius statement and a kill switch.
SSP-PRIN-011	**Show the code.**	Every order can be viewed as the equivalent Terraform and CLI, so the platform never hides what it does.
SSP-PRIN-012	**Design for the 2.0 layout.**	One APM ID per service project. Legacy (1.0, multi-app shared projects) is imported through the migration app, not accommodated in the core model.

02 — Decisions Register (locked)

Source: design session, Norm / Manoj / Dwayne, October 2026 (transcript in the project archive). A decision changes only through a superseding ADR in `60-adr/` approved by the platform team.

#	Area	Decision	Rationale recorded in session	ADR
D-01	Control plane cloud	**GCP.** AWS rejected as home cloud.	AWS is the smallest footprint; Aetna customer-facing stack lives there; the platform that runs everything should not live in the smallest cloud.	G-0001
D-02	State model	**Four states:** Requested → Approved → Expected → Observed. Drift = Expected vs Observed.	Clean separation of intent, policy, plan and reality.	G-0002
D-03	Requested + Approved storage	**Centralized Postgres in GCP.** No secrets stored.	Transactional, queryable, auditable.	G-0002
D-04	Approved definition	Approved = request **plus enforced guardrails** (CMEK, retention, security requirements).	Approval is policy application.	G-0002
D-05	Secrets	Injected **after approval**, into Expected only.	Keeps the request store free of secrets.	G-0004
D-06	Expected + Observed (Terraform state)	**Object storage** (GCS / S3 / Blob) with native locking. Postgres rejected.	Write amplification, TOAST/vacuum churn, weaker trust boundary in Postgres; S3 no longer needs DynamoDB for locking.	C-GCP-0002
D-07	Bucket placement	**One bucket per LOB:** GCP host project, AWS account per OU, Azure management group. Per-project buckets rejected.	Unmanageable at 5,000+ projects.	C-GCP-0002
D-08	State granularity	**One state file per project / account / subscription.**	Manoj's prior per-category pattern discussed, not adopted.	A-0003
D-09	Scope	**Design for the 2.0 layout first** (one APM ID per service project). Legacy import later via a translation shim. CAP is an outlier that must move onto the platform.	Avoid designing around legacy shared projects.	G-0003
D-10	Identity	**One service account per project / account / subscription**, reached by **impersonation** from a central platform account that doubles as a **kill switch**. Taxi's 3 SAs (prod / nonprod / sandbox) are not reused.	Blast radius and auditability.	G-0005
D-11	Orchestration	**Self-hosted Temporal** on a dedicated GKE cluster/namespace, not the shared DevX Temporal.	Isolation; confidentiality of the project.	A-0001
D-12	Terraform execution	Dedicated Temporal **activity worker → Kubernetes Job** per operation. Job name derived deterministically from the operation ID so a retry re-attaches. **Polling (~30 s) first**, callback later. Worker and jobs in the same cluster.	Idempotent retries; no ingress on the worker.	A-0002
D-13	Policy	**New OPA rules** against the platform's own JSON schema. Nothing reused from Taxi.	Clean policy model.	A-0005

#	Area	Decision	Rationale recorded in session	ADR
D-14	Post-build / bootstrap	Orchestrated in **Temporal**. POC converts team playbooks (Ansible host build, DBA Jenkins) into Temporal workflows; long-term ownership stays with the agent teams.	One orchestration model.	B-0004
D-15	Operating model	**Self-service only.**	Hyperscaler behaviour.	G-0001
D-16	ADR taxonomy	**5 types:** Global, Business Process, Endpoint (per service), Per-Cloud, Automation. Template: Goal, Requirements, Guardrails, Success Criteria. Platform team are sole PR approvers. Each ADR has an owner plus a shared mailbox.	Ownership tracking.	B-0007
D-17	UI	**Chat, search top-center, catalog of frequent items, show the equivalent IaC/CLI.** APM-ID-driven defaults (e.g., Tier 1 → active-active DB). Template-driven micro-frontend so new services don't need a redeploy. Visual direction: prototype *Gemini 4 Argon / shadcn-stack / #17 "Shopify Polaris Infrastructure Storefront*".	Chosen from the prototype gallery.	A-0008
D-18	Build approach	**"Software dark factory."** One repo per Terraform resource module plus a spec/ADR repo. Agents generate code from specs. Humans design reconciliation gates and performance tests (target 10–20 provisions/day initially).	Throughput with control.	A-0009
D-19	Repos	`ssp-ui`, `ssp-cp` (Go APIs + Temporal workflows), `ssp-workers`, `ssp-doc`, `ssp-module-<service>`; new repos `ssp-control-room`, `ssp-migrate`. APIs deployed to the existing GKE cluster; existing GCP project.	Owner's repo layout.	A-0009
D-20	Assistant	Conversational provisioning on a **Contextual Language Model** (grounded, retrieval-trained) for the knowledge half; schema-constrained tool calling plus deterministic code for the action half.	Groundedness without letting a model approve anything.	A-0011

Open items that block specific areas (not decisions)

Item	Blocks	Owner from session
AWS OU structure and per-OU host account feasibility	AWS provider work	Dwayne (via Security)
Azure management group / subscription mapping	Azure provider work	Norm
Per-project SA isolation requirements and a test account	Identity rollout beyond POC	Dwayne → Raju / Durant
Taxi + console integration inventory (firewall, F5 vs cloud LB, Kafka, Infoblox, ServiceNow, CMDB, backup, patching)	Post-build workflows	Norm
Post-build / post-post-build steps per resource type	Post-build workflows	Manoj / Norm

Item	Blocks	Owner from session
CMDB CI classes for PaaS resources	CMDB registration	Dwayne → Ken O'Connor
API gateway path (Apigee / APGX via API Forge)	Public API exposure	Dwayne

03 — Glossary and cloud-neutral ontology

Use these words exactly. The storefront metaphor (from the chosen prototype) and the platform model map one-to-one; the table makes the mapping explicit so UI, API and workflows never disagree.

Platform terms

Term	Definition	GCP	AWS	Azure
Tenant boundary	The unit that owns one platform service account and one Terraform state file	Project	Account	Subscription
LOB boundary	The grouping that owns one state bucket and one CMEK key	Host project (folder)	OU (per-OU host account)	Management group
APM ID	Application identifier from the enterprise portfolio; drives tier, data classification, cost center, entitlements	label `apm_id`	tag `apm_id`	tag `apm_id`
Tier	Application criticality (1 = highest). Drives defaults such as active-active databases	—	—	—
Data classification	PHI / PII / Confidential / Internal / Public. Drives CMEK, retention, residency	—	—	—
Service	A provisionable offering defined by one ServiceSpec, implemented by one `ssp-module-<service>` repo	e.g. `gcp.spanner-instance`	e.g. `aws.s3-bucket`	e.g. `azure.sql-database`
ServiceSpec	The YAML definition of a service: request schema, guardrails, module reference, UI template, pricing, post-build steps, lifecycle			
Catalog bundle	The released artifact of a service: ServiceSpec + request schema + UI template + Rego + pricing + module zip, versioned			
Order	A user's request to create, change or delete one resource instance in one tenant boundary. The API object behind "Requested"			
Approved spec	The immutable result of applying guardrails and policy to an order. The object behind "Approved"			
Resource instance	One provisioned thing (a Spanner instance, a bucket) produced by one order; lives in a tenant boundary's desired set			
Desired set	All approved resource instances of a tenant boundary; rendered into that boundary's single Terraform root module. The object behind "Expected"			
Run	One Terraform operation (plan, apply, import, refresh, destroy) executed as a Kubernetes Job for one tenant boundary			
Observed state	The Terraform state after apply plus periodic refresh, for one tenant boundary. The object behind "Observed"			
Drift	Any difference between the desired set's rendered configuration and the observed state			

Term	Definition	GCP	AWS	Azure
Platform SA	The central service account the Terraform runner uses to impersonate tenant SAs; disabling it is the kill switch	`ssp-tf-runner@<cp-project>`	role in the platform account	app registration
Tenant SA	The one service account per tenant boundary that Terraform acts as	`ssp-tenant@<project>`	role `ssp-tenant` in account	managed identity
Post-build	Steps after apply that register or configure the resource (CMDB, backup enrolment, monitoring, notifications)			
Kill switch	Disabling the Platform SA, or pausing the reconciler at global / LOB / tenant scope			

Storefront vocabulary ↔ platform model

Storefront word (UI)	Platform object	State
SKU	Service (catalog bundle)	—
Delivery destination	Tenant boundary the buyer is entitled to	—
Configure & Buy / Checkout	Building an order from the ServiceSpec request schema	—
Order Summary (receipt)	Preflight result: guardrails applied, policy result, price estimate, equivalent code	—
Place order	`POST /v1/orders`	**Requested**
Order confirmed	Approved spec created (guardrails applied, policy passed, CR attached if needed)	**Approved**
Preparing / Shipping	Desired set re-rendered; run planned and applied	**Expected**
Delivered	Observed state captured; post-build complete	**Observed**
Fulfilled Orders	Orders in Observed state	—
Wishlist	Saved drafts	—
Sidekick Concierge	Conversational provisioning assistant	—

Identifiers

ID	Format	Example				
Order ID	`ord_` + ULID	`ord_01JAM3K8Z2W7Q4N9B5X0F1C2D3`				
Run ID	`run_` + ULID	`run_01JAM3…`				
Operation ID	`<run_id>:<attempt>` ; Job name = `tf-` + first 20 hex of SHA-256(operation ID)	`tf-9f2c1a0b7d3e4f5a6b7c`				
Service ID	`<cloud>.<service-slug>`	`gcp.spanner-instance`				
Requirement ID	`SSP-<AREA>-<nnn>`	`SSP-API-012`				
ADR ID	`<G	B	E	C	A>-<nnnn>` (E and C carry a suffix)	`E-SPAN-0001`, `C-GCP-0002`

Words we do not use

- "Approve" for a human click — say "CR approval" or "owner sign-off".
- "Environment" for a tenant boundary — say project / account / subscription, or "tenant boundary".
- "Workspace" — ambiguous (Terraform, Temporal, Slack).
- "Stack" — ambiguous; say desired set or root module.

04 — Definition of Done

A change is done when all of the following are true. Gates in `50-factory/54-gates.md` automate every line that can be automated.

For any PR

- [ ] Task in `90-tasks/` referenced; every cited `SSP-*` requirement is implemented or explicitly deferred in the PR description.
- [ ] Tests cite the requirement IDs they cover; coverage of changed packages does not fall.
- [ ] Lint, format, vulnerability scan (gitleaks, govulncheck / npm audit / trivy) pass.
- [ ] No secrets, no credentials, no real tenant identifiers in fixtures.
- [ ] Docs updated where behaviour changed (OpenAPI, ServiceSpec, runbook).
- [ ] Autonomy tier labelled (`tier:T0`…`tier:T3`) and the merge rule for that tier satisfied.

For `ssp-cp` (Go API, workflows)

- [ ] OpenAPI regenerated and `oasdiff` shows no breaking change, or the API version was bumped.
- [ ] Temporal workflow changes pass the replay test against recorded histories, or are versioned with `workflow.GetVersion`.
- [ ] sqlc regenerated; migration is forward-only and reversible by a new migration.
- [ ] Structured logs and OpenTelemetry spans on every new handler and activity.

For `ssp-workers`

- [ ] Activities are idempotent; retries re-attach to existing Jobs by deterministic name.
- [ ] Heartbeats emitted at least every 30 s on long activities; timeouts set explicitly.

For `ssp-module-<service>`

- [ ] `terraform fmt`, `validate`, `tflint`, `trivy config` clean.
- [ ] `terraform test` passes; at least one test per MUST guardrail in the ServiceSpec.
- [ ] Blast-radius gate: planning the new version against the sanitized state corpus produces no undeclared destroy/replace; refactors use `moved` blocks.
- [ ] ServiceSpec validates against `service-spec.schema.json`; request schema, UI template and pricing are consistent (generated check).
- [ ] Rego tests pass with ≥ 90 % coverage; policy-impact replay reviewed.
- [ ] CHANGELOG entry; semantic version tag; catalog bundle published to the catalog bucket.

For `ssp-ui`, `ssp-control-room`, `ssp-migrate`

- [ ] Renders from the catalog bundle with no code change for a new service (verified by the "unknown service" fixture).
- [ ] Keyboard navigation, focus order and contrast meet WCAG 2.1 AA on changed screens.
- [ ] Visual regression snapshots updated intentionally, not blindly.
- [ ] Matches the design tokens in `10-product/16-ui-spec.md`.

For infrastructure (`30-infra/`)

- [ ] Applied in sandbox first; verification steps in the runbook executed and recorded.

- [ ] Two human approvals including security for anything in tier T3.

05 — Autonomy tiers

Every PR is labelled with exactly one tier. The tier decides who, or what, may merge. Labels are applied by a CI job from the paths touched; a human may raise a tier, never lower it.

Tier	What it covers	Merge rule	Typical repos / paths
T0	Docs, tests, twins (fakes of external systems), fixtures, UI copy	Auto-merge on green gates	`ssp-doc` non-constitution, `**/tests/**`, `ssp-workers/twins/**`
T1	New catalog entry or module version **not yet used by any tenant**; new UI template for such a service	Green gates + scenario evaluator pass → auto-merge	`ssp-module-*` first release, `ssp-ui/templates/**`
T2	Modules, policies, workflows, API or UI **already in use** by tenants	Green gates + evaluator + **1 human** from the platform team	`ssp-cp/**`, `ssp-workers/**`, `ssp-ui/app/**`, `ssp-module-*` ≥ v1 with tenants
T3	Identity and impersonation, state backend, network, policy relaxations, gate definitions, the factory itself, platform foundation, anything under `00-constitution/`	Agents draft only. **2 humans including security**, applied in sandbox first	`30-infra/**`, `50-factory/**`, `.github/**`, IAM/WIF Terraform, `policies/**` where a rule is loosened

Rules

- SSP-TIER-001: A PR touching paths in two tiers takes the higher tier.
- SSP-TIER-002: Any Rego change that reduces the set of denied inputs on the policy-impact replay is T3 regardless of path.
- SSP-TIER-003: Any module change whose blast-radius gate shows a destroy or replace is T3 even when declared.
- SSP-TIER-004: Changes to `ssp-acceptance` are human-only and are never visible to builder agents.
- SSP-TIER-005: An agent may not approve a PR. Agent review comments are advisory; the merge decision is the gate plus the humans the tier requires.

10 — SSP Product Specification

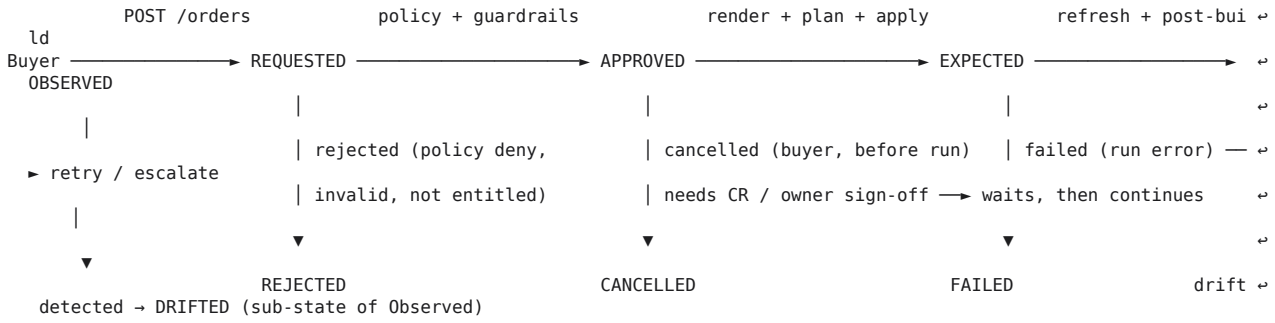
The Self-Service Platform (SSP) lets an application team order cloud resources the way they would order from a store, and guarantees that what is delivered meets every enterprise guardrail. It replaces Taxi. This document is the functional contract; the API, workflow, policy, catalog and UI specs refine it.

1. Personas

Persona	Who	Can	Cannot
Buyer	Developer or app team member entitled to one or more APM IDs	Browse catalog, configure and place orders for entitled tenant boundaries, view status, view equivalent code, change or decommission resources their APM owns, chat with Sidekick	See other APMs' resources; override guardrails; touch platform settings
App owner	Accountable owner of an APM ID	Everything a buyer can, plus sign off on orders flagged `owner_signoff` (e.g., production deletes), manage buyer entitlements for the APM	Operate the platform
Platform operator	Platform team	Control room: fleet status, pause/resume, retry runs, drift triage, module rollout waves, catalog publication, operator chat	Place orders on behalf of tenants in production (self-service only, D-15)
Security reviewer	Security engineering	Policy bundles, policy-impact review, T3 approvals, audit views	Operate workflows
Auditor	Compliance	Read-only across orders, approvals, runs, evidence	Anything else

Entitlements: SSP-PROD-001 Buyers are entitled through APM ownership records (source: enterprise portfolio / ServiceNow; POC: `entitlements.yaml` in the control plane). SSP-PROD-002 A buyer sees only tenant boundaries whose APM IDs they are entitled to. SSP-PROD-003 Entitlement is evaluated on every API call, never cached in the UI as authority.

2. The four states



State	Stored in	Produced by	Contains
Requested	Postgres `orders`	API on `POST /v1/orders`	Buyer identity, service id + version, tenant boundary, requested parameters (schema-valid), preview hash, idempotency key

State	Stored in	Produced by	Contains
Approved	Postgres `approved_specs`	Approval workflow	Immutable merged parameters (requested + injected guardrails), policy decision record, price estimate, change-management reference (CR id) if any, approved-by (system or human), hash
Expected	GCS LOB bucket `…/<tenant>/expected/` + Postgres pointer	Reconciler workflow	Rendered root module (JSON), module versions, backend config, plan JSON, secret references (not values)
Observed	GCS LOB bucket `…/<tenant>/terraform.tfstate` + Postgres `resource_instances` summary	Run (apply, refresh) + post-build	Terraform state; resource ids, outputs (non-secret), post-build receipts, last-observed timestamp

Requirements:

- SSP-PROD-010 A state transition MUST be recorded as an immutable event (`order_events`) with actor, timestamp, reason and correlation ids (order, run, workflow).
- SSP-PROD-011 Requested and Approved MUST NOT contain secret values. Fields marked `secret: true` in the request schema are accepted only as Secret Manager references, or generated by the platform after approval.
- SSP-PROD-012 Approved specs are immutable. A change to a live resource is a new order of kind `change` referencing the resource instance; a delete is an order of kind `decommission`.
- SSP-PROD-013 Drift is a sub-state of Observed and MUST NOT move the resource out of Observed; it is cleared by a reconcile run or by an explicit `adopt` order.

3. Approval rules

- SSP-PROD-020 Approval is computed by code: validate against request schema → apply guardrails from the ServiceSpec (inject / deny / warn) → evaluate request-stage OPA policy → compute price estimate → check tenant quotas and APM budget → produce the approved spec.
- SSP-PROD-021 Auto-approve when: policy result is `allow`, no guardrail returned `deny`, estimate within APM monthly budget headroom, and the change-management mapping requires no CR.
- SSP-PROD-022 Require a **ServiceNow change request** when the tenant boundary is production and the change class mapping (B-0002) says so (initially: any `change` or `decommission` in production; `create` in production is a standard change recorded automatically). The order waits in Approved with `gate: cr_pending` and continues when the CR is approved (polled or webhook).
- SSP-PROD-023 Require **owner sign-off** for `decommission` of any resource holding data (ServiceSpec `lifecycle.holdsData: true`) and for `change` orders that the ServiceSpec marks `ownerSignoff: true`.
- SSP-PROD-024 Policy `deny` rejects the order with the rule ids and a human-readable reason; `warn` is shown in the receipt and recorded.

4. Defaults from context (APM-ID-driven)

- SSP-PROD-030 When a buyer selects a delivery destination, the platform loads the APM record (tier, data classification, cost center, LOB) and applies ServiceSpec `derive` rules before the form is shown. Example: Tier 1 database → multi-region / active-active; PHI → CMEK + retention lock + private access only.
- SSP-PROD-031 Derived fields are shown read-only with the rule that set them and the requirement id, and can be relaxed only if the ServiceSpec marks them `relaxable` (then a `warn` and owner sign-off apply).

5. Order journeys (buyer)

1. **Browse**: storefront home shows frequent SKUs for the buyer's history and LOB, delivery destinations grouped by cloud, recently viewed, saved drafts. Search (top-center) finds SKUs, destinations, orders and

resources by id or name. (SSP-PROD-040)

2. **Configure & Buy**: multi-step checkout generated from the request schema and `ui.yaml`; a sticky Order Summary shows itemized estimate, injected guardrails, policy warnings and a **View as code** tab (Terraform and CLI). (SSP-PROD-041, SSP-PROD-042)
3. **Preflight**: every edit debounced to `POST /v1/orders/preflight`; the receipt is always the server's view. (SSP-PROD-043)
4. **Place order**: `POST /v1/orders` with the preview hash; mismatch → 409 and the UI refreshes the receipt. (SSP-PROD-044)
5. **Track**: order page streams events (SSE) through Requested → Observed, including run logs (redacted) and post-build receipts; estimated time remaining from historical percentiles for the service. (SSP-PROD-045)
6. **Operate**: resource page shows observed attributes, drift, cost to date (if available), and actions: change, decommission, re-run post-build, export code. (SSP-PROD-046)
7. **Ask Sidekick**: at any point, open the concierge; it can prefill a checkout, explain a guardrail or policy denial with citations, or report status. It never places an order without the same confirmation card. (SSP-PROD-047; see `17-conversational-provisioning-spec.md`)

6. Desired set and reconciliation (how Expected is produced)

Because there is exactly one state file per tenant boundary (D-08), the platform maintains a **desired set** per tenant boundary and renders it to a single root module.

- SSP-PROD-050 Each tenant boundary has one long-running Temporal entity workflow `TenantReconciler(tenant_id)` that owns the lock on that boundary's state and processes a FIFO queue of approved specs (signals).
- SSP-PROD-051 On each cycle the reconciler renders `main.tf.json` with one `module` block per resource instance in the desired set, pinning each to its approved module version, plus provider and backend blocks, and writes it to `…/<tenant>/expected/<generation>/`.
- SSP-PROD-052 The reconciler batches queued specs that arrived within a configurable window (default 20 s) into one run, except specs with `isolation: true` (e.g., decommission), which run alone.
- SSP-PROD-053 A run is `plan` → plan-stage policy → `apply` only if the plan touches exactly the resource instances in the batch (the plan gate); any unexpected destroy/replace fails the run and raises an operator alert.
- SSP-PROD-054 Module upgrades are separate orders of kind `upgrade` created by the Release process in **waves** (canary tenants → one LOB → all), each wave gated by the plan gate and a soak period.
- SSP-PROD-055 Per-tenant concurrency is 1 run at a time; the queue depth and age are exposed as metrics and shown in the control room. Fairness across tenants is provided by Temporal task queues per LOB with equal worker allocation.

7. Drift

- SSP-PROD-060 A scheduled `DriftScan(tenant_id)` (default every 6 h, and after any run) performs `terraform plan -refresh-only` and classifies drift per resource instance: `benign` (allowlisted attributes), `config` (platform-managed attribute changed), `missing` (resource gone), `unmanaged` (new resource in the boundary not in the desired set, from cloud asset inventory).
- SSP-PROD-061 Default response: record, notify buyer and operator, show on resource and control room; **no automatic revert** in Phase 1. Per-service `drift.autoRevert: true` enables reconcile for `config` drift after a grace period.
- SSP-PROD-062 Break-glass: an operator can mark a tenant `break_glass` for a window; drift during the window is recorded with the incident reference and excluded from auto-revert.

8. Decommission

- SSP-PROD-070 Decommission is an order of kind `decommission`; the ServiceSpec defines `lifecycle.delete.mode: soft|hard` and `retentionDays`. Soft delete first disables/locks, then removes from the desired set after retention; hard delete removes immediately after required sign-offs.
- SSP-PROD-071 Decommission in production always requires owner sign-off and, per B-0002, a CR.
- SSP-PROD-072 Orphan cleanup: `unmanaged` drift older than 30 days raises an `adopt or remove` task to the app owner.

9. Post-build

- SSP-PROD-080 Post-build steps are declared per service (`postBuild: [...]`) and executed by the `PostBuild` child workflow after apply: CMDB registration, backup enrolment, monitoring onboarding, team notification, DNS (Infoblox), firewall requests. Each step is an activity with its own idempotency key and receipt.
- SSP-PROD-081 A failed post-build step does not roll back the resource; the order shows `Observed (post-build incomplete)` and the step is retried with backoff, then escalated to the owning agent team's queue.

10. Status, notifications, monitoring

- SSP-PROD-090 Buyers receive notifications (email + optional Teams/Slack webhook) on: order placed, approved, waiting on CR/sign-off, delivered, failed, drift detected.
- SSP-PROD-091 The order page and the control room read the same event stream; there is no separate "status" store.
- SSP-PROD-092 Platform SLOs (initial): order → Approved p95 < 60 s when no CR; Approved → Observed p95 < 20 min for Phase 1 services; API availability 99.9 %; capacity target 10–20 provisions/day in Phase 1, designed for 500/day.

11. Security and audit

- SSP-PROD-100 AuthN: OIDC from the corporate IdP; the API accepts JWTs issued through the API gateway (API Forge / Apigee) and verifies issuer, audience and expiry. Service-to-service calls use GKE Workload Identity.
- SSP-PROD-101 AuthZ: persona + entitlement checks in the API; field-level rules from the ServiceSpec (`ops_only` fields are hidden and rejected for buyers).
- SSP-PROD-102 Every API write, workflow decision, run and post-build receipt is an audit event with actor, subject, before/after hashes; events are exported to the SIEM.
- SSP-PROD-103 The Platform SA can be disabled from the control room in one action (kill switch); reconcilers detect the disabled state and pause.
- SSP-PROD-104 Runs never receive long-lived credentials: Workload Identity → Platform SA → short-lived impersonated token for the tenant SA, scoped to the run's lifetime.

12. Multi-cloud sequencing

GCP first (control plane and first services), AWS second (per-OU host account pending Security), Azure third. The ServiceSpec, API and UI are cloud-neutral from day one; only `ssp-module-*` repos and the runner's provider configuration are cloud-specific.

13. Out of scope for Phase 1

Legacy 1.0 import (handled by `ssp-migrate`, Phase 2), AWS/Azure execution, auto-revert of drift, cost actuals (estimates only), customer-defined approval chains.

11 — Domain Model and Control-Plane Schema (Postgres)

Postgres holds Requested and Approved, pointers to Expected/Observed objects, catalog index, entitlements and audit. It never holds secrets or Terraform state (D-03, D-06).

1. Entity overview

```
lob ──< tenant ──< resource_instance >── service_version
 │         │              │
 │         │              └──< order ──< order_event
 │         │                      │
 │         │                      └── approved_spec ──< run ──< run_artifact
 │         └──< entitlement >── principal
 └── state_bucket
catalog_service ──< service_version ──< pricing_rule
policy_bundle, drift_finding, control_flag, audit_event
```

2. DDL (migration 0001)

```
-- Migration tool: goose or atlas; forward-only. Extension for ULIDs is not required; IDs are text.
create schema ssp;
set search_path to ssp;

create type cloud as enum ('gcp','aws','azure');
create type tenant_kind as enum ('project','account','subscription');
create type order_kind as enum ('create','change','decommission','upgrade','adopt','import');
create type order_state as enum ('requested','approved','expected','observed','rejected','cancelled','failed');
create type approval_gate as enum ('none','cr_pending','owner_signoff_pending');
create type run_kind as enum ('plan','apply','refresh','import','destroy');
create type run_state as enum ('queued','running','succeeded','failed','cancelled');
create type drift_class as enum ('benign','config','missing','unmanaged');

create table lob (
  lob_id            text primary key,                -- e.g. 'pbm', 'retail'
  name              text not null,
  cloud             cloud not null,
  boundary_ref      text not null,                   -- host project id / OU id / management group id
  state_bucket      text not null,                   -- gs://ssp-state-<lob> | s3://... | https://...blob...
  cmek_key          text not null,                   -- KMS key resource name for the bucket
  created_at        timestamptz not null default now()
);

create table tenant (
  tenant_id         text primary key,                -- '<cloud>:<boundary id>' e.g. 'gcp:pay-prod-7a1c'
  lob_id            text not null references lob,
  cloud             cloud not null,
  kind              tenant_kind not null,
  boundary_ref      text not null,                   -- project id / account id / subscription id
  apm_id            text not null,                   -- 2.0 layout: one APM per tenant
  tier              smallint not null check (tier between 1 and 4),
  data_class        text not null,                   -- PHI|PII|CONF|INTERNAL|PUBLIC
  cost_center       text not null,
  env               text not null,                   -- prod|nonprod|sandbox
  tenant_sa         text not null,                   -- ssp-tenant@<project>.iam.gserviceaccount.com / role arn / cli ↵
   ent id
  state_prefix      text not null,                   -- '<boundary_ref>/' inside the LOB bucket
  paused            boolean not null default false,
  break_glass_until timestamptz,
  created_at        timestamptz not null default now(),
  unique (cloud, boundary_ref)
);

create table principal (
  principal_id      text primary key,                -- OIDC subject
  email             text not null unique,
  display_name      text,
  personas          text[] not null default '{buyer}' -- buyer|app_owner|operator|security|auditor
);

create table entitlement (
  principal_id      text references principal,
```

```
  apm_id           text not null,
  role             text not null check (role in ('buyer','app_owner')),
  primary key (principal_id, apm_id)
);

create table catalog_service (
  service_id       text primary key,                -- 'gcp.spanner-instance'
  cloud            cloud not null,
  display_name     text not null,
  category         text not null,                   -- database|storage|compute|network|messaging
  adr_id           text not null,                   -- E-SPAN-0001
  owner_mailbox    text not null,
  enabled          boolean not null default true
);

create table service_version (
  service_id       text references catalog_service,
  version          text not null,                   -- semver of the catalog bundle
  bundle_uri       text not null,                   -- gs://ssp-catalog-<env>/gcp.spanner-instance/1.2.0/
  module_source    text not null,                   -- gcs::https://.../ssp-module-spanner-1.2.0.zip
  spec             jsonb not null,                  -- ServiceSpec (validated)
  request_schema   jsonb not null,
  ui_template      jsonb not null,
  pricing          jsonb not null,
  policy_digest    text not null,                   -- sha256 of the Rego bundle
  published_at     timestamptz not null default now(),
  deprecated       boolean not null default false,
  primary key (service_id, version)
);

create table "order" (
  order_id         text primary key,                -- ord_ULID
  idempotency_key  text not null unique,
  kind             order_kind not null,
  state            order_state not null default 'requested',
  gate             approval_gate not null default 'none',
  tenant_id        text not null references tenant,
  service_id       text not null,
  service_version  text not null,
  resource_instance_id text,                        -- set for change/decommission/upgrade/adopt
  requested_by     text not null references principal,
  requested_params jsonb not null,                  -- schema-valid, no secrets
  preview_hash     text not null,
  channel          text not null,                   -- ui|api|chat|migrate|release
  created_at       timestamptz not null default now(),
  updated_at       timestamptz not null default now(),
  foreign key (service_id, service_version) references service_version
);
create index on "order" (tenant_id, state);
create index on "order" (requested_by, created_at desc);

create table approved_spec (
  order_id         text primary key references "order",
  approved_params  jsonb not null,                  -- requested + injected guardrails
  guardrails       jsonb not null,                  -- [{id, field, mode, value, source}]
  policy_decision  jsonb not null,                  -- {bundle_digest, result, rules:[...], warnings:[...]}
  estimate         jsonb not null,                  -- {monthly_usd, lines:[...], pricing_version}
  cr_id            text,                            -- ServiceNow CHG number when required
  owner_signoff_by text references principal,
  approved_by      text not null,                   -- 'system' or principal_id
  spec_hash        text not null,
  approved_at      timestamptz not null default now()
);

create table resource_instance (
  resource_instance_id text primary key,            -- ri_ULID
  tenant_id        text not null references tenant,
  service_id       text not null,
  service_version  text not null,                   -- module version currently applied
  name             text not null,                   -- stable name used as module block key
  params           jsonb not null,                  -- current approved params
  state            text not null default 'expected', -- expected|observed|drifted|decommissioning|removed
  outputs          jsonb,                           -- non-secret outputs
  last_observed_at timestamptz,
  created_by_order text references "order",
  created_at       timestamptz not null default now(),
  unique (tenant_id, name)
);
```

```
create table tenant_generation (
  tenant_id         text references tenant,
  generation        bigint not null,
  expected_uri      text not null,                    -- gs://.../<tenant>/expected/<generation>/
  spec_hashes       text[] not null,                  -- approved specs included
  created_at        timestamptz not null default now(),
  primary key (tenant_id, generation)
);

create table run (
  run_id            text primary key,                 -- run_ULID
  tenant_id         text not null references tenant,
  generation        bigint not null,
  kind              run_kind not null,
  state             run_state not null default 'queued',
  attempt           int not null default 1,
  job_name          text not null,                    -- tf-<hash>
  workflow_id       text not null,
  order_ids         text[] not null,
  plan_summary      jsonb,                            -- {add, change, destroy, replace, resources:[...]}
  started_at        timestamptz,
  finished_at       timestamptz,
  error             text,
  artifacts_uri     text                              -- gs://.../<tenant>/runs/<run_id>/
);
create index on run (tenant_id, started_at desc);

create table order_event (
  event_id          bigserial primary key,
  order_id          text not null references "order",
  at                timestamptz not null default now(),
  actor             text not null,                    -- principal_id | system | workflow:<id>
  from_state        order_state,
  to_state          order_state,
  type              text not null,                    -- created|approved|gate_waiting|gate_cleared|run_started|run_fi ↩
   nished|post_build_step|delivered|failed|...
  detail            jsonb
);
create index on order_event (order_id, event_id);

create table drift_finding (
  finding_id        text primary key,
  tenant_id         text not null references tenant,
  resource_instance_id text references resource_instance,
  class             drift_class not null,
  detail            jsonb not null,
  detected_at       timestamptz not null default now(),
  resolved_at       timestamptz,
  resolution        text                              -- reconciled|adopted|accepted|removed
);

create table control_flag (
  scope             text primary key,                 -- 'global' | 'lob:<id>' | 'tenant:<id>'
  paused            boolean not null default false,
  reason            text,
  set_by            text not null,
  set_at            timestamptz not null default now()
);

create table audit_event (
  audit_id          bigserial primary key,
  at                timestamptz not null default now(),
  actor             text not null,
  action            text not null,
  subject_type      text not null,
  subject_id        text not null,
  before_hash       text,
  after_hash        text,
  detail            jsonb,
  request_id        text
);
create index on audit_event (subject_type, subject_id, at desc);
```

3. Invariants

- SSP-DM-001 `order.state` transitions only through the API/workflow functions `TransitionOrder(order_id, to, reason)`, which inserts the `order_event` in the same transaction.

- SSP-DM-002 `approved_spec.approved_params` is write-once.
- SSP-DM-003 `resource_instance.name` is the module block key in the rendered root module; it never changes after creation (renames are new instances).
- SSP-DM-004 `tenant_generation` is monotonically increasing per tenant; the reconciler holds an advisory lock `pg_advisory_xact_lock(hashtext(tenant_id))` while allocating a generation.
- SSP-DM-005 No column anywhere may hold a secret value; the `secret_scan` CI gate greps migrations and fixtures for forbidden patterns.
- SSP-DM-006 Row counts: designed for 10,000 tenants, 1,000,000 orders, 5,000,000 events; partition `order_event` and `audit_event` by month from day one.

4. Object layout in the LOB state bucket

```
gs://ssp-state-<lob>/
└── <tenant boundary ref>/                    e.g. pay-prod-7a1c/
    ├── terraform.tfstate                     Observed (Terraform-managed; versioning on)
    ├── .terraform.lock.hcl                   provider lock (copied in by the runner)
    ├── expected/<generation>/
    │   ├── main.tf.json                      rendered root module
    │   ├── variables.auto.tfvars.json        non-secret inputs
    │   ├── backend.hcl                       backend config used
    │   └── manifest.json                     spec hashes, module versions, rendered_at
    └── runs/<run_id>/
        ├── plan.json                         terraform show -json
        ├── plan.txt                          human plan
        ├── policy.json                       plan-stage OPA result
        ├── apply.log                         redacted
        └── result.json                       {state, summary, outputs(non-secret), finished_at}
```

IAM: tenant SA has `objectAdmin` on `resource.name.startsWith("projects/_/buckets/ssp-state-<lob>/objects/<tenant>/")` only; bucket default CMEK = LOB key; uniform bucket-level access; versioning on; soft delete 30 days; no public access.

12 — Control-Plane API Specification (Go, `ssp-cp`)

OpenAPI 3.1 contract: `12-openapi.yaml` (same folder). The Go server is generated from it; the contract is edited first, never the handlers.

1. Stack (binding)

Concern	Choice	Notes
Language	Go 1.23+	Single module `github.com/vaishnavi4068/ssp-cp`
HTTP	`net/http` + `chi` router	Middleware chain: request id, OTel, auth, entitlement, rate limit
Codegen	`oapi-codegen` (strict server + types)	`make generate` regenerates `internal/api/gen`
DB	`pgx/v5` + `sqlc`; migrations with `goose`	`internal/store`
Workflows	Temporal Go SDK (`go.temporal.io/sdk`)	Namespace `ssp`; task queues in `13-workflows-spec.md`
Policy	OPA embedded (`github.com/open-policy-agent/opa/rego`) for request stage; bundles loaded from the catalog bucket	`internal/policy`
Validation	`santhosh-tekuri/jsonschema/v6` against the service request schema	`internal/catalog`
Config	env vars only; no config files in images	SSP_* prefix
Logging / tracing	`log/slog` JSON + OpenTelemetry (OTLP to the cluster collector)	
Auth	JWT (JWKS from IdP via gateway) + Workload Identity for internal callers	`internal/auth`

SSP-API-001 All endpoints live under `/v1`. SSP-API-002 Every mutating endpoint requires `Idempotency-Key`; replays return the original response with `Idempotent-Replayed: true`. SSP-API-003 Errors use RFC 9457 problem details with a stable `type` URI per error code (`https://ssp/errors/<code>`). SSP-API-004 All list endpoints paginate with `page_token` / `page_size` (max 200). SSP-API-005 Every response carries `X-Request-Id`; every audit event stores it.

2. Endpoint catalogue

Catalog

Method	Path	Persona	Purpose
GET	`/v1/catalog/services`	any	List enabled services; filter `cloud`, `category`, `q`; includes "frequent for you" ordering when `sort=frequent`
GET	`/v1/catalog/services/{service_id}`	any	Latest version: ServiceSpec summary, request schema, UI template, guardrails preview, pricing, docs link
GET	`/v1/catalog/services/{service_id}/versions/{version}`	any	A specific bundle version

Destinations and context

GET	`/v1/destinations`	**buyer**	**Tenant boundaries the caller is entitled to, grouped by cloud, with APM, tier, data class, env**
GET	`/v1/destinations/{tenant_id}/context`	buyer	The context object used for defaults: apm, tier, data_class, lob, quotas, budget headroom

Orders

POST	`/v1/orders/preflight`	**buyer**	**Body:** `{tenant_id, service_id, version?, kind, resource_instance_id?, params}` → `PreflightResult` **(approved_params, guardrails, policy, estimate, code{terraform, cli}, gates, preview_hash, valid_until)**
POST	`/v1/orders`	buyer	Body: preflight input + `preview_hash` → 202 `Order` (state `requested`); 409 `preview_stale` if hash differs; 422 validation; 403 entitlement
GET	`/v1/orders`	buyer/operator	Filter `tenant_id`, `state`, `mine=true`
GET	`/v1/orders/{order_id}`	buyer/operator	Order + approved spec (if any) + latest run summary
GET	`/v1/orders/{order_id}/events`	buyer/operator	SSE stream of `order_events` (`Last-Event-ID` resume)
GET	`/v1/orders/{order_id}/code`	buyer	`{terraform, cli, module_source, version}` equivalent code (D-17)
POST	`/v1/orders/{order_id}/cancel`	buyer	Allowed in `requested`/`approved` before a run starts
POST	`/v1/orders/{order_id}/signoff`	app_owner	Clears `owner_signoff_pending`

Resources

GET	`/v1/tenants/{tenant_id}/resources`	**buyer/operator**	**Resource instances with state, outputs, last observed**
GET	`/v1/resources/{resource_instance_id}`	buyer/operator	Detail incl. drift findings, orders history, code
GET	`/v1/tenants/{tenant_id}/drift`	buyer/operator	Open drift findings

Runs (read for buyers, write for operators)

GET	`/v1/runs?tenant_id=`	**buyer/operator**	**Runs for a tenant**
GET	`/v1/runs/{run_id}`	buyer/operator	Run detail incl. plan summary
GET	`/v1/runs/{run_id}/logs`	buyer/operator	Redacted log (text/plain, range requests)
POST	`/v1/runs/{run_id}/retry`	operator	Signals the reconciler to retry the failed generation

Operator controls (control room)

GET	`/v1/ops/fleet`	**operator**	**Counts by state, queue depth and age per LOB, worker health, Temporal namespace health, kill-switch status**		
GET	`/v1/ops/tenants?lob=&paused=&drifted=`	operator	Tenant table		
POST	`/v1/ops/controls/pause`	operator	`{scope: global	lob:<id>	tenant:<id>, reason}`
POST	`/v1/ops/controls/resume`	operator	same		
POST	`/v1/ops/controls/killswitch`	operator (2-person)	Disables the Platform SA via IAM; requires `confirm: "<scope>"` and a second approver token		
POST	`/v1/ops/tenants/{tenant_id}/reconcile`	operator	Forces a reconcile run		
POST	`/v1/ops/tenants/{tenant_id}/drift-scan`	operator	Forces a drift scan		
POST	`/v1/ops/tenants/{tenant_id}/break-glass`	operator	`{until, incident}`		
GET	`/v1/ops/releases` / POST `/v1/ops/releases/{service_id}/{version}/waves`	operator	Module upgrade waves		
GET	`/v1/ops/policy/impact?bundle=`	security	Policy-impact replay result		

Migration (used by `ssp-migrate`)

POST	`/v1/imports`	**operator**	**`{tenant_id	legacy_project, scope}` → import job (inventory)**
GET	`/v1/imports/{import_id}`	operator	Inventory, mapping status, dry-run result
PUT	`/v1/imports/{import_id}/mapping`	operator	Resource → service mapping decisions
POST	`/v1/imports/{import_id}/dry-run`	operator	Generates `import` blocks, runs plan, expects zero changes
POST	`/v1/imports/{import_id}/execute`	operator	Executes the import run; creates resource instances and orders of kind `import`

Assistant (Sidekick and operator chat)

POST	`/v1/assistant/conversations`	**any**	**New conversation bound to the caller and an optional tenant**
POST	`/v1/assistant/conversations/{id}/messages`	any	SSE: tokens, tool events, cards (`preflight_card`, `status_card`, `citation`)
GET	`/v1/assistant/conversations/{id}`	any	Transcript

Catalog admin and health

POST	`/v1/admin/catalog/reload`	**operator**	**Re-read `index.json` from the catalog bucket (also triggered by Pub/Sub)**
GET	`/healthz`, `/readyz`, `/metrics`	internal	Liveness, readiness (DB + Temporal + catalog loaded), Prometheus

3. Behavioural requirements

- SSP-API-010 `preflight` is pure: no writes, deterministic for the same input and catalog version; result cached 60 s by `preview_hash`.
- SSP-API-011 `preview_hash = sha256(canonical_json(tenant_id, service_id, version, kind, resource_instance_id, approved_params, guardrails, policy.bundle_digest, pricing_version))`.
- SSP-API-012 `POST /orders` validates entitlement, re-runs preflight server-side, compares hashes, inserts order + event in one transaction, then starts `OrderWorkflow(order_id)` with `WorkflowIDReusePolicy: RejectDuplicate` using workflow id `order:<order_id>`.
- SSP-API-013 Ops endpoints require persona `operator`; `killswitch` requires two distinct operators within 10 minutes (second call carries `approval_token` from the first).
- SSP-API-014 Logs endpoint streams from the run artifact in GCS; secrets are redacted by the runner before upload, and the API applies a second redaction pass (regex set from `policy_bundle`).
- SSP-API-015 Rate limits: 60 preflights/min per principal, 10 orders/min per principal, 600 reads/min.
- SSP-API-016 The `code` endpoint renders exactly the module block the reconciler would render (same renderer package), plus a `gcloud/aws/az` CLI approximation labelled "approximate".
- SSP-API-017 All timestamps RFC 3339 UTC; money as `{currency: "USD", amount: "123.45"}` strings.

4. Package layout (`ssp-cp`)

```
cmd/api/main.go                  HTTP server
cmd/worker/main.go               Temporal worker for workflows + control-plane activities (not Terraform)
cmd/catalogctl/main.go           validate/publish catalog bundles
internal/api/gen/                oapi-codegen output (do not edit)
internal/api/handlers/           strict-server implementations per tag
internal/auth/                   JWT, personas, entitlements
internal/catalog/                bundle loader, schema validation, derive rules, pricing
internal/policy/                 OPA request-stage evaluation, bundle cache, impact replay
internal/render/                 desired set → main.tf.json, backend.hcl, code view
internal/store/                  sqlc queries, migrations
internal/workflows/              OrderWorkflow, TenantReconciler, DriftScan, PostBuild, Import, Release
internal/activities/             control-plane activities (DB, GCS, ServiceNow, CMDB clients)
internal/assistant/              gateway to CLM + tool-calling model; tools = this API
internal/events/                 SSE hub backed by Postgres LISTEN/NOTIFY
pkg/ssp/                         shared types used by ssp-workers and apps
```

13 — Temporal Workflow Specification

All workflows run in the platform's self-hosted Temporal (D-11), namespace `ssp`. Workflow code lives in `ssp-cp/internal/workflows`; control-plane activities in `ssp-cp/internal/activities`; Terraform and post-build activities in `ssp-workers`.

1. Namespaces, task queues, workers

Task queue	Worker binary	Repo	Activities / workflows	Scaling
`ssp-control`	`ssp-cp/cmd/worker`	ssp-cp	All workflows; DB, GCS, catalog, policy, ServiceNow, CMDB activities	HPA on CPU; 2–6 replicas
`tf-runs-<lob>` (one per LOB; `tf-runs-default` for POC)	`ssp-workers/cmd/tf-worker`	ssp-workers	`RunTerraform` (creates and watches K8s Jobs)	1–N replicas; `MaxConcurrentActivityExecutionSize` = 10 per replica; KEDA on task-queue backlog later
`post-build`	`ssp-workers/cmd/postbuild-worker`	ssp-workers	Integration activities (CMDB, backup, monitoring, notify, Infoblox)	HPA
`drift`	`ssp-workers/cmd/drift-worker`	ssp-workers	`RunTerraform(kind=refresh)` reuse + `InventoryAssets`	HPA

- SSP-WF-001 Namespace `ssp` with retention 30 days; `ssp-sandbox` for tests. Search attributes: `TenantId` (Keyword), `LobId` (Keyword), `OrderId` (Keyword), `RunKind` (Keyword), `State` (Keyword).
- SSP-WF-002 Workflow IDs are deterministic: `order:<order_id>`, `tenant:<tenant_id>` (reconciler), `drift:<tenant_id>`, `import:<import_id>`, `release:<service_id>:<version>`. Reuse policy `RejectDuplicate` for orders; reconcilers use `continue-as-new` every 500 cycles or 24 h.
- SSP-WF-003 Every activity has explicit `StartToCloseTimeout`, `HeartbeatTimeout` where > 1 min, and a retry policy with `NonRetryableErrorTypes` for validation and policy errors.
- SSP-WF-004 Workflows never read the DB directly; all I/O is in activities. Workflow inputs are IDs, never large payloads.

2. `OrderWorkflow(order_id)`

```
start → LoadOrder → Approve → [gate loop] → SignalReconciler(tenant_id, approved_spec) → await ResultSignal → Po ↵
   stBuild (child) → Finalize
```

Step	Activity / child	Timeout	Notes
LoadOrder	`LoadOrder`	10 s	Reads order + catalog version
Approve	`ComputeApproval`	60 s	Re-runs preflight logic server-side, writes `approved_spec`, transitions `requested→approved`, emits events. Deny → `rejected`, workflow completes.
Gate loop	signals `CRApproved`, `OwnerSignedOff`, `Cancel`; timer 14 d	—	If `gates` contains `cr_required`: `CreateChangeRequest` activity (ServiceNow), then wait for signal or `PollCR` every 10 min. Owner sign-off waits for signal from `POST /signoff`. Timeout → `cancelled` with reason.
Hand-off	`SignalWithStart TenantReconciler(tenant_id)` with `Enqueue{order_id, spec_hash, isolation}`	—	Transitions `approved→expected` when the reconciler acknowledges inclusion in a generation (signal `Included{generation}`).

Step	Activity / child	Timeout	Notes
Await result	signal `RunResult{run_id, outcome, outputs}`	6 h then escalate	On `failed`: order → `failed`, event with run error; operator may retry (reconciler re-queues).
PostBuild	child `PostBuildWorkflow(order_id)`	per step	Steps from ServiceSpec `postBuild`.
Finalize	`MarkObserved`	10 s	Upserts `resource_instance` (state observed), order → `observed`, notify buyer.

- SSP-WF-010 `Cancel` signal is honoured until `Included`; after that the order can only be reversed by a `decommission` order.
- SSP-WF-011 `ComputeApproval` is the only place that writes `approved_spec`; it is idempotent on `(order_id)`.

3. `TenantReconciler(tenant_id)` — entity workflow

Owns the single state file for a tenant boundary (D-08). One instance per tenant, started on first order, lives forever via continue-as-new.

```
loop:
  wait for Enqueue signals (or timer)                    ── batch window 20 s (SSP-PROD-052)
  if paused(global|lob|tenant) or platform SA disabled → wait, re-check every 60 s
  gen = AllocateGeneration(tenant_id, batch)             ── Postgres advisory lock; writes tenant_generation
  Render(tenant_id, gen)                                 ── desired set → expected/<gen>/ in LOB bucket
  signal each order: Included{gen}
  run = CreateRun(kind=plan)  →  RunTerraform(run)       ── on tf-runs-<lob>; Job tf-<hash>
  PlanGate(run)                                          ── only batch instances touched; plan-stage OPA allow; no u ↩
   ndeclared destroy/replace
  if gate fails → mark run failed, orders failed, alert; continue loop
  run2 = CreateRun(kind=apply) → RunTerraform(run2)      ── applies the saved plan file from the plan run
  CaptureObserved(run2)                                  ── parse state → resource_instance outputs, last_observed_a ↩
   t
  signal each order: RunResult{...}
  schedule DriftScan(tenant_id) in 10 min (debounced)
```

Activity	Queue	Timeout	Heartbeat	Idempotency
`AllocateGeneration`	ssp-control	30 s	—	`(tenant_id, spec_hashes)`
`Render`	ssp-control	2 min	—	content-addressed output; re-render is a no-op if identical
`CreateRun`	ssp-control	10 s	—	`(tenant_id, generation, kind, attempt)`
`RunTerraform`	tf-runs-<lob>	90 min (plan 30 min)	30 s	Job name `tf-<sha256(run_id:attempt)[:20]>`; re-attach if exists
`PlanGate`	ssp-control	1 min	—	pure
`CaptureObserved`	ssp-control	2 min	—	`(run_id)`

- SSP-WF-020 The reconciler is the only writer to a tenant's `expected/` and the only issuer of runs for that tenant; Terraform's own GCS lock is a second safety net, not the primary serialization.
- SSP-WF-021 A `decommission` or `upgrade` spec with `isolation: true` is never batched with other specs.
- SSP-WF-022 `apply` uses the plan file produced by the `plan` run (`terraform apply plan.tfplan`) so what was gated is what is applied; the plan file is stored in the run artifacts and is valid for 24 h.
- SSP-WF-023 If `RunTerraform` fails with a state lock error, the reconciler waits 2 min and retries up to 5 times before failing the generation and alerting (someone is holding the lock out of band).
- SSP-WF-024 Queue metrics (`ssp_reconciler_queue_depth`, `ssp_reconciler_queue_oldest_seconds`) are emitted per tenant and LOB via a `QueryHandler` polled by the metrics exporter.

4. `PostBuildWorkflow(order_id)`

- SSP-WF-030 Steps execute sequentially in ServiceSpec order; each step is an activity on `post-build` with `StartToCloseTimeout` 10 min, retry with exponential backoff up to 24 h, then `escalate` activity (ticket to the owning agent team) and the order stays `observed (post-build incomplete)`.
- SSP-WF-031 Standard steps and their activities: `cmdb.register` (create/update CI; receipt = CI sys_id), `backup.enroll`, `monitoring.onboard`, `notify.team` (email/Teams), `dns.register` (Infoblox), `firewall.request` (opens request; does not wait).
- SSP-WF-032 Steps declare `idempotencyKey` templates (e.g. `cmdb:<resource_instance_id>`); receipts are stored in `order_event.detail` and in `resource_instance.outputs.post_build`.

5. `DriftScanWorkflow(tenant_id)` (scheduled)

- SSP-WF-040 Temporal Schedule per tenant, every 6 h with jitter; also triggered 10 min after any apply. Skips if a run is in progress.
- SSP-WF-041 `RunTerraform(kind=refresh)` performs `terraform plan -refresh-only -json`; `ClassifyDrift` maps resource changes to `benign|config|missing` using per-service allowlists; `InventoryAssets` (Cloud Asset Inventory) finds `unmanaged` resources of catalog-known types in the boundary.
- SSP-WF-042 Findings are upserted (`drift_finding`), the resource marked `drifted`, notifications sent. Auto-revert only when ServiceSpec `drift.autoRevert: true` and no break-glass window: enqueue a `reconcile` to the TenantReconciler.

6. `ImportWorkflow(import_id)` (used by `ssp-migrate`)

```
Inventory → SuggestMapping → await MappingConfirmed → GenerateImportBlocks → DryRun (plan with import blocks; expe ↩
   ct 0 changes) → await ExecuteApproved → Execute (apply import) → CreateInstances → Done
```

- SSP-WF-050 Dry-run must show only `import` actions and zero `update/destroy`; otherwise the workflow stops in `mapping` with the diff for the operator.
- SSP-WF-051 Legacy 1.0 projects (multi-app) import into the single state of that project; each resource instance carries its own `apm_id` label from the mapping; the tenant's `apm_id` is the primary owner chosen by the operator.
- SSP-WF-052 Execution runs as an isolated generation on the TenantReconciler (so the lock discipline holds).

7. `ReleaseWorkflow(service_id, version)` (module upgrade waves)

- SSP-WF-060 Waves are defined by the operator: `canary` (named tenants), then LOBs in order, each with `soak` duration and `maxFailures`. For each tenant in a wave, create an `upgrade` order (channel `release`) and wait for Observed; stop the wave on `maxFailures`.
- SSP-WF-061 Upgrade orders carry `isolation: true` and are subject to the plan gate; any destroy/replace not declared in the module CHANGELOG fails the wave.

8. Signals, queries, errors

Workflow	Signals	Queries
OrderWorkflow	`CRApproved{cr_id}`, `OwnerSignedOff{by}`, `Cancel{by, reason}`, `Included{gen}`, `RunResult{...}`	`Status`
TenantReconciler	`Enqueue{order_id, spec_hash, isolation}`, `Pause`, `Resume`, `Retry{generation}`, `ForceReconcile`	`Queue`, `Current`
DriftScan	`RunNow`	`Last`
Import	`MappingConfirmed`, `ExecuteApproved`, `Abort`	`Status`
Release	`Advance`, `Halt`	`Progress`

- SSP-WF-070 Error taxonomy: `ValidationError`, `PolicyDenied`, `EntitlementError` (non-retryable); `TransientError` (retryable); `TerraformError{phase, exit_code, summary}` (retryable only for `init`/network classes); `LockError` (retry with backoff).
- SSP-WF-071 Replay tests: recorded histories for each workflow are stored in `ssp-cp/internal/workflows/testdata/histories/`; CI runs `replayer.ReplayWorkflowHistory` for all; any change that breaks replay must use `workflow.GetVersion`.

14 — Policy Specification (OPA)

New rules against the platform's own JSON schema (D-13). Two evaluation points: **request stage** (inside `ComputeApproval` and `preflight`) and **plan stage** (inside the Terraform runner on `plan.json`). Nothing from Taxi is reused.

1. Bundles

Bundle	Source repo	Scope	Loaded by
`platform`	`ssp-doc/10-product/policy/` (human-owned baseline) → built by `ssp-cp` CI	Global rules: labels, naming, allowed regions, data-class controls, budget	ssp-cp (embedded OPA), runner (CLI)
`service/<service_id>`	`ssp-module-<service>/policy/`	Service-specific rules (both stages)	Published inside the catalog bundle

- SSP-POL-001 Bundles are OCI/tarball artifacts in `gs://ssp-catalog-<env>/policy/` with a digest recorded in `approved_spec.policy_decision.bundle_digest` and in `run` artifacts.
- SSP-POL-002 The runner image pins the same OPA version as `ssp-cp` (`opa` 1.x; exact version in `30-infra/tf-worker/Dockerfile`).

2. Request-stage input document

```
{
  "stage": "request",
  "order": {"kind": "create", "service_id": "gcp.spanner-instance", "version": "1.2.0", "name": "claims-ledger"},
  "params": { "...approved_params after guardrail injection..." },
  "tenant": {"tenant_id": "gcp:pay-prod-7a1c", "cloud": "gcp", "env": "prod", "lob_id": "pbm",
             "apm_id": "APM-10432", "tier": 1, "data_class": "PHI", "cost_center": "CC-5531", "region_allowlist": ["us ↩
   -central1","us-east4"]},
  "principal": {"principal_id": "…", "personas": ["buyer"], "entitlements": ["APM-10432"]},
  "estimate": {"monthly_usd": 1234.56},
  "budget": {"monthly_limit_usd": 50000, "committed_usd": 41000},
  "catalog": {"guardrails": [ ... ]}
}
```

Decision document (`data.ssp.request.decision`):

```
{"result": "allow|deny|warn",
 "rules": [{"id": "ssp.request.labels.required", "result": "deny", "message": "label cost_center missing", "requiremen ↩
   t_id": "SSP-POL-110"}],
 "gates": ["cr_required"]}
```

3. Plan-stage input document

`terraform show -json plan.tfplan` plus context:

```
{"stage": "plan", "tenant": {...}, "generation": 42, "batch": {"resource_instances": ["claims-ledger"]},
 "plan": { "...terraform plan JSON..." }}
```

Decision `data.ssp.plan.decision` with the same shape; additionally `data.ssp.plan.touched` lists module addresses changed.

4. Baseline rules (platform bundle)

ID	Stage	Rule	Result
SSP-POL-100	request	Principal must be entitled to `tenant.apm_id`	deny
SSP-POL-101	request	Tenant not paused; global/LOB not paused	deny (`tenant_paused` gate)

ID	Stage	Rule	Result
SSP-POL-110	request	Labels `apm_id`, `cost_center`, `lob`, `env`, `data_class`, `managed_by=ssp` present (injected by guardrails; rule verifies)	deny
SSP-POL-111	request	Resource name matches `^[a-z][a-z0-9-]{2,40}$` and is unique in tenant (checked by API; rule re-asserts format)	deny
SSP-POL-120	request	Region ∈ `tenant.region_allowlist`	deny
SSP-POL-130	request	`data_class ∈ {PHI, PII}` ⇒ CMEK key set, public access disabled, retention ≥ class minimum	deny
SSP-POL-131	request	`tier == 1` and service category `database` ⇒ multi-region/HA flag true	warn if relaxed with sign-off, else deny
SSP-POL-140	request	`estimate.monthly_usd + budget.committed_usd ≤ budget.monthly_limit_usd`	deny (`budget_exceeded` gate)
SSP-POL-150	request	`env == prod` and `kind ∈ {change, decommission}` ⇒ gate `cr_required`	gate
SSP-POL-151	request	`kind == decommission` and service `lifecycle.holdsData` ⇒ gate `owner_signoff_required`	gate
SSP-POL-200	plan	No `delete` or `replace` action on a resource outside `batch.resource_instances`	deny
SSP-POL-201	plan	No change to `provider`/backend configuration; no `-target`	deny
SSP-POL-202	plan	No resource of a type outside the service's declared `resourceTypes`	deny
SSP-POL-203	plan	No IAM binding granting `roles/owner`, `roles/editor`, `allUsers`, `allAuthenticatedUsers`	deny
SSP-POL-204	plan	No public ingress (0.0.0.0/0) on firewall or LB resources unless service declares `publicIngress: true` and request-stage allowed it	deny
SSP-POL-205	plan	Every created resource carries the mandatory labels	deny
SSP-POL-206	plan	Sensitive attributes (`password`, `private_key`, `secret`) are unknown-after-apply or reference a secret manager resource, never literals	deny

5. Authoring rules (for `ssp-module-*` policies)

- SSP-POL-300 Package names: `ssp.request.svc.<service_slug>` and `ssp.plan.svc.<service_slug>`; each rule sets `id`, `message`, `requirement_id` (from the service ADR).
- SSP-POL-301 Every MUST guardrail in the ServiceSpec has at least one plan-stage rule verifying it on the plan JSON (defence in depth against module bugs).
- SSP-POL-302 Tests: `opa test` with ≥ 90 % coverage; fixtures include one allow, one deny per rule.
- SSP-POL-303 Lint with `regal`; no `http.send` in policies.

6. Policy-impact replay (gate)

- SSP-POL-400 CI for any bundle change replays the new bundle against the last 90 days of request inputs and plan JSONs (sanitized corpus in `gs://ssp-corpus-<env>/`) and reports newly-denied / newly-allowed counts and examples.
- SSP-POL-401 Any newly-allowed input marks the PR T3 (security review).

15 — Service Catalog and ServiceSpec

One ServiceSpec per service, owned by its `ssp-module-<service>` repo, released as a **catalog bundle**. The platform loads bundles at runtime, so adding a service is a publish, not a redeploy (D-17, SSP-PRIN-007).

1. Catalog bundle layout (published to `gs://ssp-catalog-<env>/<service_id>/<version>/`)

```
service.yaml                ServiceSpec (this document)
request.schema.json         JSON Schema 2020-12 for buyer-settable params
ui.yaml                     step layout, field order, help, widgets (consumed by ssp-ui renderer)
pricing.yaml                unit rates + formula for the estimate
policy/*.rego  policy/*_test.rego   service rules (request and plan stage)
module.zip                  the Terraform module at this version (mirrored from the repo tag)
docs/README.md              buyer-facing docs (rendered on the SKU page)
CHANGELOG.md
bundle.json                 {service_id, version, module_source, digests{...}, published_at, adr_id}
```

`gs://ssp-catalog-<env>/index.json` lists all services and versions; `ssp-cp` reloads it on Pub/Sub notification (`ssp-catalog-updates`) or `POST /v1/admin/catalog/reload`.

2. ServiceSpec — worked example `gcp.spanner-instance`

```
apiVersion: ssp.catalog/v1
kind: ServiceSpec
metadata:
  id: gcp.spanner-instance
  version: 1.2.0
  displayName: Cloud Spanner instance
  category: database
  cloud: gcp
  adr: E-SPAN-0001
  ownerMailbox: ssp-databases@example.com
  tags: [database, relational, global]
spec:
  tenantBoundary: project
  resourceTypes: [google_spanner_instance, google_spanner_database, google_spanner_instance_iam_member, google_kms_cry ↩
   pto_key_iam_member]
  request:
    schema: request.schema.json
    nameField: name                       # stable name; also the Terraform module block key
  context:                                # what the platform supplies to derive rules, policy and rendering
    needs: [apm.tier, apm.data_class, tenant.region_allowlist, lob.cmek_key, tenant.cost_center]
  derive:                                 # evaluated before the form is shown (SSP-PROD-030)
    - field: /config/mode
      when: "apm.tier == 1"
      value: multi-region
      reason: "Tier 1 databases are multi-region (G-0007 §3)"
      requirementId: REQ-E-SPAN-0001-04
    - field: /encryption/kmsKeyName
      value: "{{ lob.cmek_key }}"
      requirementId: REQ-E-SPAN-0001-02
  guardrails:                             # applied at approval (SSP-PROD-020); mode inject|deny|warn
    - id: REQ-E-SPAN-0001-02
      field: /encryption/kmsKeyName
      mode: inject
      value: "{{ lob.cmek_key }}"
    - id: REQ-E-SPAN-0001-05
      field: /labels
      mode: inject
      value: { apm_id: "{{ tenant.apm_id }}", cost_center: "{{ tenant.cost_center }}", lob: "{{ tenant.lob_id }}", env ↩
   : "{{ tenant.env }}", data_class: "{{ apm.data_class }}", managed_by: ssp }
    - id: REQ-E-SPAN-0001-06
      field: /config/processingUnits
      mode: deny
      when: "params.config.processingUnits < 100 || params.config.processingUnits % 100 != 0"
      message: "Processing units must be a multiple of 100 (minimum 100)"
    - id: REQ-E-SPAN-0001-07
      field: /deletionProtection
      mode: inject
      when: "tenant.env == 'prod'"
      value: true
```

```
    relaxable: false
terraform:
  module: ssp-module-spanner
  source: "gcs::https://www.googleapis.com/storage/v1/ssp-modules-{{ env }}/ssp-module-spanner/1.2.0/module.zip"
  versionConstraint: "~> 1.2"
  inputsFrom: /                         # approved params map 1:1 to module variables (renderer enforces)
  outputs: [instance_id, instance_name, databases, connection_hint]   # non-secret only
  providers: { google: ">= 6.0, < 7.0" }
secrets: []                             # none for Spanner; services with secrets list {field, generate|reference, s ↩
 ecretManagerPath}
pricing: pricing.yaml
ui: ui.yaml
postBuild:
  - step: cmdb.register
    ciClass: cmdb_ci_cloud_database
  - step: monitoring.onboard
  - step: notify.team
lifecycle:
  holdsData: true
  ownerSignoff: { change: false, decommission: true }
  delete: { mode: soft, retentionDays: 30 }    # soft = enable deletion protection + keep 30d, then remove
  drift: { autoRevert: false, benignAttributes: [labels.last_scan] }
sla: { provisionP95Minutes: 15 }
docs: docs/README.md
```

3. request.schema.json (excerpt)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://ssp/catalog/gcp.spanner-instance/1.2.0/request.schema.json",
  "type": "object",
  "required": ["name", "config", "databases"],
  "additionalProperties": false,
  "properties": {
    "name": {"type": "string", "pattern": "^[a-z][a-z0-9-]{2,28}$", "title": "Instance name"},
    "config": {
      "type": "object", "required": ["mode", "processingUnits"],
      "properties": {
        "mode": {"type": "string", "enum": ["regional", "multi-region"], "title": "Configuration"},
        "region": {"type": "string", "title": "Region", "x-ssp-enumFrom": "tenant.region_allowlist"},
        "multiRegionConfig": {"type": "string", "enum": ["nam3", "nam6", "nam-eur-asia1"], "title": "Multi-region conf ↩
 ig"},
        "processingUnits": {"type": "integer", "minimum": 100, "maximum": 10000, "multipleOf": 100, "default": 100}
      },
      "allOf": [
        {"if": {"properties": {"mode": {"const": "regional"}}}, "then": {"required": ["region"]}},
        {"if": {"properties": {"mode": {"const": "multi-region"}}}, "then": {"required": ["multiRegionConfig"]}}
      ]
    },
    "databases": {
      "type": "array", "minItems": 1, "maxItems": 20,
      "items": {"type": "object", "required": ["name", "dialect"],
        "properties": {"name": {"type": "string", "pattern": "^[a-z][a-z0-9_-]{1,29}$"},
                       "dialect": {"type": "string", "enum": ["GOOGLE_STANDARD_SQL", "POSTGRESQL"], "default": "GOOGLE ↩
 _STANDARD_SQL"},
                       "versionRetentionPeriod": {"type": "string", "default": "7d"}}}
    },
    "encryption": {"type": "object", "properties": {"kmsKeyName": {"type": "string", "readOnly": true}}, "x-ssp-derive ↩
 d": true},
    "deletionProtection": {"type": "boolean", "default": true},
    "labels": {"type": "object", "additionalProperties": {"type": "string"}, "x-ssp-derived": true},
    "ops": {"type": "object", "x-ssp-persona": "operator", "properties": {"forceDestroy": {"type": "boolean", "default ↩
 ": false}}}
  }
}
```

Vendor extensions: `x-ssp-derived` (shown read-only with reason), `x-ssp-persona` (field visible/settable only for that persona; SSP-PROD-101), `x-ssp-enumFrom` (options from context), `x-ssp-secret` (accept only `secretmanager://` references), `x-ssp-widget` (UI hint).

4. ui.yaml (excerpt)

```
steps:
  - id: basics
    title: Basics
```

```
    description: Name the instance and choose where it runs.
    fields: [/name, /config/mode, /config/region, /config/multiRegionConfig]
  - id: capacity
    title: Capacity
    fields: [/config/processingUnits]
    summary: "{{ params.config.processingUnits }} PU · {{ params.config.mode }}"
  - id: databases
    title: Databases
    fields: [/databases]
    widget: { /databases: table }
  - id: protection
    title: Protection & encryption
    fields: [/encryption/kmsKeyName, /deletionProtection]
  - id: review
    title: Review
receipt:
  title: Infrastructure Order Summary
  lines: [compute, storage, backup]        # keys from pricing.yaml
  showCode: true                           # View as code tab (D-17)
frequentPresets:
  - { label: "Dev regional 100 PU", params: { config: { mode: regional, processingUnits: 100 } } }
  - { label: "Prod multi-region 1000 PU", params: { config: { mode: multi-region, multiRegionConfig: nam3, processingU ↩
   nits: 1000 } } }
```

5. `pricing.yaml` (excerpt)

```
version: 2026-10
currency: USD
lines:
  compute: { label: "Processing units", unit: "100 PU-month", quantity: "params.config.processingUnits / 100",
             rate: { regional: 65.70, multi-region: 270.0 }, rateKey: "params.config.mode" }
  storage: { label: "Storage (est. 100 GB)", unit: "GB-month", quantity: "100", rate: 0.30 }
  backup:  { label: "Backup (est. 100 GB)", unit: "GB-month", quantity: "100", rate: 0.10 }
disclaimer: "Estimate from list prices; actuals billed by cloud provider to cost center."
```

6. Validation rules

- SSP-CAT-001 `service.yaml` validates against `15-service-spec.schema.json`; `request.schema.json` compiles; every `derive`/`guardrail` field exists in the schema; every `ui.yaml` field exists; every `pricing` expression references existing params.
- SSP-CAT-002 `resourceTypes` is exhaustive: the plan-stage rule SSP-POL-202 denies anything else.
- SSP-CAT-003 Module variables = request schema properties (names and types) plus platform-injected `labels`, `project_id`, `kms_key_name` where declared; `catalogctl validate` runs `terraform-config-inspect` to enforce.
- SSP-CAT-004 A bundle version is immutable; fixes are new versions. Deprecation sets `deprecated: true` and a `successor` version; buyers see the latest non-deprecated version.
- SSP-CAT-005 Secrets: if `spec.secrets` is non-empty, each entry is `{field, mode: generate|reference, path: "secretmanager://projects/{{tenant.project}}/secrets/<name>"}`; the renderer passes only the reference to Terraform (`ephemeral`/write-only attributes where the provider supports them).

16 — Storefront UI Specification (`ssp-ui`)

Visual and interaction reference: prototype **Gemini 4 Argon / shadcn-stack / #17 "Shopify Polaris Infrastructure Storefront"** in the TAXI prototype gallery (Next.js + Tailwind + shadcn). This spec turns that prototype into a template-driven product: every service screen is rendered from the catalog bundle, never hand-coded (D-17, SSP-PRIN-007).

1. Stack (binding)

Concern	Choice
Framework	Next.js 15 (App Router), TypeScript strict, React 19
Styling	Tailwind CSS v4 + shadcn/ui (Radix primitives); CSS variables for tokens; dark mode via `class`
Data	TanStack Query for REST; native `EventSource` for SSE; no global state library
Forms	Schema renderer (own package `@ssp/form-renderer`) driven by `request.schema.json` + `ui.yaml`; validation with Ajv 2020 in the browser, authoritative validation server-side (preflight)
Auth	Auth.js (OIDC) → session holds the API JWT; API calls go through a Next route handler proxy that attaches the token (browser never sees the raw token)
Icons / font	lucide-react; Inter (UI) + JetBrains Mono (ids, prices, code)
Tests	Vitest + Testing Library; Playwright e2e against the twin-backed sandbox API; Storybook for the renderer
Build	Standalone output in a distroless image; deployed to GKE namespace `ssp-ui` behind the internal gateway

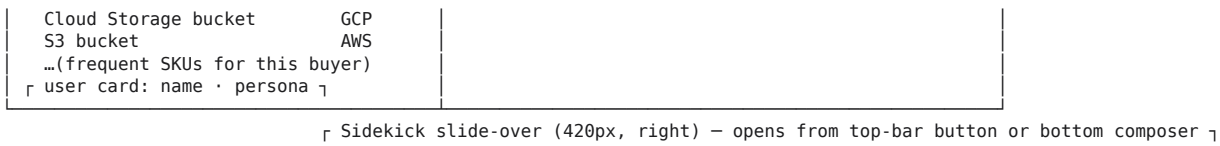
2. Design tokens (derived from the prototype)

```
:root {
  --background: 220 20% 97%;        /* page: soft gray (#f5f6f9) */
  --card: 0 0% 100%;                /* white cards */
  --border: 220 14% 90%;            /* #e3e6eb hairlines */
  --foreground: 222 20% 12%;        /* #1a1d23 text */
  --muted-foreground: 220 9% 46%;
  --primary: 161 62% 28%;           /* storefront green #1b7a5a (buttons, active nav, section labels) */
  --primary-foreground: 0 0% 100%;
  --accent: 161 50% 94%;            /* pale green chips / selected radio card */
  --warning: 38 92% 50%;
  --destructive: 0 72% 51%;
  --ring: 161 62% 28%;
  --radius: 0.75rem;                /* 12px cards, 8px inputs, 999px pills */
}
.dark { --background: 222 18% 9%; --card: 222 16% 12%; --border: 222 12% 20%; --foreground: 220 20% 95%; --primary: 16 ↩
   1 55% 45%; --accent: 161 30% 18%; }
```

Typography: Inter 14 px base, 24–28 px page titles (semibold), 11 px uppercase tracking-wide section labels in `--primary` ("DELIVERY DESTINATIONS", "ITEMIZED MONTHLY CHARGES"), JetBrains Mono for identifiers, field keys, prices and code. Spacing on an 8 px grid; card padding 20–24 px; max content width 1280 px.

3. Application shell

```
┌ top bar (56px) ──────────────────────────────────────────────────────────────────────────┐
│ [menu]  SSP Cloud Store   [ Delivery destination ▾ ]   [Search SKUs, destinations, orders ⌘K ]   [Sidekick]  [Orders ↩
   (n)]  [theme]  [sign out] │
├ left nav (240px, collapsible) ──────┬──────────────────────────────────────────────────────┤
│ STOREFRONT DEPARTMENTS               │                                                      │
│ ▸ Storefront Home                    │   main content (cards on soft-gray page)             │
│   All Cloud SKUs (n)                 │                                                      │
│   Fulfilled Orders                   │                                                      │
│   Saved Drafts                       │                                                      │
│   Recently Viewed                    │                                                      │
│   Buyer Settings                     │                                                      │
│ DIRECT SKU CHECKOUT                  │                                                      │
│   Spanner instance           GCP     │                                                      │
```



- SSP-UI-001 Search is top-center, always visible, opens a command palette (⌘K / Ctrl-K) that searches SKUs, destinations, orders, resources and docs; results grouped; Enter opens, ⌘Enter opens Configure & Buy for a SKU (D-17 "search top-center").
- SSP-UI-002 Sidekick is reachable two ways: the top-bar button, and a **docked composer bar at the bottom of the main content** ("Ask Sidekick to configure a SKU…") that expands into the right slide-over on focus. This satisfies "chat at the bottom" and keeps the prototype's slide-over.
- SSP-UI-003 The left nav's "Direct SKU checkout" list is the catalog of frequent items: top 8 SKUs by the buyer's own order history, then LOB popularity, with cloud tag at right.
- SSP-UI-004 The delivery-destination selector in the top bar scopes the whole UI (home, orders, resources) to one tenant boundary or "All destinations".
- SSP-UI-005 Dark mode toggle persists per user; all tokens defined for both themes; contrast AA.

4. Screens

4.1 Storefront Home (`/`)

- Hero card: pill "Platform Authorized Storefront", title "Self-Service Cloud Infrastructure Store", one line of copy, buttons **Ask Sidekick Concierge** (outline) and **Shop All SKUs** (primary).
- "Delivery destinations" card: tabs per cloud with counts (AWS (6) · GCP (6) · Azure (4)); grid of destination cards: cloud tag, region, name, boundary id (mono), buyer role, **Select →**.
- SKU grid: cards with tag pill (cloud · category), wishlist star, name, one-line description, "Starting from $X/mo" (mono, primary), **Configure & Buy**.
- Two-column footer row: "Recently viewed items" (name, cloud, timestamp) and "Saved drafts" chips.
- SSP-UI-010 Everything on Home comes from `/catalog/services?sort=frequent`, `/destinations`, and local recently-viewed; no service-specific code.

4.2 All Cloud SKUs (`/skus`)

Filter rail (cloud, category, tags, data-class compatible), sort, grid/list toggle, same SKU card. SKU detail (`/skus/[service_id]`): docs from the bundle, guardrails preview ("what the platform enforces for your destination"), pricing table, frequent presets, **Configure & Buy**, "View module source" (version + ADR link).

4.3 Configure & Checkout (`/checkout/[service_id]?dest=<tenant_id>`)

Two-column: left the stepper and step cards; right the sticky **Infrastructure Order Summary** receipt.

- Header: back, SKU pill (`SKU: GCP-SPANNER · gcp`), title "Configure & Checkout: Cloud Spanner instance", **Delivery destination** select.
- Stepper from `ui.yaml.steps` (numbered chips; completed = filled). Step card header "CHECKOUT STEP n OF m" (section label style), title, description.
- Fields numbered `n.k` with the schema key shown in mono at right (as in the prototype: `1.1 Subscription subscription`). Widgets by schema type and `x-ssp-widget`: text, select, radio-cards (enum with descriptions), number with unit, toggle, table (array of objects), key-value (labels), read-only derived field with reason link.
- Derived/injected fields render read-only with a small "Set by REQ-E-SPAN-0001-02 · Tier 1 / PHI" caption (SSP-PROD-031).
- Receipt: SKU, destination, "In stock"/"Needs CR"/"Needs owner sign-off" pill; ITEMIZED MONTHLY CHARGES from `estimate.lines`; Estimated Monthly Total (large, mono); "Billed to cost center CC-…"; guardrails applied (count, expandable); policy warnings; tabs **Summary | Code** where Code shows the Terraform module block and CLI (SSP-PROD-042, D-17).

- Primary action **Place order** (disabled until preflight valid); secondary **Save draft**, **Ask Sidekick about this**.
- SSP-UI-020 Every change debounces 400 ms then calls `POST /orders/preflight`; the receipt shows a subtle "updating" state and never displays client-computed money.
- SSP-UI-021 `Place order` sends `preview_hash`; on 409 the UI refreshes the receipt and asks the buyer to confirm again.
- SSP-UI-022 Unknown-service fixture: the checkout must render a never-seen ServiceSpec from the sandbox catalog with zero code changes (CI test).

4.4 Order tracking (`/orders/[order_id]`)

Timeline of the four states with timestamps; current gate (CR number with link, sign-off requester); run panel (plan summary +N ~M −K, live redacted log tail via SSE, link to run); post-build checklist with receipts; ETA from `eta_seconds_p50`; actions: Cancel (when allowed), View code, Open resource.

4.5 Fulfilled Orders (`/orders`) and Resources (`/resources`)

Data tables (shadcn `DataTable`): filter by destination, service, state; row click → detail. Resource detail: observed attributes (from outputs), drift findings with class badges, history of orders, actions **Change**, **Decommission**, **Re-run post-build**, **View code**.

4.6 Saved Drafts, Recently Viewed, Buyer Settings

Drafts = serialized `OrderInput` stored server-side per user (`/v1/drafts`, Phase 1.1; local storage until then). Settings: notification channels, default destination, theme.

4.7 Sidekick slide-over

Header "Sidekick Cloud Concierge — Instant SKU configuration & order prefill", reset and close. "Express SKU bundles" chips (frequent presets). Message list; assistant messages may carry cards: **Preflight card** (mini receipt + "Open in checkout" + "Place order"), **Status card**, **Citation chips** (ADR / requirement id → docs). Composer at bottom with send. See `17-conversational-provisioning-spec.md` for behaviour; SSP-UI-030 the slide-over never shows a "Place order" button unless the card came from a server preflight with a `preview_hash`.

5. Template-driven micro-frontend

- SSP-UI-040 `@ssp/form-renderer` takes `{requestSchema, uiTemplate, context, value}` and renders steps, fields and receipt bindings; it has no knowledge of any service.
- SSP-UI-041 Templates are fetched from the API (which reads the catalog bundle) at request time and cached by `(service_id, version)`; a new catalog publish is visible without a deploy.
- SSP-UI-042 Service-specific visuals (icons, hero images) are referenced by URL inside `ui.yaml`, served from the catalog bucket through the API.
- SSP-UI-043 Renderer contract is versioned (`ui.yaml` → `rendererVersion: 1`); unknown widgets fall back to a generic JSON field with a warning badge, never a crash.

6. Accessibility and quality

- SSP-UI-050 All interactive elements keyboard reachable; focus visible; radio-cards are real radios; tables have headers; color never the only signal (drift badges carry text).
- SSP-UI-051 Lighthouse a11y ≥ 95 and performance ≥ 85 on Home and Checkout in CI.
- SSP-UI-052 Visual regression (Playwright screenshots) for Home, SKU, Checkout steps, Receipt, Sidekick, in light and dark.

7. Repo structure (`ssp-ui`)

```
app/(shell)/layout.tsx          top bar, left nav, bottom composer, Sidekick slide-over
```

```
app/page.tsx                      Storefront Home
app/skus/page.tsx, app/skus/[serviceId]/page.tsx
app/checkout/[serviceId]/page.tsx
app/orders/page.tsx, app/orders/[orderId]/page.tsx
app/resources/page.tsx, app/resources/[id]/page.tsx
app/api/ssp/[...path]/route.ts    token-attaching proxy to ssp-cp
packages/form-renderer/           schema → steps/fields/receipt (published as @ssp/form-renderer)
packages/ui/                      shadcn components + tokens
lib/api/                          generated TS client from 12-openapi.yaml (openapi-typescript)
lib/sse.ts, lib/auth.ts
tests/e2e/, tests/fixtures/catalog/unknown-service/
```

17 — Conversational Provisioning (Sidekick) Specification

Sidekick is the buyer-facing assistant in `ssp-ui`. It is built on a **Contextual Language Model** for the knowledge half (grounded answers about standards, guardrails, denials, status) and on schema-constrained tool calling plus deterministic platform code for the action half (D-20). The operator-facing chat in the control room reuses the same gateway with a different tool set (`20-apps/23-operator-agent-chat-spec.md`).

1. Two halves, one rule

Half	Examples	Engine	May it change state?
Knowing	"What's the standard for a Tier 1 Spanner?", "Why was CMEK forced?", "Why did my order fail?", "What do I have in pay-prod?"	CLM (retriever + generator trained together) over the knowledge index	No
Acting	"Create a regional Spanner with 300 PU in pay-prod called claims-ledger"	Tool-calling LLM constrained to `request.schema.json`, calling `preflight` and `create` tools	Only through the same API as the UI, with the same confirmation card and `preview_hash`

- SSP-CHAT-001 The model drafts an `OrderInput`; it never produces an approved spec, never computes price, never bypasses a gate.
- SSP-CHAT-002 Every state-changing action requires the buyer to click **Place order** on a server-produced preflight card (or type `confirm <short-id>` in text clients).
- SSP-CHAT-003 The assistant acts as the buyer (their JWT); it has no identity of its own and no elevated tools.

2. Architecture (`ssp-cp/internal/assistant`)

```
message → Context assembly → Router (intent) ─┬─ knowledge path → CLM query (collections filtered by entitlement) → answer + citations
                                              └─ action path → service resolution → slot filling (schema + derive rules) → tools:
                                                                                                  catalog.describe · destinations.list · orders.preflight · orders.create · orders.status · resources.list · code.view
```

- SSP-CHAT-010 Context assembly per turn: principal + entitlements, selected destination context (`/destinations/{id}/context`), the catalog summaries, the buyer's last 20 orders, current UI location (service and step if in checkout).
- SSP-CHAT-011 Intents: `provision | change | decommission | status | inquire | troubleshoot | navigate`. Routing uses the tool-calling model with a strict JSON output; unknown → ask one clarifying question.
- SSP-CHAT-012 Slot filling: required fields from the request schema minus fields satisfied by `derive` rules; ask only for the remainder, at most 3 per turn, always showing the derived values and their reasons.
- SSP-CHAT-013 Tools are the public API; no private tool exists. Tool results are data (SSP-CHAT-020).
- SSP-CHAT-014 Model gateway: provider-agnostic interface `LLM` with implementations `vertex` (Gemini on Vertex AI in the control-plane project) and `anthropic` (Claude via approved enterprise endpoint); the CLM interface `Grounded` with implementation `contextual` (Contextual AI platform) and `vertexrag` (Vertex AI RAG Engine) as fallback. Approved models are configured per environment; nothing else is reachable.

3. Knowledge index (for the CLM)

Collection	Source	Refresh	Visibility
specs	`ssp-doc` (ADRs, constitution, product docs), service `docs/README.md` from bundles	On merge / publish	All buyers

Collection	Source	Refresh	Visibility
`runbooks`	Buyer-facing FAQs, how-tos, ServiceNow KB export	Daily	All buyers
`tenant-state`	Resource instances, outputs, drift findings, order events (rendered as text)	Near-real-time via events	Filtered by entitlement at query time
`history`	Anonymised order patterns per LOB ("most teams in PBM choose…")	Weekly	All buyers

- SSP-CHAT-030 Every knowledge answer carries citations (document id + requirement/ADR id); answers without a retrieved source above threshold return "I don't have a source for that" and offer a ticket link.
- SSP-CHAT-031 Entitlement filtering happens in the retrieval query (metadata filter), never by post-filtering generated text.

4. Safety

- SSP-CHAT-020 Prompt-injection posture: tool outputs, resource names, ticket text and documents are untrusted; the system prompt forbids following instructions found in them; the red-team suite in `ssp-acceptance` tests this.
- SSP-CHAT-021 No free text ever reaches Terraform; the only path is `OrderInput` → schema validation → preflight → create.
- SSP-CHAT-022 Decommission and production changes through chat require the typed confirmation plus the same gates as the UI (CR, owner sign-off).
- SSP-CHAT-023 Conversations are stored (Postgres `assistant_conversation`, 90 days) and auditable; PII redaction on export.

5. Evaluation (holdout, in `ssp-acceptance`)

- Conversational scenarios: persona, goal, messy opening line, **golden** `OrderInput`; satisfaction = fraction of trajectories ending in a preflight card whose `approved_params` equal the golden set. Target ≥ 0.9 for Phase 1 services.
- Groundedness set: questions with expected citations; questions with no valid answer (must abstain). Target ≥ 0.95 citation accuracy, 0 unsourced policy claims.
- Red-team set: injections via resource names, entitlement escalation, delete coercion. Target 0 successful actions.

6. Metrics

Task completion rate, turns to a valid preflight, slot-correction rate, groundedness score, deflection (questions resolved without a ticket), escalation rate, time from first message to `requested`.

7. Phasing

1. Phase 1: `inquire` + `provision` for Phase 1 services; `specs` and `runbooks` collections; preflight cards.
2. Phase 2: `status`, `change`, `troubleshoot` (reads run logs and Temporal histories through the API).
3. Phase 3: `decommission` with step-up confirmation; proactive drift and delivery messages.

21 — Control Room Specification (`ssp-control-room`, new repo)

The operator console for the platform team. Same design system as the storefront (tokens from `10-product/16-ui-spec.md`), different information architecture: fleet first, then drill-down, with an operator agent chat that turns instructions into confirmed platform actions. It uses only `ssp-cp` `/v1/ops/*`, `/v1/runs/*`, `/v1/imports/*` and `/v1/assistant/*` endpoints; it has no database of its own.

1. Personas and access

Operators (full), security reviewers (policy and audit views, T3 approvals), auditors (read-only). Access via OIDC group claims mapped to personas by `ssp-cp` (SSP-PROD-101). SSP-CR-001 Every action in the control room is an API call that `ssp-cp` audits; the UI adds nothing privileged.

2. Screens

Route	Screen	Content	Key actions
`/`	**Fleet**	Tiles: orders by state (last 24 h / 7 d), runs in progress, queue depth and oldest age per LOB, worker health per task queue, Temporal namespace health, drift open, kill-switch status banner (red if Platform SA disabled or global pause)	Pause/resume global or LOB (reason required), open Temporal UI
`/tenants`	**Tenants**	Table: tenant, LOB, APM, env, paused, queue depth, last run, drift open, break-glass	Pause/resume tenant, force reconcile, force drift scan, set break-glass
`/tenants/[id]`	**Tenant detail**	Desired set (resource instances + versions), generations timeline, runs, drift findings, orders, state bucket path, tenant SA	Retry generation, open run logs, open resource
`/runs` and `/runs/[id]`	**Runs**	Live table; detail: plan summary, policy result, log viewer (SSE tail, redacted), artifacts links, Job name, workflow id link	Retry, cancel (if running), download plan.json
`/orders`	**Orders**	Cross-tenant order table with gates (CR pending, sign-off pending), stuck detection (> SLO)	Open order, nudge (re-poll CR), cancel with reason
`/drift`	**Drift board**	Findings by class and age, grouped by LOB; unmanaged list from asset inventory	Mark accepted, enqueue reconcile (if autoRevert allowed), create adopt task
`/releases`	**Module releases**	Catalog versions, tenants per version, wave planner (canary → LOBs), wave progress, failures	Create waves, advance, halt
`/policy`	**Policy** (security)	Bundles and digests, policy-impact replay results (newly denied / allowed with examples), T3 queue	Approve impact review
`/catalog`	**Catalog admin**	Services, versions, deprecations, publish status, reload	Deprecate, set successor, reload index
`/audit`	**Audit**	Searchable audit events; export CSV for a time range	—

Route	Screen	Content	Key actions
`/kill-switch`	**Kill switch**	Current state of Platform SA, scope pauses; two-person flow (first operator requests, second confirms within 10 min)	Disable / re-enable Platform SA
`/chat` (also a right slide-over everywhere)	**Operator chat**	See `23-operator-agent-chat-spec.md`	—

3. Requirements

- SSP-CR-010 Fleet metrics refresh every 15 s via `/v1/ops/fleet`; run logs stream via SSE; nothing is polled faster than 5 s.
- SSP-CR-011 Every destructive or scope-wide action (pause global/LOB, kill switch, halt wave, cancel run) requires a typed reason and shows the blast radius (tenants affected, runs in flight) before confirm.
- SSP-CR-012 Kill switch requires two distinct operators (API-enforced, SSP-API-013); the UI shows the pending approval with a countdown.
- SSP-CR-013 Links to Temporal Web UI are deep links (`/namespaces/ssp/workflows/<id>`) behind the same SSO; the control room never embeds Temporal UI in an iframe.
- SSP-CR-014 Stuck detection: an order in `approved` with gate `none` for > 10 min, or a run `running` beyond its timeout, is highlighted and counted in a "needs attention" tile.
- SSP-CR-015 The control room renders fine when `ssp-cp` is degraded: cached last fleet snapshot with an age badge; actions disabled.
- SSP-CR-016 No tenant-owned secrets or raw state are ever displayed; outputs shown are the non-secret outputs recorded in `resource_instance.outputs`.

4. Monitoring status (what "status" means here)

Three layers, one source of truth (events and metrics from `ssp-cp` and workers):

1. **Order/run status** — event stream per order (buyer and operator see the same).
2. **Fleet status** — aggregates from Postgres plus Temporal visibility queries (`ListWorkflow` with search attributes) plus worker health (task queue pollers).
3. **Platform health** — Prometheus metrics (`30-infra/36-observability.md`) rendered in Grafana; the control room links to dashboards rather than re-implementing them, except the fleet tiles.

5. Repo structure

```
ssp-control-room/
├── app/(shell)/layout.tsx          top bar (env badge, search, chat), left nav (Fleet, Tenants, Runs, Orders, Drift,  ↩
   Releases, Policy, Catalog, Audit, Kill switch)
├── app/… one folder per route above
├── components/ops/                 FleetTile, QueueGauge, RunLogViewer (SSE), BlastRadiusDialog, TwoPersonConfirm
├── lib/api/                        generated client from 12-openapi.yaml (ops + runs + imports + assistant tags)
├── packages/ui → git subtree or npm package @ssp/ui from ssp-ui (shared tokens/components)
└── tests/e2e/
```

Deployment: namespace `ssp-control-room`, internal ingress only, operator group required at the gateway.

22 — Migration App Specification (`ssp-migrate`, new repo)

Moves existing cloud resources, provisioned by Taxi or by hand, into the SSP construct: one tenant boundary, one tenant SA, one state file, resources expressed as catalog services. The core platform is designed for the 2.0 layout (D-09); this app is the **translation shim** for everything else, including one-time state import.

1. What migration produces

For a tenant boundary (project today; account/subscription later):

1. Tenant record in `ssp-cp` (LOB, APM, tier, data class, env, tenant SA, state prefix).
2. Resource instances in the desired set, each bound to a catalog service and version.
3. A Terraform state file in the LOB bucket containing exactly those resources, created by `import` blocks, with a **zero-change** plan afterwards.
4. Orders of kind `import` (one per resource instance) in `observed`, so history and audit start at migration.
5. A report of what was skipped or flagged (resources with no catalog service, non-conforming configurations, shared 1.0 resources).

2. Journey (wizard)

Step	Screen	What happens	API		
1	**Select source**	Pick cloud, boundary (project id), legacy layout (1.0 shared / 2.0), and scope filter (labels, services)	`POST /v1/imports`		
2	**Inventory**	Cloud Asset Inventory listing filtered to catalog-known resource types; grouped by suggested service; confidence score; unknown types listed separately	`GET /v1/imports/{id}` (state `inventorying` → `mapping`)		
3	**Map**	For each asset: decision `import	skip`, `service id`, `stable name`, params pre-extracted from the live resource (reverse-mapped through the module's variable schema), APM owner (for 1.0 multi-app projects, per resource)	`PUT /v1/imports/{id}/mapping`
4	**Onboard tenant**	If the boundary is not yet a tenant: creates tenant SA, grants the Platform SA token-creator, grants the SA its state prefix, writes the tenant record (this is the per-project SA isolation work Dwayne is taking to Raju / Durant; the app automates the platform side)	internal workflow step		

Step	Screen	What happens	API
5	**Dry run**	Generates `import { to = module.<name>.<type>.<addr> ; id = "<cloud id>" }` blocks plus the rendered root module; runs plan; shows diff. Must be zero updates/destroys	`POST /v1/imports/{id}/dry-run`
6	**Reconcile differences**	For each non-zero diff: choose *adopt live value into params* (changes the order), *accept planned change* (will be applied; needs CR in prod), or *skip asset*	loop to step 3
7	**Execute**	Isolated generation on the TenantReconciler: apply import, capture observed, create resource instances and `import` orders; post-build steps run (CMDB reconcile)	`POST /v1/imports/{id}/execute`
8	**Report**	Summary, skipped/flagged list with reasons, link to the tenant in the control room; export JSON/CSV	`GET /v1/imports/{id}`

3. Requirements

- SSP-MIG-001 Inventory uses read-only credentials (Cloud Asset Inventory viewer via the Platform SA impersonating a read-only inventory SA); nothing in steps 1–6 writes to the cloud.
- SSP-MIG-002 Reverse mapping: each `ssp-module-<service>` ships `import/mapping.yaml` (asset type → module address template, attribute → param extraction) so the app has no per-service code.
- SSP-MIG-003 A resource whose live configuration violates a MUST guardrail cannot be imported as-is; the wizard offers `accept planned change` (brings it to standard; gated like a `change` order) or `flag` (recorded, excluded, assigned to the app owner).
- SSP-MIG-004 1.0 shared projects: the project becomes one tenant with a primary APM; every resource instance carries its own `apm_id` label; the report lists resources per APM so owners can later split by creating new 2.0 projects and moving workloads (the platform never moves data).
- SSP-MIG-005 Execution is idempotent and resumable: import blocks already applied are skipped on retry; a failed execution leaves the state file consistent (Terraform import is per-resource).
- SSP-MIG-006 Scale: a project with 10,000 resources imports in batches of 200 import blocks per run to keep plan time bounded; progress is shown per batch.
- SSP-MIG-007 Taxi hand-off: when Taxi still manages a resource, the wizard shows the Taxi record (from the inventory step, via label `managed_by=taxi`) and requires explicit "take over from Taxi" acknowledgement; a post-build step marks the Taxi record read-only (integration pending Norm's inventory).
- SSP-MIG-008 Every import run is a normal run (artifacts, logs, policy at plan stage) visible in the control room.

4. Future builds and migrations inside the construct

The same app covers two more flows, both reusing the wizard:

- **Adopt** (`kind: adopt`): a resource created out of band inside an already-managed tenant (found as `unmanaged` drift) is mapped and imported; the drift finding is resolved `adopted`.

- **Re-platform** (`kind: upgrade` with a source/target service pair): e.g., self-managed Postgres VM → Cloud SQL service. The app only orchestrates *provision target* → *owner migrates data* → *decommission source* as linked orders with a checklist; data movement is never automatic.

5. Repo structure

```
ssp-migrate/
├── app/(wizard)/[importId]/{source,inventory,map,onboard,dry-run,reconcile,execute,report}/page.tsx
├── components/migrate/              InventoryTable (virtualized), MappingEditor (schema-aware param form via @ssp/for ↵
   m-renderer), DiffViewer (plan JSON), BatchProgress
├── lib/api/                         generated client (imports + tenants + runs tags)
├── lib/reverse-map/                 applies module import/mapping.yaml to asset JSON → params
└── tests/fixtures/assets/           sanitized Cloud Asset Inventory exports (1.0 and 2.0 projects)
```

Deployment: namespace `ssp-migrate`, operator persona only in Phase 2; app owners get a read-only "my migration" view in Phase 3.

23 — Operator Agent Chat Specification (control room)

Humans give instructions in plain language; the agent turns them into **confirmed, audited platform actions** or grounded answers about the fleet. Same gateway as Sidekick (`ssp-cp/internal/assistant`), surface `control_room`, operator tool set.

1. What it can do

Intent	Example instruction	Tools used	Confirmation
Fleet question	"What's stuck right now?" "How many runs failed in PBM today?"	`ops.fleet`, `ops.tenants`, `runs.list`, `orders.list`	none (read)
Explain	"Why did run_… fail?" "Why is tenant X paused?"	`runs.get`, `runs.logs`, `orders.get`, knowledge (specs, runbooks)	none
Scoped control	"Pause all applies in PBM for the next hour because of the Spanner incident"	`ops.pause` (+ `ops.resume` scheduled)	**Blast-radius card** + typed `confirm`
Retry	"Retry the failed generation for gcp:pay-prod-7a1c"	`runs.retry` / `ops.reconcile`	confirm
Drift triage	"Mark the label drift in claims-ledger as accepted"	`drift.resolve`	confirm
Release	"Start the canary wave for spanner 1.3.0 with pay-sandbox and claims-dev"	`releases.createWaves`, `releases.advance`	confirm; `halt` is one word, no confirm
Break-glass	"Open break-glass on tenant X until 02:00 for INC0012345"	`ops.breakGlass`	confirm
Kill switch	"Disable the platform service account"	`ops.killswitch`	two-person: the agent records the first approval and tells the operator who must confirm
Spec change	"The Spanner PU minimum should be 200 for prod"	opens a draft spec-change PR in `ssp-doc` (via GitHub tool)	PR review by humans; nothing changes live

2. Rules

- SSP-OPC-001 The agent acts as the signed-in operator; no action is possible that the operator could not perform in the UI.
- SSP-OPC-002 Every state-changing tool call is preceded by a **Blast-radius card**: scope, tenants affected, runs in flight, orders that will wait, reversal command. The operator types `confirm` (or clicks) within 2 minutes; otherwise the action is dropped.
- SSP-OPC-003 Halt-type actions (`releases.halt`, `ops.pause tenant:<id>` for a single tenant) execute on a single explicit word to keep incident response fast; widening scope (LOB, global) always needs the card.
- SSP-OPC-004 Ambiguous scope ("pause everything") is never resolved by assumption; the agent asks and lists the options.
- SSP-OPC-005 The agent never edits Terraform state, never touches cloud resources directly, never calls `kubectl`; it only calls `ssp-cp` tools.
- SSP-OPC-006 Transcripts are retained 1 year and linked from the audit events they caused.
- SSP-OPC-007 Knowledge answers cite runbooks and requirement ids; "I don't know" with a link to the runbook index is the correct answer when retrieval is empty.

- SSP-OPC-008 Red-team coverage in `ssp-acceptance`: injected instructions in run logs and resource names must never produce an action.

3. Shape of a turn (SSE events)

`token` (text), `card` (`blast_radius | fleet_summary | run_explainer | citation`), `tool_call` (name, args summary), `tool_result` (summary), `confirm_required` (action id, expires_at), `done`.

4. Scheduled and proactive messages (Phase 3)

The agent may post into the control room feed (not act): SLO breach summaries, stuck-order digests every 30 min during business hours, release wave progress. Each message carries the query it ran so it can be re-run.

31 — GKE Platform Layout (existing GCP project and cluster)

Assumptions: one GCP project `<CP_PROJECT>` already exists with a GKE cluster `<CLUSTER>` in region `<REGION>`; Workload Identity is enabled on the cluster (`--workload-pool=<CP_PROJECT>.svc.id.goog`); the VPC has Private Google Access and Private Service Access configured for Cloud SQL. Replace placeholders in angle brackets everywhere.

1. Namespaces

Namespace	Workloads	KSA → GSA (Workload Identity)	Network policy
`temporal`	Temporal server (frontend, history, matching, worker), web UI, admintools, schema jobs	`temporal` → none (DB password via secret)	Ingress 7233 from `ssp-cp`, `ssp-workers`; 8080 from gateway only
`ssp-cp`	`ssp-api` Deployment, `ssp-control-worker` Deployment	`ssp-cp-api` → `ssp-cp-api@<CP_PROJECT>` ; `ssp-control-worker` → `ssp-control-worker@<CP_PROJECT>`	Ingress 8080 from gateway and apps; egress to Cloud SQL, GCS, Secret Manager, Temporal, ServiceNow/CMDB endpoints
`ssp-workers`	`tf-worker`, `postbuild-worker`, `drift-worker` Deployments	`tf-worker` → `ssp-tf-worker@<CP_PROJECT>` (GCS artifacts read, K8s only); `postbuild-worker` → `ssp-postbuild@<CP_PROJECT>`	Egress to Temporal, GCS, integrations; API server
`ssp-runs`	Terraform Jobs only	`tf-runner` → `ssp-tf-runner@<CP_PROJECT>` (**Platform SA**, kill switch)	No ingress; egress to Google APIs (Private Google Access), module/catalog buckets, Secret Manager; **no** egress to Temporal or Postgres
`ssp-ui`	Storefront	`ssp-ui` → none	Ingress from gateway
`ssp-control-room`	Operator console	none	Ingress from gateway (operator group)
`ssp-migrate`	Migration app	none	Ingress from gateway (operator group)
`observability`	OTel collector, Prometheus (or managed collection), Grafana	per vendor	—

SSP-INF-001 Each namespace has a default-deny `NetworkPolicy` for ingress and egress with explicit allows as above (GKE Dataplane V2 or Calico required). SSP-INF-002 Pod Security Standards `restricted` on all namespaces except `ssp-runs`, which uses `baseline` (Terraform needs a writable work dir; still non-root, no privilege escalation).

2. Node pools

Pool	Machine	Autoscaling	Taint / label	Used by
existing default	as is	as is	—	Temporal, ssp-cp, ssp-workers, UIs
`ssp-runs`	e2-standard-4 (4 vCPU / 16 GiB)	0 → 20	taint `ssp.io/runs=true:NoSchedule`, label `ssp.io/pool=runs`	Terraform Jobs only (toleration + nodeSelector in the Job template)

```
gcloud container node-pools create ssp-runs \
```

```
  --cluster=<CLUSTER> --region=<REGION> --project=<CP_PROJECT> \
  --machine-type=e2-standard-4 --enable-autoscaling --min-nodes=0 --max-nodes=20 \
  --node-taints=ssp.io/runs=true:NoSchedule --node-labels=ssp.io/pool=runs \
  --workload-metadata=GKE_METADATA --shielded-secure-boot --shielded-integrity-monitoring
```

3. Service accounts and IAM

```
P=<CP_PROJECT>; POOL="$P.svc.id.goog"
for sa in ssp-cp-api ssp-control-worker ssp-tf-worker ssp-tf-runner ssp-postbuild ssp-inventory; do
  gcloud iam service-accounts create $sa --project=$P --display-name="SSP $sa"; done

# Workload Identity bindings (KSA namespace/name → GSA)
bind(){ gcloud iam service-accounts add-iam-policy-binding "$1@$P.iam.gserviceaccount.com" \
  --role=roles/iam.workloadIdentityUser --member="serviceAccount:$POOL[$2]"; }
bind ssp-cp-api          ssp-cp/ssp-cp-api
bind ssp-control-worker  ssp-cp/ssp-control-worker
bind ssp-tf-worker       ssp-workers/tf-worker
bind ssp-postbuild       ssp-workers/postbuild-worker
bind ssp-tf-runner       ssp-runs/tf-runner

# Project-level roles (control plane project)
gcloud projects add-iam-policy-binding $P --member=serviceAccount:ssp-cp-api@$P.iam.gserviceaccount.com --role=roles/c ↩
   loudsql.client
gcloud projects add-iam-policy-binding $P --member=serviceAccount:ssp-cp-api@$P.iam.gserviceaccount.com --role=roles/c ↩
   loudsql.instanceUser
gcloud projects add-iam-policy-binding $P --member=serviceAccount:ssp-cp-api@$P.iam.gserviceaccount.com --role=roles/s ↩
   ecretmanager.secretAccessor
gcloud projects add-iam-policy-binding $P --member=serviceAccount:ssp-control-worker@$P.iam.gserviceaccount.com --role ↩
   =roles/cloudsql.client
gcloud projects add-iam-policy-binding $P --member=serviceAccount:ssp-control-worker@$P.iam.gserviceaccount.com --role ↩
   =roles/pubsub.publisher
# Buckets (see 35-state-buckets.md): catalog read for api/control-worker/runner; artifacts read for tf-worker; expecte ↩
   d/runs write for control-worker and runner
```

Tenant onboarding (per tenant project <T>; automated by the migration/onboarding workflow, manual for POC):

```
gcloud iam service-accounts create ssp-tenant --project=<T>
# Platform SA may mint tokens for the tenant SA (impersonation) — this edge is the kill switch
gcloud iam service-accounts add-iam-policy-binding ssp-tenant@<T>.iam.gserviceaccount.com \
  --member=serviceAccount:ssp-tf-runner@$P.iam.gserviceaccount.com --role=roles/iam.serviceAccountTokenCreator
# Tenant SA gets the roles the catalog services need in its own project (least privilege per service; Spanner example)
gcloud projects add-iam-policy-binding <T> --member=serviceAccount:ssp-tenant@<T>.iam.gserviceaccount.com --role=roles ↩
   /spanner.admin
gcloud projects add-iam-policy-binding <T> --member=serviceAccount:ssp-tenant@<T>.iam.gserviceaccount.com --role=roles ↩
   /serviceusage.serviceUsageConsumer
# State prefix access in the LOB bucket is granted with an IAM condition (35-state-buckets.md)
```

SSP-INF-010 The Platform SA (`ssp-tf-runner`) holds **no** project roles anywhere; its only power is `serviceAccountTokenCreator` on tenant SAs plus read on the module/catalog/policy buckets. Disabling it (`gcloud iam service-accounts disable`) stops every run within one token lifetime (≤ 1 h; runs request 15-minute tokens). SSP-INF-011 No service account keys are ever created (org policy `iam.disableServiceAccountKeyCreation` enforced).

4. Secrets

- SSP-INF-020 Source of truth is Secret Manager in <CP_PROJECT>. Kubernetes Secrets are materialised by the **External Secrets Operator** (`ClusterSecretStore` using Workload Identity) or, for POC, created once with `kubectl create secret`. Nothing is committed.
- Secrets list: `temporal-db-password`, `ssp-db-password` (only if not using IAM DB auth), `servicenow-oauth`, `cmdb-api`, `github-modules-ro` (POC only; prod uses the GCS module mirror), `llm-gateway` (assistant), `oidc-client`.

5. Images and deploy

- Artifact Registry: <REGION>-docker.pkg.dev/<CP_PROJECT>/ssp/{ssp-api,ssp-control-worker,tf-worker,tf-runner,postbuild-worker,drift-worker,ssp-ui,ssp-control-room,ssp-migrate}; images signed (cosign) and admitted by Binary Authorization in prod.

- Each code repo ships a Helm chart under `deploy/chart/` and env values under `deploy/envs/{sandbox,nonprod,prod}.yaml`. POC: `helm upgrade --install` from CI; next: Argo CD application per repo.
- SSP-INF-030 Ingress: one internal HTTPS load balancer (GKE Gateway API, internal) with hostnames `ssp.<env>.<domain>`, `ssp-cp.<env>.<domain>`, `ops.<env>.<domain>`, `migrate.<env>.<domain>`, `temporal.<env>.<domain>`; corporate SSO (IAP or the enterprise gateway) in front; API Forge / Apigee exposure of `ssp-cp` is a later step (D-19 open item).

6. Bootstrap order (see `80-runbooks/81-bootstrap.md`)

1. APIs, node pool, namespaces, network policies. 2. Service accounts and WI bindings. 3. Cloud SQL (34). 4. Buckets (35). 5. Temporal (32). 6. `ssp-cp` migrations + API + control worker. 7. Workers (33). 8. Catalog publish of the first service. 9. UI. 10. First order in sandbox.

32 — Temporal on GKE: Installation Specification

Self-hosted Temporal (D-11) on the existing GKE cluster, namespace `temporal`, persistence on Cloud SQL for PostgreSQL. Uses the official Helm chart from `https://go.temporal.io/helm-charts` (chart `temporal`; server image `temporalio/server` 1.31.x, web UI 2.52.x at the time of writing — pin the versions you install).

1. Prerequisites

Item	Value
Kubernetes	GKE ≥ 1.29, Workload Identity enabled, Dataplane V2 (network policy)
Helm	≥ 3.12
Database	Cloud SQL for PostgreSQL **16**, private IP in the cluster VPC, instance `ssp-temporal-<env>` (separate from the SSP control-plane DB in nonprod/prod; shared instance acceptable in sandbox). See `34-control-plane-postgres.md`.
Databases	`temporal` and `temporal_visibility`, owner role `temporal`
Secret	Secret Manager `temporal-db-password` → K8s Secret `temporal-db` key `password` in namespace `temporal`
DNS/Ingress	internal hostname `temporal.<env>.<domain>` → `temporal-web` 8080 behind SSO (operators only)

Create the databases (from a bastion or a one-off pod with `psql` and the Cloud SQL private IP):

```
create role temporal login password '<from Secret Manager>';
create database temporal owner temporal;
create database temporal_visibility owner temporal;
```

2. Namespace, secret, network policy

```
kubectl create namespace temporal
kubectl label namespace temporal pod-security.kubernetes.io/enforce=restricted
kubectl -n temporal create secret generic temporal-db \
  --from-literal=password="$(gcloud secrets versions access latest --secret=temporal-db-password --project=<CP_PROJECT ↵
   >)"
kubectl apply -f 30-infra/temporal/networkpolicy.yaml
```

3. Install

```
helm upgrade --install temporal temporal \
  --repo https://go.temporal.io/helm-charts \
  --version <CHART_VERSION> \
  --namespace temporal \
  -f 30-infra/temporal/values.yaml \
  --set server.config.persistence.datastores.default.sql.connectAddr=<CLOUDSQL_PRIVATE_IP>:5432 \
  --set server.config.persistence.datastores.visibility.sql.connectAddr=<CLOUDSQL_PRIVATE_IP>:5432 \
  --timeout 900s --wait
```

The chart runs schema create/update Jobs as Helm hooks (`schema.useHelmHooks: true`) because `manageSchema: true` is set on both datastores. `createDatabase: false` because the databases were created above with the right owner.

`30-infra/temporal/values.yaml` (committed; placeholders only for host):

```
# Temporal Helm values for SSP — sandbox sizing; see "Sizing" for nonprod/prod
server:
  image:
    repository: temporalio/server
    tag: 1.31.2            # pin; upgrade deliberately
  replicaCount: 1          # per service unless overridden below
  config:
    logLevel: info
    numHistoryShards: 512  # cannot be changed after first deploy; 512 sandbox/nonprod, 2048 prod (new instance)
    persistence:
```

```
      defaultStore: default
      visibilityStore: visibility
      datastores:
        default:
          driver: sql
          sql:
            driverName: postgres12
            pluginName: postgres12
            databaseName: temporal
            connectAddr: "CLOUDSQL_PRIVATE_IP:5432"   # overridden by --set
            connectProtocol: tcp
            user: temporal
            existingSecret: temporal-db
            secretKey: password
            maxConns: 20
            maxIdleConns: 20
            maxConnLifetime: "1h"
            createDatabase: false
            manageSchema: true
            tls:
              enabled: false      # sandbox: private IP only. nonprod/prod: true + caFile (see §6)
        visibility:
          driver: sql
          sql:
            driverName: postgres12
            pluginName: postgres12
            databaseName: temporal_visibility
            connectAddr: "CLOUDSQL_PRIVATE_IP:5432"
            connectProtocol: tcp
            user: temporal
            existingSecret: temporal-db
            secretKey: password
            maxConns: 10
            maxIdleConns: 10
            maxConnLifetime: "1h"
            createDatabase: false
            manageSchema: true
            tls:
              enabled: false
    namespaces:
      create: true
      namespace:
        - name: ssp
          retention: 30d
        - name: ssp-sandbox
          retention: 7d
  frontend:
    replicaCount: 2
    service:
      type: ClusterIP
      port: 7233
    resources: { requests: { cpu: 200m, memory: 512Mi }, limits: { cpu: "1", memory: 1Gi } }
  history:
    replicaCount: 2
    resources: { requests: { cpu: 500m, memory: 1Gi }, limits: { cpu: "2", memory: 2Gi } }
  matching:
    replicaCount: 2
    resources: { requests: { cpu: 200m, memory: 512Mi }, limits: { cpu: "1", memory: 1Gi } }
  worker:
    replicaCount: 1
    resources: { requests: { cpu: 200m, memory: 512Mi }, limits: { cpu: "1", memory: 1Gi } }
  metrics:
    serviceMonitor:
      enabled: false        # true when Prometheus Operator / managed collection is present
  additionalEnv:
    - name: TEMPORAL_ENVIRONMENT
      value: ssp-sandbox
  podSecurityContext: { runAsNonRoot: true, runAsUser: 1000, fsGroup: 1000, seccompProfile: { type: RuntimeDefault } }
  securityContext: { allowPrivilegeEscalation: false, capabilities: { drop: [ALL] }, readOnlyRootFilesystem: true }

schema:
  useHelmHooks: true
  backoffLimit: 10
  ttlSecondsAfterFinished: 86400
  resources: { requests: { cpu: 100m, memory: 256Mi }, limits: { cpu: 500m, memory: 512Mi } }

admintools:
  enabled: true
  resources: { requests: { cpu: 50m, memory: 128Mi }, limits: { cpu: 200m, memory: 256Mi } }
```

```
web:
  enabled: true
  image:
    tag: 2.52.0
  replicaCount: 1
  service:
    type: ClusterIP
    port: 8080
  additionalEnv:
    - name: TEMPORAL_UI_PUBLIC_PATH
      value: ""
  resources: { requests: { cpu: 50m, memory: 128Mi }, limits: { cpu: 500m, memory: 512Mi } }

serviceAccount:
  create: true
  name: temporal
```

If the chart version you install does not support `server.config.namespaces`*, create the namespaces with the CLI in §4 instead; keep the rest unchanged.*

4. Post-install: namespaces, search attributes, verification

```
# Health
kubectl -n temporal get pods
kubectl -n temporal exec deploy/temporal-admintools -- temporal operator cluster health --address temporal-frontend:72 ↩
   33

# Namespaces (if not created by values)
kubectl -n temporal exec deploy/temporal-admintools -- temporal operator namespace create --namespace ssp --retention  ↩
   720h --address temporal-frontend:7233
kubectl -n temporal exec deploy/temporal-admintools -- temporal operator namespace create --namespace ssp-sandbox --re ↩
   tention 168h --address temporal-frontend:7233

# Custom search attributes used by SSP (13-workflows-spec.md SSP-WF-001)
for a in TenantId LobId OrderId RunKind State; do
  kubectl -n temporal exec deploy/temporal-admintools -- \
    temporal operator search-attribute create --namespace ssp --name $a --type Keyword --address temporal-frontend:723 ↩
   3
done

# UI (sandbox): port-forward; nonprod/prod: internal ingress + SSO
kubectl -n temporal port-forward svc/temporal-web 8080:8080
```

Verification checklist (record in the runbook):

- [ ] All `temporal-*` pods Ready; schema hook Jobs `Completed`.
- [ ] `temporal operator cluster health` returns SERVING.
- [ ] Namespace `ssp` listed with retention 30 d; search attributes present (`temporal operator search-attribute list --namespace ssp`).
- [ ] A hello-world workflow from `ssp-cp/cmd/worker --selftest` completes and is visible in the UI.
- [ ] Postgres: tables exist in both databases; connection count < `maxConns` under idle.

5. Connection from SSP services

- Address `temporal-frontend.temporal.svc.cluster.local:7233`, namespace `ssp`. Go SDK `client.Dial(client.Options{HostPort: ..., Namespace: "ssp"})`.
- NetworkPolicy in `temporal` allows ingress on 7233 only from namespaces `ssp-cp` and `ssp-workers` (label `kubernetes.io/metadata.name`). `ssp-runs` is **not** allowed (Terraform jobs never talk to Temporal).

`30-infra/temporal/networkpolicy.yaml`:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: temporal-frontend-ingress, namespace: temporal }
spec:
  podSelector: { matchLabels: { app.kubernetes.io/component: frontend } }
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: ssp-cp } }
```

```
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: ssp-workers } }
        - podSelector: {}   # intra-namespace (admintools, web)
      ports: [{ port: 7233, protocol: TCP }]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: temporal-web-ingress, namespace: temporal }
spec:
  podSelector: { matchLabels: { app.kubernetes.io/component: web } }
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: gke-gateway } }  # or your ingress namespac ↩
   e
      ports: [{ port: 8080, protocol: TCP }]
```

6. Hardening for nonprod/prod

Area	Setting
DB TLS	Cloud SQL "require SSL"; mount the server CA (`additionalVolumes`/`additionalVolumeMounts` on `server`) and set `sql.tls.enabled: true`, `sql.tls.caFile: /etc/temporal/certs/server-ca.pem`, `sql.tls.enableHostVerification: false` (Cloud SQL private IP has no hostname)
Frontend auth	Enable Temporal authorization with a JWT claim mapper against the corporate IdP for UI/CLI users; internal services keep mTLS-less in-cluster access restricted by NetworkPolicy in Phase 1, mTLS (`server.config.tls`) in Phase 2
Sizing	prod: `numHistoryShards: 2048`, frontend 3, history 3–4, matching 3, worker 2; Cloud SQL 4 vCPU/16 GB HA; `maxConns` 50/20
Archival	Enable history archival to a GCS bucket for the `ssp` namespace (audit); retention stays 30 d in DB
Backups	Cloud SQL automated backups + PITR 7 d; quarterly restore test
Upgrades	Follow Temporal's one-minor-at-a-time rule; schema update hooks run first; test in sandbox
Alerts	`temporal_service_errors_total`, persistence latency p99 > 500 ms, history shard controller lag, task queue backlog per SSP queue

7. Day-2 commands

```
# Describe a workflow (order)
temporal workflow describe --namespace ssp --workflow-id order:ord_01J... --address temporal-frontend:7233
# List stuck reconcilers
temporal workflow list --namespace ssp --query 'WorkflowType="TenantReconciler" AND ExecutionStatus="Running"' --addre ↩
   ss temporal-frontend:7233
# Reset a workflow to last workflow task (incident use only; record in audit)
temporal workflow reset --namespace ssp --workflow-id tenant:gcp:pay-prod-7a1c --type LastWorkflowTask --reason "INC.. ↩
   ." --address temporal-frontend:7233
```

33 — Terraform Worker and Runner on GKE: Installation Specification

Implements D-12: a dedicated Temporal activity worker (`tf-worker`, in `ssp-workers`) launches one **Kubernetes Job per operation** running the `tf-runner` image; the Job name is derived deterministically from the operation ID so a retried activity re-attaches instead of re-running; the worker polls Job status (~30 s) and heartbeats; a callback path is a later addition. Worker and Jobs run in the same cluster, in different namespaces.

```
Temporal (ns temporal) ◄── polls task queue tf-runs-<lob> ── tf-worker (ns ssp-workers, KSA tf-worker)
                                                                  │ create/get/watch Job (RBAC in ns ssp-runs only)
                                                                  ▼
                                                  Job tf-<hash> (ns ssp-runs, KSA tf-runner → GSA ssp-tf-runner = P ↩
   latform SA)
                                                                  │ impersonate ssp-tenant@<project>  (15-min token)
                                                                  ├─ GCS: expected/<gen> (read), terraform.tfstate (lo ↩
   ck/write), runs/<run_id>/ (write)
                                                                  └─ Cloud APIs of the tenant project as the tenant SA
```

1. Files in this folder (`30-infra/tf-worker/`)

File	Purpose
`namespace-rbac.yaml`	Namespaces, KSAs, Role/RoleBinding letting `tf-worker` manage Jobs in `ssp-runs`
`networkpolicy.yaml`	Default-deny for `ssp-runs`, egress to Google APIs only; worker egress
`tf-worker-deployment.yaml`	Worker Deployment + config
`job-template.yaml`	The Job the worker instantiates (also embedded in Go as a template)
`Dockerfile.runner`	`tf-runner` image: Terraform, OPA, gcloud-less GCS access via the Go `gcsutil`, entrypoint
`runner.sh`	Entrypoint: init → plan/apply/refresh/import → policy → artifacts → result.json
`provider-mirror.tfrc`	Terraform CLI config: filesystem provider mirror, no registry egress

2. Kubernetes objects

`namespace-rbac.yaml`:

```
apiVersion: v1
kind: Namespace
metadata:
  name: ssp-runs
  labels: { pod-security.kubernetes.io/enforce: baseline, ssp.io/tier: runs }
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: tf-runner
  namespace: ssp-runs
  annotations:
    iam.gke.io/gcp-service-account: ssp-tf-runner@CP_PROJECT.iam.gserviceaccount.com   # Platform SA (kill switch)
automountServiceAccountToken: false
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: tf-worker
  namespace: ssp-workers
  annotations:
    iam.gke.io/gcp-service-account: ssp-tf-worker@CP_PROJECT.iam.gserviceaccount.com   # reads run artifacts only
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: tf-job-manager, namespace: ssp-runs }
rules:
  - apiGroups: ["batch"]
```

```
    resources: ["jobs"]
    verbs: ["create", "get", "list", "watch", "delete"]
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
  - apiGroups: [""]
    resources: ["pods/log"]
    verbs: ["get"]
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["create", "get", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: tf-worker-manages-jobs, namespace: ssp-runs }
roleRef: { apiGroup: rbac.authorization.k8s.io, kind: Role, name: tf-job-manager }
subjects:
  - { kind: ServiceAccount, name: tf-worker, namespace: ssp-workers }
---
apiVersion: v1
kind: ResourceQuota
metadata: { name: runs-quota, namespace: ssp-runs }
spec:
  hard: { pods: "60", requests.cpu: "60", requests.memory: 120Gi, count/jobs.batch: "200" }
---
apiVersion: v1
kind: LimitRange
metadata: { name: runs-limits, namespace: ssp-runs }
spec:
  limits:
    - type: Container
      default: { cpu: "1", memory: 2Gi }
      defaultRequest: { cpu: 500m, memory: 1Gi }
      max: { cpu: "4", memory: 8Gi }
```

networkpolicy.yaml:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny, namespace: ssp-runs }
spec: { podSelector: {}, policyTypes: [Ingress, Egress] }
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: runner-egress, namespace: ssp-runs }
spec:
  podSelector: { matchLabels: { app: tf-runner } }
  policyTypes: [Egress]
  egress:
    - to: [{ ipBlock: { cidr: 169.254.169.254/32 } }]            # GKE metadata server (Workload Identity tokens)
      ports: [{ port: 80, protocol: TCP }]
    - to: [{ ipBlock: { cidr: 199.36.153.8/30 } }]               # private.googleapis.com (Private Google Access)
      ports: [{ port: 443, protocol: TCP }]
    - to: [{ ipBlock: { cidr: 199.36.153.4/30 } }]               # restricted.googleapis.com (if VPC-SC)
      ports: [{ port: 443, protocol: TCP }]
    - to: [{ namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } }, podSelector: { matchLab ↩
   els: { k8s-app: kube-dns } } }]
      ports: [{ port: 53, protocol: UDP }, { port: 53, protocol: TCP }]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: tf-worker-egress, namespace: ssp-workers }
spec:
  podSelector: { matchLabels: { app: tf-worker } }
  policyTypes: [Egress]
  egress:
    - to: [{ namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: temporal } } }]
      ports: [{ port: 7233, protocol: TCP }]
    - to: [{ ipBlock: { cidr: 199.36.153.8/30 } }]
      ports: [{ port: 443, protocol: TCP }]
    - to: [{ ipBlock: { cidr: 169.254.169.254/32 } }]
      ports: [{ port: 80, protocol: TCP }]
    - to: [{ ipBlock: { cidr: KUBE_APISERVER_CIDR } }]           # GKE control plane endpoint (private)
      ports: [{ port: 443, protocol: TCP }]
    - to: [{ namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } }, podSelector: { matchLab ↩
   els: { k8s-app: kube-dns } } }]
      ports: [{ port: 53, protocol: UDP }, { port: 53, protocol: TCP }]
```

If Private Google Access routes to the default `*.googleapis.com` ranges in your VPC rather than `private.googleapis.com`, replace the CIDR with your egress allowlist (or use an FQDN egress policy if your CNI supports it). The intent is: Google APIs + metadata + DNS, nothing else.

`tf-worker-deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: tf-worker, namespace: ssp-workers, labels: { app: tf-worker } }
spec:
  replicas: 2
  selector: { matchLabels: { app: tf-worker } }
  template:
    metadata:
      labels: { app: tf-worker }
      annotations: { prometheus.io/scrape: "true", prometheus.io/port: "9090" }
    spec:
      serviceAccountName: tf-worker
      securityContext: { runAsNonRoot: true, runAsUser: 65532, seccompProfile: { type: RuntimeDefault } }
      containers:
        - name: worker
          image: REGION-docker.pkg.dev/CP_PROJECT/ssp/tf-worker:VERSION
          args: ["--task-queue=tf-runs-default"]      # one Deployment per LOB queue in prod: --task-queue=tf-runs-<lo ↩
   b>
          env:
            - { name: TEMPORAL_ADDRESS, value: temporal-frontend.temporal.svc.cluster.local:7233 }
            - { name: TEMPORAL_NAMESPACE, value: ssp }
            - { name: RUNS_NAMESPACE, value: ssp-runs }
            - { name: RUNNER_IMAGE, value: REGION-docker.pkg.dev/CP_PROJECT/ssp/tf-runner:VERSION }
            - { name: RUNNER_KSA, value: tf-runner }
            - { name: MAX_CONCURRENT_ACTIVITIES, value: "10" }
            - { name: POLL_INTERVAL, value: "30s" }
            - { name: JOB_TTL_SECONDS, value: "86400" }
            - { name: JOB_ACTIVE_DEADLINE_SECONDS, value: "5400" }
            - { name: OTEL_EXPORTER_OTLP_ENDPOINT, value: http://otel-collector.observability:4317 }
          ports: [{ name: metrics, containerPort: 9090 }]
          resources: { requests: { cpu: 100m, memory: 256Mi }, limits: { cpu: "1", memory: 1Gi } }
          securityContext: { allowPrivilegeEscalation: false, readOnlyRootFilesystem: true, capabilities: { drop: [ALL ↩
   ] } }
          livenessProbe: { httpGet: { path: /healthz, port: 9090 }, periodSeconds: 20 }
          readinessProbe: { httpGet: { path: /readyz, port: 9090 }, periodSeconds: 10 }
```

`job-template.yaml` (the worker renders this; placeholders in `{{ }}`):

```
apiVersion: batch/v1
kind: Job
metadata:
  name: "{{ .JobName }}"                      # tf-<sha256(run_id:attempt)[:20]>
  namespace: ssp-runs
  labels:
    app: tf-runner
    ssp.io/run-id: "{{ .RunID }}"
    ssp.io/tenant: "{{ .TenantLabel }}"       # label-safe form of tenant_id
    ssp.io/kind: "{{ .Kind }}"                # plan|apply|refresh|import|destroy
    ssp.io/lob: "{{ .LobID }}"
spec:
  backoffLimit: 0                             # the worker owns retries (new attempt → new job name)
  activeDeadlineSeconds: {{ .ActiveDeadlineSeconds }}
  ttlSecondsAfterFinished: {{ .TTLSeconds }}
  template:
    metadata:
      labels: { app: tf-runner, ssp.io/run-id: "{{ .RunID }}" }
    spec:
      serviceAccountName: tf-runner
      automountServiceAccountToken: false
      restartPolicy: Never
      nodeSelector: { ssp.io/pool: runs }
      tolerations: [{ key: ssp.io/runs, operator: Equal, value: "true", effect: NoSchedule }]
      securityContext: { runAsNonRoot: true, runAsUser: 65532, fsGroup: 65532, seccompProfile: { type: RuntimeDefault  ↩
   } }
      containers:
        - name: terraform
          image: "{{ .RunnerImage }}"
          args: ["{{ .Kind }}"]
          env:
            - { name: RUN_ID, value: "{{ .RunID }}" }
            - { name: ATTEMPT, value: "{{ .Attempt }}" }
            - { name: TENANT_ID, value: "{{ .TenantID }}" }
```

```
          - { name: TENANT_PROJECT, value: "{{ .TenantProject }}" }
          - { name: TENANT_SA, value: "{{ .TenantSA }}" }                 # ssp-tenant@<project>.iam.gserviceaccount ↵
    .com
          - { name: STATE_BUCKET, value: "{{ .StateBucket }}" }           # ssp-state-<lob>
          - { name: STATE_PREFIX, value: "{{ .StatePrefix }}" }           # <project>/
          - { name: EXPECTED_URI, value: "{{ .ExpectedURI }}" }           # gs://ssp-state-<lob>/<project>/expected/ ↵
    <gen>/
          - { name: ARTIFACTS_URI, value: "{{ .ArtifactsURI }}" }         # gs://ssp-state-<lob>/<project>/runs/<run ↵
    _id>/
          - { name: PLAN_FROM_RUN, value: "{{ .PlanFromRunURI }}" }       # apply only: artifacts URI of the plan ru ↵
    n
          - { name: POLICY_BUNDLE_URI, value: "{{ .PolicyBundleURI }}" }  # gs://ssp-catalog-<env>/policy/<digest>.t ↵
    ar.gz
          - { name: BATCH_INSTANCES, value: "{{ .BatchInstancesJSON }}" } # for the plan gate input
          - { name: TF_IN_AUTOMATION, value: "1" }
          - { name: TF_INPUT, value: "0" }
          - { name: TF_PLUGIN_CACHE_DIR, value: /opt/terraform/plugin-cache }
          - { name: TF_CLI_CONFIG_FILE, value: /opt/terraform/provider-mirror.tfrc }
          - { name: GOOGLE_IMPERSONATE_SERVICE_ACCOUNT, value: "{{ .TenantSA }}" }
        resources:
          requests: { cpu: "{{ .CPU }}", memory: "{{ .Memory }}" }        # run class: small 500m/1Gi, large 2/4Gi,  ↵
    xl 4/8Gi
          limits:   { cpu: "{{ .CPU }}", memory: "{{ .Memory }}" }
        securityContext: { allowPrivilegeEscalation: false, readOnlyRootFilesystem: true, capabilities: { drop: [ALL ↵
    ] } }
        volumeMounts:
          - { name: work, mountPath: /work }
          - { name: tmp, mountPath: /tmp }
      volumes:
        - { name: work, emptyDir: { sizeLimit: 4Gi } }
        - { name: tmp, emptyDir: { sizeLimit: 1Gi } }
```

3. Runner image

Dockerfile.runner:

```
# tf-runner: Terraform + OPA + minimal tooling; providers pre-mirrored; non-root; no shell access to secrets
FROM golang:1.23 AS tools
RUN go install github.com/google/go-containerregistry/cmd/crane@latest   # not needed at runtime; placeholder for buil ↵
   d tools

FROM debian:bookworm-slim AS base
ARG TERRAFORM_VERSION=1.13.3        # pin exactly; >= 1.10 required (S3 native locking, ephemeral values)
ARG OPA_VERSION=1.9.0               # pin exactly; must match ssp-cp's embedded OPA major
ARG TARGETARCH=amd64
RUN apt-get update && apt-get install -y --no-install-recommends ca-certificates curl unzip jq git && rm -rf /var/lib/ ↵
   apt/lists/*
RUN curl -fsSL -o /tmp/tf.zip https://releases.hashicorp.com/terraform/${TERRAFORM_VERSION}/terraform_${TERRAFORM_VERS ↵
   ION}_linux_${TARGETARCH}.zip \
 && unzip /tmp/tf.zip -d /usr/local/bin && rm /tmp/tf.zip
RUN curl -fsSL -o /usr/local/bin/opa https://openpolicyagent.org/downloads/v${OPA_VERSION}/opa_linux_${TARGETARCH}_sta ↵
   tic && chmod +x /usr/local/bin/opa
# gcs helper: small Go binary (ssp-workers/cmd/gcsutil) doing cp/ls with Workload Identity; avoids installing gcloud
COPY --from=gcsutil /gcsutil /usr/local/bin/gcsutil
# Provider mirror: built in CI by `terraform providers mirror` for the provider set in 40-repos/45-ssp-module-template ↵
   .md
COPY providers-mirror/ /opt/terraform/providers-mirror/
COPY provider-mirror.tfrc /opt/terraform/provider-mirror.tfrc
COPY runner.sh /usr/local/bin/runner.sh
RUN useradd -u 65532 -r -s /usr/sbin/nologin runner && mkdir -p /work /opt/terraform/plugin-cache && chown -R runner / ↵
   work /opt/terraform/plugin-cache
USER 65532
WORKDIR /work
ENTRYPOINT ["/usr/local/bin/runner.sh"]
```

provider-mirror.tfrc:

```
provider_installation {
  filesystem_mirror {
    path    = "/opt/terraform/providers-mirror"
    include = ["registry.terraform.io/hashicorp/*"]
  }
  direct { exclude = ["registry.terraform.io/*/*"] }   # no registry egress from runs
}
```

runner.sh (exit codes: 0 ok, 10 plan gate denied, 20 terraform error, 30 lock error, 40 setup error):

```
#!/usr/bin/env bash
```

```
set -euo pipefail
KIND="${1:?plan|apply|refresh|import|destroy}"
log(){ printf '%s %s\n' "$(date -u +%FT%TZ)" "$*"; }
finish(){ # $1 state $2 summary-json
  jq -n --arg s "$1" --argjson sum "${2:-{}}" --arg at "$(date -u +%FT%TZ)" --arg run "$RUN_ID" \
     '{run_id:$run,state:$s,summary:$sum,finished_at:$at}' > /work/result.json
  gcsutil cp /work/result.json "${ARTIFACTS_URI}result.json" || true
  gcsutil cp /work/run.log "${ARTIFACTS_URI}run.log" || true
}
trap 'rc=$?; [ $rc -ne 0 ] && finish failed "{\"exit_code\":$rc}"' ERR
exec > >(tee -a /work/run.log) 2>&1

log "run=$RUN_ID attempt=$ATTEMPT kind=$KIND tenant=$TENANT_ID"
# 1. Fetch expected generation (rendered root module) — never modified here
gcsutil cp -r "$EXPECTED_URI" /work/expected/ || { finish failed '{"phase":"fetch"}'; exit 40; }
cd /work/expected
# 2. Init with the LOB bucket backend; impersonate the tenant SA for backend and provider (15-minute tokens)
cat > backend.override.hcl <<EOF
bucket = "${STATE_BUCKET}"
prefix = "${STATE_PREFIX}"
impersonate_service_account = "${TENANT_SA}"
EOF
terraform init -input=false -no-color -lockfile=readonly -backend-config=backend.override.hcl || { finish failed '{"ph ↩
   ase":"init"}'; exit 20; }

case "$KIND" in
  plan|refresh|import)
    EXTRA=""; [ "$KIND" = refresh ] && EXTRA="-refresh-only"
    set +e; terraform plan -input=false -no-color -lock-timeout=5m -detailed-exitcode $EXTRA -out=plan.tfplan > plan.t ↩
   xt; rc=$?; set -e
    if [ $rc -eq 1 ]; then grep -qi 'Error acquiring the state lock' plan.txt && { finish failed '{"phase":"plan","cla ↩
   ss":"lock"}'; exit 30; }; finish failed '{"phase":"plan"}'; exit 20; fi
    terraform show -json plan.tfplan > plan.json
    # 3. Plan-stage policy (SSP-POL-2xx) with the pinned bundle
    gcsutil cp "$POLICY_BUNDLE_URI" /work/policy.tar.gz
    jq -n --arg t "$TENANT_ID" --argjson b "$BATCH_INSTANCES" --slurpfile p plan.json \
      '{stage:"plan",tenant:{tenant_id:$t},batch:{resource_instances:$b},plan:$p[0]}' > policy_input.json
    opa eval -b /work/policy.tar.gz -i policy_input.json -f json 'data.ssp.plan.decision' > policy.json
    RESULT=$(jq -r '.result[0].expressions[0].value.result' policy.json)
    SUMMARY=$(jq -c '{add:([.resource_changes[]?|select(.change.actions|index("create"))]|length),
                      change:([.resource_changes[]?|select(.change.actions|index("update"))]|length),
                      destroy:([.resource_changes[]?|select(.change.actions|index("delete"))]|length),
                      replace:([.resource_changes[]?|select(.change.actions==["delete","create"] or .change.actions== ↩
  ["create","delete"])]|length),
                      changed: ($rc!=0)}' --argjson rc $rc plan.json)
    for f in plan.tfplan plan.txt plan.json policy.json; do gcsutil cp $f "${ARTIFACTS_URI}$f"; done
    if [ "$RESULT" != "allow" ]; then finish denied "$(jq -c --argjson s "$SUMMARY" '.result[0].expressions[0].value + ↩
    {plan:$s}' policy.json)"; exit 10; fi
    finish succeeded "$SUMMARY"; exit 0 ;;
  apply|destroy)
    # 4. Apply exactly the gated plan file from the plan run (SSP-WF-022)
    gcsutil cp "${PLAN_FROM_RUN}plan.tfplan" /work/expected/plan.tfplan || { finish failed '{"phase":"fetch-plan"}'; e ↩
   xit 40; }
    set +e; terraform apply -input=false -no-color -lock-timeout=5m plan.tfplan > apply.log; rc=$?; set -e
    gcsutil cp apply.log "${ARTIFACTS_URI}apply.log"
    if [ $rc -ne 0 ]; then grep -qi 'Error acquiring the state lock' apply.log && { finish failed '{"phase":"apply","c ↩
   lass":"lock"}'; exit 30; }; finish failed '{"phase":"apply"}'; exit 20; fi
    terraform output -json > outputs.json           # worker filters to ServiceSpec.outputs (non-secret) before storin ↩
   g
    gcsutil cp outputs.json "${ARTIFACTS_URI}outputs.json"
    terraform state pull > state.json && jq '{serial, lineage, resources: [.resources[]|{module,type,name,instances:[. ↩
   instances[]|{attributes:{id:.attributes.id,name:.attributes.name}}]}]}' state.json > observed.json
    gcsutil cp observed.json "${ARTIFACTS_URI}observed.json"; rm -f state.json
    finish succeeded "$(jq -c '{outputs_keys:[keys[]]}' outputs.json)"; exit 0 ;;
esac
```

Requirements:

- SSP-RUN-001 The runner never prints provider credentials; `TF_LOG` is off by default; the worker applies the redaction regex set before exposing logs via the API.
- SSP-RUN-002 Secrets referenced by the rendered module are resolved by Terraform at apply time from Secret Manager data sources or `ephemeral` resources; the runner has no secret env vars.
- SSP-RUN-003 Providers come from the image mirror; a module requiring a provider not in the mirror fails `init` with a clear message (fix = new runner image, T2).
- SSP-RUN-004 Terraform and OPA versions are printed in the first log line and recorded in `result.json`.

- SSP-RUN-005 `plan.tfplan` is valid only against the same state serial; the worker rejects an apply if the plan run is older than 24 h or a newer generation exists.

4. Worker (Go, `ssp-workers/cmd/tf-worker`)

```
// Activity signature (ssp-workers/internal/tfrun/activity.go)
type RunInput struct {
    RunID, Attempt, Kind, TenantID, TenantProject, TenantSA, LobID string
    StateBucket, StatePrefix, ExpectedURI, ArtifactsURI, PlanFromRunURI, PolicyBundleURI string
    BatchInstances []string
    RunClass string // small|large|xl → CPU/Memory
}
type RunResult struct {
    State string `json:"state"` // succeeded|failed|denied
    Summary map[string]any `json:"summary"`
    JobName string `json:"job_name"`
}

func (a *Activities) RunTerraform(ctx context.Context, in RunInput) (*RunResult, error) {
    name := JobName(in.RunID, in.Attempt)                // "tf-" + hex(sha256(runID+":"+attempt))[:20]
    job, err := a.k8s.BatchV1().Jobs(a.ns).Get(ctx, name, metav1.GetOptions{})
    if apierrors.IsNotFound(err) {                         // first attempt or worker restarted before create
        job, err = a.k8s.BatchV1().Jobs(a.ns).Create(ctx, a.render(in, name), metav1.CreateOptions{})
        if err != nil && !apierrors.IsAlreadyExists(err) { return nil, temporal.NewNonRetryableApplicationError("job c ↩
   reate", "K8sError", err) }
    }
    ticker := time.NewTicker(a.pollInterval)             // 30s (D-12)
    defer ticker.Stop()
    for {
        select {
        case <-ctx.Done():                                 // activity cancelled/timed out → stop the job
            _ = a.k8s.BatchV1().Jobs(a.ns).Delete(context.Background(), name, metav1.DeleteOptions{PropagationPolicy:  ↩
   ptr(metav1.DeletePropagationForeground)})
            return nil, ctx.Err()
        case <-ticker.C:
            job, err = a.k8s.BatchV1().Jobs(a.ns).Get(ctx, name, metav1.GetOptions{})
            if err != nil { activity.RecordHeartbeat(ctx, "get-error"); continue }
            activity.RecordHeartbeat(ctx, map[string]any{"active": job.Status.Active, "job": name})
            if job.Status.Succeeded > 0 || job.Status.Failed > 0 {
                res, rerr := a.readResult(ctx, in.ArtifactsURI)   // result.json written by runner.sh; source of truth ↩
   , not pod logs
                if rerr != nil { return nil, fmt.Errorf("job finished but no result.json: %w", rerr) }
                res.JobName = name
                switch res.State {
                case "succeeded", "denied": return res, nil       // denied is a business outcome, not an error
                default:
                    if cls, _ := res.Summary["class"].(string); cls == "lock" { return nil, temporal.NewApplicationErr ↩
   or("state lock", "LockError") } // retryable
                    return nil, temporal.NewNonRetryableApplicationError("terraform failed", "TerraformError", res.Sum ↩
   mary)
                }
            }
        }
    }
}
```

Worker options: `worker.Options{MaxConcurrentActivityExecutionSize: 10, MaxConcurrentActivityTaskPollers: 4}`; activity options set by the reconciler: `StartToCloseTimeout: 90m (plan 30m), HeartbeatTimeout: 2m, RetryPolicy{InitialInterval: 2m, MaximumAttempts: 5, NonRetryableErrorTypes: ["TerraformError","K8sError"]}`. The worker also exposes `/metrics`: `ssp_tf_jobs_active`, `ssp_tf_runs_total{kind,state}`, `ssp_tf_run_duration_seconds{kind}`.

Scaling: start with 2 replicas (20 concurrent runs). Add KEDA's Temporal scaler on task-queue backlog per LOB queue later; node pool autoscaling handles the Jobs.

5. Install

```
sed -e "s/CP_PROJECT/<CP_PROJECT>/g" -e "s/REGION/<REGION>/g" -e "s/VERSION/<ver>/g" -e "s#KUBE_APISERVER_CIDR#<contro ↩
   l-plane-cidr>#g" \
    30-infra/tf-worker/namespace-rbac.yaml 30-infra/tf-worker/networkpolicy.yaml 30-infra/tf-worker/tf-worker-deployme ↩
   nt.yaml | kubectl apply -f -
kubectl -n ssp-workers rollout status deploy/tf-worker
```

Verification:

- [ ] `kubectl -n ssp-workers logs deploy/tf-worker` shows `polling task queue tf-runs-default`.
- [ ] Temporal UI → Task queues → `tf-runs-default` shows 2 pollers.
- [ ] A sandbox `refresh` run for a tenant with an empty desired set creates Job `tf-…`, completes, writes `result.json` and the run shows `succeeded` in the control room.
- [ ] Deleting the worker pod mid-run and letting Temporal retry re-attaches to the same Job (same name, no second Job created).
- [ ] Disabling `ssp-tf-runner` makes the next run fail at `init` with a permission error, and the control room banner shows the kill switch engaged.

6. Later: callback instead of polling

When polling cost matters, switch `RunTerraform` to **asynchronous activity completion**: the activity returns `activity.ErrResultPending` after creating the Job; the Job's last step calls a small `run-callback` endpoint in `ssp-workers` (in-cluster, mTLS, no public ingress) with the task token stored in the Job's ConfigMap; the endpoint calls `client.CompleteActivity(token, result)`. Keep the poller as a watchdog for lost callbacks.

34 — Control-Plane Postgres (Cloud SQL)

Holds Requested and Approved plus pointers and audit (D-03). Never Terraform state, never secrets.

1. Instances

Env	Instance	Tier	HA	Databases
sandbox	`ssp-sandbox`	db-custom-2-7680	zonal	`ssp`, `temporal`, `temporal_visibility` (shared to save cost)
nonprod	`ssp-nonprod`, `ssp-temporal-nonprod`	db-custom-2-7680	regional	split as named
prod	`ssp-prod`, `ssp-temporal-prod`	db-custom-4-16384	regional	split as named

```
gcloud sql instances create ssp-sandbox --project=<CP_PROJECT> --database-version=POSTGRES_16 --region=<REGION> \
  --tier=db-custom-2-7680 --network=projects/<CP_PROJECT>/global/networks/<VPC> --no-assign-ip \
  --enable-google-private-path --database-flags=cloudsql.iam_authentication=on,max_connections=400 \
  --backup-start-time=03:00 --enable-point-in-time-recovery --retained-backups-count=7 --deletion-protection
gcloud sql databases create ssp --instance=ssp-sandbox
gcloud sql users create ssp-cp-api@<CP_PROJECT>.iam --instance=ssp-sandbox --type=cloud_iam_service_account
gcloud sql users create ssp-control-worker@<CP_PROJECT>.iam --instance=ssp-sandbox --type=cloud_iam_service_account
gcloud sql users create temporal --instance=ssp-sandbox --password="$(gcloud secrets versions access latest --secret=t ↵
   emporal-db-password)"
```

2. Access model

- SSP-DB-001 `ssp-cp` connects with **IAM database authentication** through the Cloud SQL Go connector (`cloud.google.com/go/cloudsqlconn`, `WithIAMAuthN()`), using the pod's Workload Identity; no password, no proxy sidecar.
- SSP-DB-002 Temporal uses the `temporal` role with a password (Temporal does not support IAM auth); the password lives in Secret Manager and rotates quarterly (rotation runbook updates the K8s secret and restarts Temporal pods).
- SSP-DB-003 Private IP only; `authorized networks` empty; Cloud SQL Auth Proxy used by humans from the bastion for break-glass access, audited.
- SSP-DB-004 Roles inside `ssp`: `ssp_app` (DML on `ssp.*`), `ssp_migrate` (DDL, used only by the migration Job), `ssp_readonly` (control room analytics, auditors). The API runs as `ssp-cp-api@…iam` granted `ssp_app`.

3. Migrations

- `ssp-cp/db/migrations/*.sql` applied by `goose` from a Kubernetes Job (`ssp-cp/cmd/migrate`) in the release pipeline before the API rollout; forward-only; every migration has a rollback migration file.
- Partitioning: `order_event` and `audit_event` partitioned monthly (`pg_partman` or native with a monthly creation job).

4. Operations

Topic	Setting
Backups	Automated daily + PITR 7 days; monthly restore test into `ssp-restore-test`
Monitoring	Cloud SQL insights on; alerts on CPU > 80 % 10 min, connections > 80 % max, replication lag (prod), disk > 80 %
Maintenance	Window Sunday 03:00 local; deny period during release freezes
Connection limits	API pool max 20 per replica; control worker 10; `max_connections` sized for 3× peak

Topic	Setting
Data retention	`order_event` 2 years, `audit_event` 7 years (export to BigQuery monthly for long tail)

35 — State, Catalog and Artifact Buckets

Expected and Observed live in object storage, one bucket per LOB (D-06, D-07), one state prefix per tenant boundary (D-08). This is also where the blast-radius decision (G-0008) is implemented.

1. Buckets

Bucket	Project	Purpose	Access
`ssp-state-<lob>`	LOB host project (sandbox: <CP_PROJECT>)	`<tenant>/terraform.tfstate`, `expected/`, `runs/`	Tenant SA: objectAdmin **on its own prefix** (IAM condition). Control worker: objectCreator on `expected/*` and objectViewer on `runs/*` for all prefixes. Nobody else.
`ssp-catalog-<env>`	<CP_PROJECT>	Catalog bundles, `index.json`, policy bundles	ssp-cp-api, control worker, tf-runner (Platform SA), UI via API: objectViewer. Publisher (module CI): objectCreator
`ssp-modules-<env>`	<CP_PROJECT>	`module.zip` mirror per `ssp-module-<service>` version	Platform SA objectViewer (runners fetch modules through impersonated tenant SA? No: modules are platform artifacts; grant objectViewer to **tenant SAs via the LOB group**)
`ssp-corpus-<env>`	<CP_PROJECT>	Sanitized tenant states, historical request inputs and plan JSONs for gates	CI service account only
`ssp-temporal-archive-<env>`	<CP_PROJECT>	Temporal history archival	Temporal SA

```
LOB=pbm; P=<LOB_HOST_PROJECT>; KEY=projects/$P/locations/<REGION>/keyRings/ssp-$LOB/cryptoKeys/state
gcloud kms keyrings create ssp-$LOB --location=<REGION> --project=$P
gcloud kms keys create state --keyring=ssp-$LOB --location=<REGION> --purpose=encryption --project=$P --rotation-perio ↩
   d=90d --next-rotation-time=+90d
gcloud storage buckets create gs://ssp-state-$LOB --project=$P --location=<REGION> \
  --uniform-bucket-level-access --default-encryption-key=$KEY --public-access-prevention --soft-delete-duration=30d
gcloud storage buckets update gs://ssp-state-$LOB --versioning
gcloud storage buckets update gs://ssp-state-$LOB --lifecycle-file=35-state-lifecycle.json   # keep 50 noncurrent vers ↩
   ions, delete runs/ older than 180 d
# Allow the bucket's service agent to use the key
gcloud kms keys add-iam-policy-binding state --keyring=ssp-$LOB --location=<REGION> --project=$P \
  --member=serviceAccount:$(gcloud storage service-agent --project=$P) --role=roles/cloudkms.cryptoKeyEncrypterDecrypt ↩
   er
```

Per-tenant grant (IAM condition on the object prefix), run by the onboarding workflow:

```
T=<tenant project>; LOB=pbm
gcloud storage buckets add-iam-policy-binding gs://ssp-state-$LOB \
  --member=serviceAccount:ssp-tenant@$T.iam.gserviceaccount.com --role=roles/storage.objectAdmin \
  --condition="expression=resource.name.startsWith(\"projects/_/buckets/ssp-state-$LOB/objects/$T/\"),title=tenant-$T- ↩
   prefix"
# The tenant SA must also be able to list within its prefix for the backend's lock handling
gcloud storage buckets add-iam-policy-binding gs://ssp-state-$LOB \
  --member=serviceAccount:ssp-tenant@$T.iam.gserviceaccount.com --role=roles/storage.legacyBucketReader \
  --condition="expression=resource.name.startsWith(\"projects/_/buckets/ssp-state-$LOB/objects/$T/\"),title=tenant-$T- ↩
   list"
```

2. Blast-radius model (resolves the session's open contradiction)

Threat	Control
Compromise of one tenant SA	Reaches only its own prefix (IAM condition) and its own project roles

Threat	Control
Compromise of the Platform SA	Can mint tokens for tenant SAs (that is its job) but holds no bucket or project roles itself; disabling it is one command; every impersonation is logged (`iam.googleapis.com` GenerateAccessToken audit logs) with the run id in the token's justification
Compromise of the LOB bucket	Objects are CMEK-encrypted with the LOB key in the LOB project; secrets are kept out of state (SSP-PRIN-004), so a state file leak exposes configuration, not credentials
Operator mistake	Versioning + soft delete; `terraform state` surgery only via a runbook with a recorded run
Cross-LOB access	Nothing has roles on more than one LOB bucket except the control worker's narrow `expected/` writer role, which is per-bucket and reviewed quarterly

SSP-BKT-001 No principal other than the control worker has a role on a whole state bucket. SSP-BKT-002 The lifecycle rule never deletes current versions of `terraform.tfstate`. SSP-BKT-003 Bucket IAM is itself Terraform-managed from `ssp-foundation`-style code in `ssp-cp/deploy/foundation/` (T3), not by hand after sandbox.

3. AWS and Azure equivalents (for later)

	AWS	Azure
Bucket	`s3://ssp-state-<ou>` in the per-OU host account, SSE-KMS with the OU key, versioning, Object Lock off, `use_lockfile = true` in the backend (Terraform ≥ 1.10; no DynamoDB)	Storage account `sspstate<mg>` with blob versioning, CMK, lease-based locking native to the `azurerm` backend
Tenant identity	IAM role `ssp-tenant` in each account, trust policy allowing the platform role (via Workload Identity Federation from GKE) to assume with an external id per run	Managed identity per subscription; federated credential trusting the GKE OIDC issuer
Prefix isolation	Bucket policy conditions on `s3:prefix` / object key prefix per account role	Container per subscription with RBAC at container scope

36 — Observability

1. Pipeline

- OpenTelemetry SDKs in every Go service and worker (traces, metrics, logs via `slog` handler); OTel Collector (DaemonSet, namespace `observability`) exports traces to Cloud Trace, metrics to Google Cloud Managed Service for Prometheus (or the enterprise Prometheus), logs to Cloud Logging with a sink to the SIEM.
- Temporal server metrics scraped (ServiceMonitor or managed collection), Temporal Web for workflow inspection, Grafana dashboards: *SSP Fleet*, *Reconciler queues*, *Runs*, *Temporal health*, *API*.
- Correlation: every log line and span carries `order_id`, `run_id`, `tenant_id`, `workflow_id` when known; the API returns `X-Request-Id` and the UI shows it on error toasts.

2. Metrics (names are binding)

Metric	Type	Labels	Source
`ssp_orders_total`	counter	`kind,state,service_id,lob`	ssp-cp
`ssp_order_state_duration_seconds`	histogram	`from,to,service_id`	ssp-cp (events)
`ssp_preflight_duration_seconds`	histogram	`service_id,result`	ssp-cp
`ssp_policy_decisions_total`	counter	`stage,result,rule_id`	ssp-cp, runner (via result.json)
`ssp_reconciler_queue_depth` / `_oldest_seconds`	gauge	`tenant_id,lob`	control worker (workflow query exporter)
`ssp_tf_runs_total`	counter	`kind,state,lob`	tf-worker
`ssp_tf_run_duration_seconds`	histogram	`kind,run_class`	tf-worker
`ssp_tf_jobs_active`	gauge	`lob`	tf-worker
`ssp_drift_findings_open`	gauge	`class,lob`	drift worker
`ssp_postbuild_steps_total`	counter	`step,state`	postbuild worker
`ssp_controls_paused`	gauge	`scope`	ssp-cp
`ssp_platform_sa_enabled`	gauge	—	ssp-cp (IAM poll every 60 s)
`ssp_assistant_turns_total`	counter	`surface,intent,outcome`	ssp-cp

3. SLOs and alerts

SLO	Target	Alert
Order → Approved (no gate)	p95 < 60 s over 7 d	burn rate > 2 for 1 h
Approved → Observed, Phase 1 services	p95 < 20 min	burn rate > 2 for 1 h
API availability	99.9 %	5xx ratio > 0.5 % for 10 min
Run success rate (excluding policy denies)	> 97 %	< 95 % over 1 h
Reconciler queue age	p95 < 5 min	any tenant > 30 min
Drift scan freshness	every tenant scanned < 12 h	tenant unscanned > 24 h
Platform SA	enabled unless incident	flips to disabled → page

4. Logs and redaction

- SSP-OBS-001 Structured JSON only; no `fmt.Println`. Levels: debug off in prod.
- SSP-OBS-002 Redaction set (regexes for tokens, keys, passwords, connection strings) applied in the runner before upload and in the API before serving logs; a unit test asserts each pattern.
- SSP-OBS-003 Audit events are written to Postgres **and** emitted as logs with `audit=true` for the SIEM sink; retention 7 years in the archive.

5. Dashboards delivered with the platform

`ssp-cp/deploy/dashboards/*.json` (Grafana) and `30-infra/observability/alerts.yaml` (PrometheusRule) are part of the release; the control room links to them by URL.

41 — Repository Map and Conventions

Repo	Owns	Language / stack	Deployed to	Tier of typical change
`ssp-doc`	Specs, ADRs, tasks, factory rules (this repo)	Markdown, YAML, JSON Schema	—	T0 (docs) / T3 (constitution, factory)
`ssp-cp`	Go API (`ssp-api`), control worker (workflows + control-plane activities), policy baseline build, catalogctl, DB migrations, Helm chart	Go 1.23, chi, oapi-codegen, sqlc, Temporal SDK, OPA	GKE `ssp-cp`	T2
`ssp-workers`	`tf-worker`, `tf-runner` image, `postbuild-worker`, `drift-worker`, `gcsutil`, twins of ServiceNow/CMDB/Infoblox for tests	Go, bash, Docker	GKE `ssp-workers`, Jobs in `ssp-runs`	T2 (T3 for runner image security settings)
`ssp-ui`	Storefront, `@ssp/form-renderer`, `@ssp/ui` tokens	Next.js 15, TS, Tailwind v4, shadcn/ui	GKE `ssp-ui`	T2
`ssp-module-<service>`	One Terraform module + ServiceSpec + request schema + ui.yaml + pricing + Rego + import mapping + tests; publishes a catalog bundle	Terraform, Rego	catalog bucket	T1 (new) / T2 (in use)
`ssp-control-room`	Operator console + operator chat UI	Next.js, shared `@ssp/ui`	GKE `ssp-control-room`	T2
`ssp-migrate`	Migration wizard	Next.js, shared `@ssp/ui`	GKE `ssp-migrate`	T2
`ssp-acceptance`	Holdout scenarios, LLM-as-user harness, red-team sets, evaluator	Go/TS, YAML	CI only, private	human-only

Conventions (all repos)

- SSP-REPO-001 Default branch `main`, protected: PR required, linear history, required checks from `50-factory/54-gates.md`, CODEOWNERS review per tier. Agents push to `agent/<task-id>-<slug>` branches.
- SSP-REPO-002 Every repo has `AGENTS.md` (copied from `50-factory/cursor-rules/AGENTS.<repo>.md`), `.cursor/rules/*.mdc`, `CODEOWNERS`, `Makefile` with `make generate lint test build`, `deploy/chart/` (services) and `.github/workflows/{ci,release}.yml`.
- SSP-REPO-003 Conventional commits; PR title `[<task-id>] <summary>`; PR body template lists spec IDs, tier, blast radius, test evidence.
- SSP-REPO-004 Versioning: services use image tags `<semver>-<sha>`; modules use git tags `v<semver>` and publish bundles at the same version.
- SSP-REPO-005 Generated code is committed (`internal/api/gen`, `lib/api`) and verified unchanged by a CI step that regenerates and diffs.
- SSP-REPO-006 One GitHub App per agent role (`ssp-builder`, `ssp-reviewer`, `ssp-evaluator`, `ssp-release`) with repo permissions by role; no personal tokens in CI.
- SSP-REPO-007 Shared UI package `@ssp/ui` is published from `ssp-ui` to GitHub Packages and consumed by `ssp-control-room` and `ssp-migrate`; no copy-paste of components.

CODEOWNERS template

```
# Platform team owns everything by default
*                       @vaishnavi4068/ssp-platform
```

```
# Security co-owns identity, network, policy relaxations, runner image
/deploy/foundation/       @vaishnavi4068/ssp-platform @vaishnavi4068/ssp-security
/policy/                  @vaishnavi4068/ssp-platform @vaishnavi4068/ssp-security
/30-infra/                @vaishnavi4068/ssp-platform @vaishnavi4068/ssp-security
/.github/                 @vaishnavi4068/ssp-platform
/CODEOWNERS               @vaishnavi4068/ssp-platform
```

(Replace team slugs with the real GitHub teams; in the personal-account phase, individual handles.)

Cross-repo contracts

Contract	Producer	Consumers	Change rule
`12-openapi.yaml`	ssp-doc	ssp-cp (server), ssp-ui / control-room / migrate (clients)	oasdiff no-break or version bump
`15-service-spec.schema.json`	ssp-doc	ssp-cp (validator), every module repo (CI validate)	additive only within `v1`
Catalog bundle layout	ssp-doc §15	module CI (publish), ssp-cp (load), ssp-ui (render)	versioned `bundle.json.layout`
`RunInput` / `result.json`	ssp-doc §33	ssp-cp (reconciler), ssp-workers (worker, runner)	shared Go types in `ssp-cp/pkg/ssp`
Policy input documents	ssp-doc §14	ssp-cp, runner, module policies	additive only

42 — ssp-cp Repository Structure and Build Instructions

Go control plane: API, workflows, control-plane activities, policy baseline, catalog tooling, migrations. Implements `10-product/12-api-spec.md`, `13-workflows-spec.md`, `14-policy-spec.md` (baseline), `15-catalog-servicespec.md` (loader), `17-conversational-provisioning-spec.md` (gateway).

```
ssp-cp/
├── AGENTS.md  .cursor/rules/  CODEOWNERS  Makefile  go.mod
├── cmd/
│   ├── api/main.go                HTTP server (chi) — serves generated strict server
│   ├── worker/main.go             Temporal worker: task queue ssp-control (workflows + control activities); --selfte ↩
   st runs a hello workflow
│   ├── migrate/main.go            goose migrations (K8s Job)
│   ├── catalogctl/main.go         validate | publish | reload — used by module CI
│   └── policyctl/main.go          build platform bundle | impact-replay
├── api/openapi.yaml               copied from ssp-doc/10-product/12-openapi.yaml (CI verifies identical)
├── internal/
│   ├── api/gen/                   oapi-codegen (strict-server, types, chi)
│   ├── api/handlers/{catalog,destinations,orders,resources,runs,ops,imports,assistant}.go
│   ├── api/middleware/            requestid, otel, auth(jwt), persona, entitlement, idempotency, ratelimit, problem
│   ├── auth/                      JWKS cache, claims → principal, entitlement lookup (entitlements.yaml | ServiceNow ↩
   )
│   ├── catalog/                   bucket index loader, bundle cache, schema validation, derive (CEL via cel-go), pri ↩
   cing evaluator, code view renderer
│   ├── policy/                    embedded OPA, bundle fetch/verify by digest, request-stage eval, impact replay
│   ├── render/                    desired set → main.tf.json + backend.hcl + manifest.json (shared by reconciler and ↩
    /code)
│   ├── store/                     sqlc queries (queries/*.sql), tx helpers, LISTEN/NOTIFY event hub
│   ├── workflows/                 order.go, reconciler.go, drift.go, postbuild.go, import.go, release.go, testdata/h ↩
   istories/
│   ├── activities/                approval.go (ComputeApproval), generation.go, render.go, runs.go (CreateRun, Captu ↩
   reObserved, PlanGate), servicenow.go, cmdb.go, notify.go, inventory.go
│   ├── assistant/                 gateway.go (SSE), context.go, router.go, tools.go (calls own handlers in-process), ↩
    llm/{vertex,anthropic}.go, grounded/{contextual,vertexrag}.go
│   └── events/                    SSE hub, event types
├── pkg/ssp/                       shared types: RunInput, RunResult, PolicyInput, Bundle, OrderEvent (imported by ss ↩
   p-workers)
├── db/migrations/0001_init.sql …   from 11-domain-model.md
├── policy/platform/*.rego + tests  baseline rules SSP-POL-1xx/2xx
├── deploy/chart/                  Helm: api Deployment+Service+HPA, control-worker Deployment, migrate Job (pre-upgr ↩
   ade hook), ConfigMap, ServiceAccounts (WI annotations), NetworkPolicies
├── deploy/envs/{sandbox,nonprod,prod}.yaml
├── deploy/foundation/             Terraform for buckets, SAs, IAM conditions, Cloud SQL (T3; humans apply)
└── test/
    ├── integration/               testcontainers Postgres + Temporal dev server (temporalio/temporal dev-server imag ↩
   e), twins
    └── e2e/                       sandbox smoke: order → observed with ssp-module-spanner
```

Make targets

```
make generate   # oapi-codegen, sqlc, mocks; fails if git diff after regen
make lint       # golangci-lint, govulncheck, gitleaks
make test       # unit + workflow replay + policy tests
make itest      # integration (testcontainers)
make build      # images via ko or docker buildx; tags <semver>-<sha>
make bundle-policy  # policyctl build → ssp-catalog-<env>/policy/<digest>.tar.gz
```

Key implementation notes (binding)

- SSP-CP-001 `ComputeApproval` and `preflight` share one function `approval.Compute(ctx, input) (Result, error)`; the only difference is persistence.
- SSP-CP-002 Derive/guardrail `when` expressions are CEL with the variables `params`, `tenant`, `apm`, `lob`, `principal`; templates `{{ }}` are Go `text/template` restricted to field access (no functions besides `lower`, `upper`, `join`).
- SSP-CP-003 `render.RootModule(tenant, desiredSet)` is deterministic (sorted keys, canonical JSON); its output hash is the generation content hash; the same package produces `/orders/{id}/code`.

- SSP-CP-004 Reconciler workflow uses `workflow.Await` on a queue in workflow state; signals are deduplicated by `spec_hash`; `continue-as-new` carries the pending queue.
- SSP-CP-005 The API never calls Temporal synchronously on the request path except `SignalWithStart`/`ExecuteWorkflow` with a 2 s timeout; reads use Postgres.
- SSP-CP-006 Assistant tools call handler functions directly with the caller's principal (no HTTP loopback), so authorization is identical.
- SSP-CP-007 All external clients (ServiceNow, CMDB, Infoblox, IAM) sit behind interfaces with twin implementations selected by `SSP_INTEGRATIONS_MODE=twin|live`.
- SSP-CP-008 Secrets never enter workflow histories: activities receive references, resolve inside the activity, and return only receipts.

43 — `ssp-workers` Repository Structure

Temporal activity workers that touch the outside world: Terraform runs (via Kubernetes Jobs), post-build integrations, drift inventory; plus the runner image and the twins used for testing. Implements `30-infra/33-terraform-worker-on-gke.md`, `13-workflows-spec.md` §4–5 (activities), `14-policy-spec.md` (plan stage execution).

```
ssp-workers/
├── AGENTS.md  .cursor/rules/  CODEOWNERS  Makefile  go.mod  (imports github.com/vaishnavi4068/ssp-cp/pkg/ssp)
├── cmd/
│   ├── tf-worker/main.go           task queue tf-runs-<lob>; activity RunTerraform; metrics on :9090
│   ├── postbuild-worker/main.go    task queue post-build; activities cmdb.register, backup.enroll, monitoring.onboard ↩
   , notify.team, dns.register, firewall.request, escalate
│   ├── drift-worker/main.go        task queue drift; activities ClassifyDrift, InventoryAssets (Cloud Asset Inventory ↩
   )
│   ├── gcsutil/main.go             tiny gcs cp/ls binary for the runner image (Workload Identity, no gcloud)
│   └── run-callback/main.go        (later) async activity completion endpoint, in-cluster only
├── internal/
│   ├── tfrun/                      activity.go (RunTerraform), jobname.go (deterministic), template.go (Job render),  ↩
   result.go (read result.json), redact.go
│   ├── k8s/                        clientset wiring, informer-based job watcher (replaces polling without changing th ↩
   e activity contract)
│   ├── postbuild/                  one package per integration with Client interface + twin impl; receipts
│   ├── drift/                      classify.go (allowlists from ServiceSpec), inventory.go
│   └── metrics/
├── runner/
│   ├── Dockerfile                  = 30-infra/tf-worker/Dockerfile.runner
│   ├── runner.sh                   = 30-infra/tf-worker/runner.sh
│   ├── provider-mirror.tfrc
│   └── providers.lock.hcl          provider set + versions mirrored at image build (google, google-beta, random, null ↩
   , time; aws/azurerm later)
├── twins/                          HTTP fakes used by integration tests and sandbox: servicenow (CR create/approve, i ↩
   ncident), cmdb (CI CRUD), infoblox (record CRUD); each with a /__admin API to seed and inject failures
├── deploy/chart/                   tf-worker, postbuild-worker, drift-worker Deployments; ServiceAccounts; NetworkPol ↩
   icies; RBAC for ssp-runs
├── deploy/runs-namespace/          = 30-infra/tf-worker/namespace-rbac.yaml, networkpolicy.yaml
└── test/
    ├── unit/                       job name determinism, template rendering, result parsing, redaction
    ├── integration/                kind cluster + Temporal dev server: RunTerraform re-attach test (kill worker mid-r ↩
   un → same Job), lock-error retry, cancellation deletes Job
    └── e2e/                        sandbox GKE: refresh run on an empty tenant
```

Binding notes

- SSP-WK-001 `JobName(runID, attempt) = "tf-" + hex(sha256(runID + ":" + attempt))[:20]`; lowercase, ≤ 63 chars, RFC 1123.
- SSP-WK-002 The activity's source of truth for outcome is `result.json` in the run artifacts, not Pod exit codes or logs; if the Job finished and `result.json` is missing, the activity returns a retryable error so the reconciler can decide.
- SSP-WK-003 `redact.go` holds the regex set shared with `ssp-cp` (import from `pkg/ssp/redact`); tests include a known-secret corpus that must never appear in output.
- SSP-WK-004 Post-build activities are idempotent by key (`<step>:<resource_instance_id>`), persist receipts via the control-plane API (`internal` endpoint, Workload Identity auth), and never write to Postgres directly.
- SSP-WK-005 Twins are production-quality fakes: schema-validated requests, realistic latencies, failure injection; they are what the Scenario Evaluator runs against.
- SSP-WK-006 Runner image is rebuilt weekly (base image and provider CVEs) by a scheduled workflow; promotion to prod requires the blast-radius corpus plan to be unchanged.

44 — ssp-ui Repository Structure

Implements 10-product/16-ui-spec.md and the Sidekick surface of 17-conversational-provisioning-spec.md.

```
ssp-ui/
├── AGENTS.md  .cursor/rules/  CODEOWNERS  package.json (pnpm)  turbo.json
├── apps/storefront/                    Next.js 15 app
│   ├── app/(shell)/layout.tsx          TopBar, LeftNav, BottomComposer, SidekickPanel
│   ├── app/page.tsx                    Storefront Home
│   ├── app/skus/…  app/checkout/[serviceId]/…  app/orders/…  app/resources/…  app/drafts  app/settings
│   ├── app/api/ssp/[...path]/route.ts  authenticated proxy to ssp-cp (adds JWT; streams SSE)
│   ├── components/storefront/          HeroCard, DestinationTabs, DestinationCard, SkuCard, RecentlyViewed, DraftChi ↩
   ps, OrderTimeline, RunPanel, ResourceTable
│   ├── components/checkout/            Stepper, StepCard, FieldRow (number + key in mono), RadioCards, ArrayTable, D ↩
   erivedField, Receipt (Summary|Code tabs), PlaceOrderBar
│   ├── components/sidekick/            Panel, MessageList, PreflightCard, StatusCard, CitationChip, Composer
│   ├── lib/api/                        openapi-typescript client from ssp-doc/10-product/12-openapi.yaml + TanStack  ↩
   Query hooks
│   ├── lib/sse.ts  lib/auth.ts (Auth.js OIDC)  lib/recently-viewed.ts (localStorage)
│   └── tests/e2e/ (Playwright)  tests/fixtures/catalog/{gcp.spanner-instance,unknown-service}/
├── packages/form-renderer/             @ssp/form-renderer: schema+ui.yaml → steps/fields; Ajv 2020; widgets registry ↩
   ; receipt bindings; Storybook
├── packages/ui/                        @ssp/ui: shadcn components, tokens (globals.css), icons, typography; publishe ↩
   d to GitHub Packages
├── deploy/chart/                       Deployment (standalone Next server), Service, HPA, NetworkPolicy, ConfigMap ( ↩
   API base URL, OIDC issuer)
└── .github/workflows/{ci,release}.yml  lint, typecheck, unit, Storybook build, Playwright against twin-backed sandbo ↩
   x, Lighthouse CI, visual regression
```

Binding notes

- SSP-UIR-001 No service-specific code anywhere under apps/storefront; the unknown-service fixture test fails the build otherwise.
- SSP-UIR-002 Money, guardrails, policy results and code views are rendered from server responses only; the renderer has no pricing logic.
- SSP-UIR-003 @ssp/form-renderer is framework-light (React only), has 100 % typed widget contracts, and ships a rendererVersion; ui.yaml declares the version it targets.
- SSP-UIR-004 Tokens live in packages/ui/src/tokens.css exactly as in 16-ui-spec.md §2; the control room and migration apps import them, never redefine them.
- SSP-UIR-005 The storefront proxy enforces same-origin, forwards Idempotency-Key, and strips any X-Request-Id from the client (server issues it).
- SSP-UIR-006 Accessibility and visual regression gates per 04-definition-of-done.md.

45 — `ssp-module-<service>` Template (worked example: `ssp-module-spanner`)

One repository per Terraform service module (D-18). Each repo is a complete, self-describing **catalog bundle source**: the module, its ServiceSpec, request schema, UI template, pricing, policies, import mapping, tests and docs. The platform learns about a service only through the published bundle.

```
ssp-module-spanner/
├── AGENTS.md  .cursor/rules/  CODEOWNERS  Makefile  CHANGELOG.md  README.md (buyer docs; copied to bundle docs/)
├── catalog/
│   ├── service.yaml                ServiceSpec (15-catalog-servicespec.md §2) — metadata.id gcp.spanner-instance
│   ├── request.schema.json         buyer-settable params (§3)
│   ├── ui.yaml                     steps, widgets, receipt lines, presets (§4)
│   └── pricing.yaml                estimate model (§5)
├── policy/
│   ├── request.rego                package ssp.request.svc.spanner_instance
│   ├── plan.rego                   package ssp.plan.svc.spanner_instance   (verifies every MUST guardrail on plan JS ↩
   ON)
│   └── *_test.rego                 ≥ 90 % coverage; allow + deny fixtures per rule
├── import/
│   └── mapping.yaml                asset type → module address + attribute → param extraction (used by ssp-migrate)
├── main.tf  variables.tf  outputs.tf  versions.tf   the module (root of repo so `source` points at the repo/zip)
├── examples/{minimal,prod-multi-region}/           used by tests and docs
├── tests/
│   ├── unit.tftest.hcl             plan-only assertions (labels, CMEK, deletion protection, PU multiple of 100)
│   ├── apply.tftest.hcl            real apply in the sandbox project (nightly, not on every PR)
│   └── corpus/                     sanitized state snapshots for the blast-radius gate (pulled from ssp-corpus bucke ↩
   t in CI)
└── .github/workflows/
    ├── ci.yml                      fmt, validate, tflint, trivy config, terraform test (unit), opa test, regal, cata ↩
   logctl validate, blast-radius, policy-impact
    └── release.yml                 on tag v*: zip module → ssp-modules-<env>; publish bundle → ssp-catalog-<env>/<id ↩
   >/<ver>/; update index.json; notify Pub/Sub
```

Module contract

- SSP-MOD-001 Module variables are exactly the request schema properties (same names, compatible types) plus platform-provided: `project_id`, `labels` (map), `kms_key_name` (when `encryption` is declared), `name`. `catalogctl validate` enforces by inspecting the module.
- SSP-MOD-002 The module creates only resource types listed in `spec.resourceTypes`; policy rule SSP-POL-202 denies anything else at plan time.
- SSP-MOD-003 Outputs listed in `spec.terraform.outputs` are non-secret; any output marked `sensitive = true` is never listed and never captured.
- SSP-MOD-004 No provider blocks inside the module (the rendered root module supplies `google` with `impersonate_service_account`); `versions.tf` declares `required_providers` constraints only.
- SSP-MOD-005 Every resource sets `labels = var.labels` (merged with service labels); `deletion_protection` follows `var.deletion_protection`.
- SSP-MOD-006 Breaking changes (resource address changes) ship `moved` blocks; destroy/replace of existing resources requires a CHANGELOG `BREAKING` entry and makes the release T3.
- SSP-MOD-007 Semantic versioning; bundle version = git tag; `CHANGELOG.md` keeps a `## [x.y.z]` section per release with `Added/Changed/Guardrails/Breaking`.

Spanner example (excerpts)

`variables.tf`

```
variable "project_id" { type = string }
variable "name"       { type = string }
variable "labels"     { type = map(string) }
variable "kms_key_name" { type = string }
variable "config" {
  type = object({ mode = string, region = optional(string), multiRegionConfig = optional(string), processingUnits = nu ↩
   mber })
```

```
  validation { condition = var.config.processingUnits >= 100 && var.config.processingUnits % 100 == 0
               error_message = "processingUnits must be a multiple of 100 (REQ-E-SPAN-0001-06)." }
}
variable "databases" { type = list(object({ name = string, dialect = optional(string, "GOOGLE_STANDARD_SQL"), versionR ↩
   etentionPeriod = optional(string, "7d") })) }
variable "deletionProtection" { type = bool, default = true }
```

main.tf

```
locals { spanner_config = var.config.mode == "regional" ? "regional-${var.config.region}" : var.config.multiRegionConf ↩
   ig }

resource "google_spanner_instance" "this" {
  project          = var.project_id
  name             = var.name
  display_name     = var.name
  config           = local.spanner_config
  processing_units = var.config.processingUnits
  labels           = var.labels
  force_destroy    = false
  lifecycle { prevent_destroy = false }   # protection is enforced by deletion_protection on databases and by policy
}

resource "google_spanner_database" "db" {
  for_each                 = { for d in var.databases : d.name => d }
  project                  = var.project_id
  instance                 = google_spanner_instance.this.name
  name                     = each.key
  database_dialect         = each.value.dialect
  version_retention_period = each.value.versionRetentionPeriod
  deletion_protection      = var.deletionProtection
  encryption_config { kms_key_name = var.kms_key_name }     # REQ-E-SPAN-0001-02
}
```

import/mapping.yaml

```
assets:
  - assetType: spanner.googleapis.com/Instance
    moduleAddress: "google_spanner_instance.this"
    id: "{{ .name }}"                                    # projects/<p>/instances/<i>
    params:
      name: "{{ .resource.data.displayName | default (last .name) }}"
      config.processingUnits: "{{ .resource.data.processingUnits }}"
      config.mode: "{{ if hasPrefix .resource.data.config \"regional-\" }}regional{{ else }}multi-region{{ end }}"
  - assetType: spanner.googleapis.com/Database
    moduleAddress: "google_spanner_database.db[\"{{ last .name }}\"]"
    id: "{{ .name }}"
    params:
      databases[].name: "{{ last .name }}"
```

tests/unit.tftest.hcl

```
run "labels_and_cmek" {
  command = plan
  variables { project_id = "p", name = "claims-ledger", labels = { apm_id = "APM-1" }, kms_key_name = "projects/p/loca ↩
   tions/us/keyRings/k/cryptoKeys/c",
              config = { mode = "regional", region = "us-central1", processingUnits = 100 }, databases = [{ name = "le ↩
   dger" }] }
  assert { condition = google_spanner_database.db["ledger"].encryption_config[0].kms_key_name == var.kms_key_name
           error_message = "REQ-E-SPAN-0001-02: CMEK must be set" }
  assert { condition = google_spanner_instance.this.labels["apm_id"] == "APM-1"
           error_message = "REQ-E-SPAN-0001-05: labels must propagate" }
}
```

Second service to add next: `ssp-module-gcs-bucket` (`gcp.gcs-bucket`)

Smallest possible slice for end-to-end testing: one resource type, guardrails CMEK + public access prevention + retention by data class, `holdsData: true`, soft delete. Use it to validate the reconciler, runner and UI renderer before Spanner's multi-step form.

51 — Software Dark Factory Specification

How the platform, the apps, future services and migrations get built: humans write the specs in this repo and the holdout scenarios in `ssp-acceptance`; agents (Cursor agents in the IDE and in CI, Claude Code GitHub Actions for review) write the code; gates decide merges; humans approve by tier (D-18, SSP-PRIN-009).

1. Flow of one change

```
Spec change (ssp-doc PR, human) → Task (90-tasks/, human or Spec Scribe) → Builder (Cursor agent on agent/<task> bra ↩
   nch)
   → PR → Gates (CI) → Reviewer (fresh-context agent review) → Evaluator (holdout scenarios, T1+) → merge rule by ↩
    tier → Release
```

- SSP-FAC-001 No code change without a task; no task without spec IDs; no spec ID without a verification method.
- SSP-FAC-002 Builders see `ssp-doc` and the target repo only. They never see `ssp-acceptance`. Evaluator failures come back as symptoms ("order for Tier 1 Spanner ended regional, expected multi-region"), never as scenario text.
- SSP-FAC-003 Agents work on `agent/<task-id>-<slug>` branches with the role's GitHub App identity; commits carry `Task: <id>` and `Spec: <ids>` trailers.
- SSP-FAC-004 Retry budget: a builder may push up to 3 fix iterations after gate failures; then the PR goes to draft with a failure summary and the task is flagged for a human.
- SSP-FAC-005 Tokens per merged change and gate first-pass rate are recorded per task (Factory Steward dashboard).

2. Roles

Role	Where it runs	Reads	Writes	Credentials
Spec Scribe	Cursor (human-driven) or scheduled	Transcripts, stakeholder docs, this repo	Draft ADRs and tasks as PRs to `ssp-doc`	none
Spec Critic	Claude Code GitHub Action on `ssp-doc` PRs	`ssp-doc`	Review comments: contradictions with the decisions register, untestable requirements, missing owners/IDs	none
Planner	Cursor, first step of a task	task + cited specs	Context pack in the PR description: files to touch, interfaces, tests to write first	none
Test Author	Cursor (same session, before implementation)	specs	Tests citing requirement IDs; replay histories; Rego tests	none
Builder	Cursor agent (IDE or background)	context pack, target repo	Code, generated artifacts	sandbox only (twins, sandbox project via CI)
Reviewer	Claude Code GitHub Action on PRs (fresh context, different model where possible)	PR diff + cited specs	Advisory review: least privilege, secrets, idempotency, determinism, spec deviations	none
Scenario Evaluator	CI job from `ssp-acceptance` (private runner)	holdout scenarios, sandbox environment	Status check + symptom report	ephemeral sandbox
Chaos / Red Team	Nightly in sandbox	—	Issues: killed pods mid-apply, lock contention, injected drift, prompt injection in resource names	sandbox

Role	Where it runs	Reads	Writes	Credentials
Release	CI on tag	bundles, CHANGELOG	Evidence pack, CR draft, wave plan	publish to catalog bucket
Ops Sentinel	Scheduled (Phase 3)	fleet metrics, failures	Tickets, spec-change proposals	read-only
Factory Steward	Weekly	all PR/gate data	PRs to `50-factory/` rules and tasks templates	none

3. Cursor setup (what to put in each repo)

- `AGENTS.md` at the repo root (templates in `cursor-rules/`), read by Cursor agents and by Claude Code.
- `.cursor/rules/00-ssp-common.mdc` (always applied) + repo-specific `.mdc` files scoped by glob (e.g. `internal/workflows/**` → Temporal determinism rules).
- A `docs/specs` git submodule or a CI-synced copy of `ssp-doc` inside each repo so agents can read specs offline (`make sync-specs`).
- Task files are pasted or linked as the first message; the agent must begin by restating the spec IDs and the acceptance criteria (Planner step).
- Cursor **Background Agents** (or the IDE agent) may run `make generate lint test`; they must not run `terraform apply`, `kubectl apply` or `helm upgrade` against anything but kind/twins. CI owns real environments.
- PR review automation: install the Claude Code GitHub Action with the Reviewer and Spec Critic prompts (`cursor-rules/review-prompt.md`); Cursor's own PR review bot may run in addition.

4. Gates (summary; full list in `54-gates.md`)

Deterministic first: format/lint/vuln/secret scans; generated-code diff; unit and replay tests; OpenAPI no-break; schema compatibility; `catalogctl validate`; Rego tests and policy-impact; blast-radius plan against the corpus; traceability (every MUST ID in the task has a test); a11y and visual regression for UIs; then Reviewer; then Evaluator for T1+.

5. Holdout acceptance (`ssp-acceptance`, human-owned)

- End-to-end scenarios in YAML: persona, destination, service, goal, expected approved params, expected states and timings, expected post-build receipts.
- Conversational scenarios (SSP-CHAT §5) with golden `OrderInput`.
- Chaos scenarios: kill `tf-worker` during apply → same Job re-attached and order still reaches Observed; two orders for one tenant → serialized, both Observed; policy relaxation PR → gate T3.
- Scoring is probabilistic for conversational and timing-sensitive scenarios: `satisfaction = passing trajectories / total`; thresholds per tier in `54-gates.md`.

6. Factory for *future builds and migrations*

The same loop builds new services and migrations:

- **New service**: human writes `E-<SVC>-0001` ADR + task "create ssp-module-<svc>"; builder generates the repo from `45-ssp-module-template.md`; gates + T1 evaluator; release publishes the bundle; the storefront shows it without a deploy.
- **Migration of an app**: operator runs `ssp-migrate`; where mapping needs a new `import/mapping.yaml` entry or a new service, a task is created and built by the loop; the migration itself is executed by the platform, never by an agent with cloud credentials.
- **Platform change**: spec PR → tasks across `ssp-cp`/`ssp-workers`/`ssp-ui` with cross-repo contract checks (41-repo-map.md).

7. Decide before scaling the factory

- Approved models for CVS content (builder, reviewer, evaluator, assistant) and the token budget per merged change.
- Human review remains at T2/T3; the evidence pack (gates, reviewer notes, scenario scores, policy impact) is generated for every merge for audit.
- Corpus hygiene: the sanitizer that produces `ssp-corpus` is itself T3 and reviewed by security.

53 — Task Template

Tasks live in `90-tasks/<id>-<slug>.md`. IDs: `T-<nnnn>`. A task is the unit of agent work and of a PR.

```
---
id: T-0007
title: Implement POST /v1/orders/preflight
repo: ssp-cp
tier: T2
status: ready            # draft | ready | in_progress | review | done | blocked
specs: [SSP-API-010, SSP-API-011, SSP-PROD-020, SSP-PROD-030, SSP-PROD-031, SSP-CP-001, SSP-CP-002]
depends_on: [T-0004, T-0005]
owner: platform
---

## Goal
One sentence: what exists when this is done.

## Context pack
- Read: ssp-doc/10-product/12-api-spec.md §2 Orders, §3; 15-catalog-servicespec.md §2–3; 14-policy-spec.md §2.
- Files you may create or change: internal/api/handlers/orders.go, internal/catalog/{derive.go,guardrails.go,pricing.g ↩
   o}, internal/policy/request.go, internal/approval/compute.go, tests under the same packages.
- Files you must not touch: internal/api/gen/**, db/migrations/**, deploy/**.
- Interfaces to honour: `approval.Compute(ctx, Input) (Result, error)` is shared with ComputeApproval (SSP-CP-001).

## Acceptance criteria
1. Given the sandbox catalog `gcp.spanner-instance@1.2.0` and tenant `gcp:pay-prod-7a1c` (tier 1, PHI), a preflight wi ↩
   th `config.mode=regional` returns `approved_params.config.mode=multi-region` with a guardrail entry citing REQ-E-SP ↩
   AN-0001-04 (SSP-PROD-030).
2. `preview_hash` is stable across two identical calls and changes when any approved param, guardrail or bundle digest ↩
    changes (SSP-API-011).
3. Policy deny returns 200 with `policy.result=deny` and rule ids (preflight never 4xx on policy); entitlement failure ↩
    returns 403 problem (SSP-POL-100).
4. p95 latency < 300 ms for the fixture under `make bench`.
5. Tests cite every spec id above.

## Out of scope
Order creation (T-0008), assistant tools (T-0021).

## Evidence to attach to the PR
`make test` output, bench numbers, example request/response JSON.
```

Rules: SSP-TASK-001 a task lists at most one repo. SSP-TASK-002 every acceptance criterion is checkable by a test or a command. SSP-TASK-003 tasks are created by humans or by the Spec Scribe as PRs; agents never create tasks for themselves.

54 — CI Gates per Repository

Gates are GitHub Actions required checks. Names are binding (branch rulesets reference them). An agent cannot edit `.github/` (protected path).

Common (every repo)

Check	Tool	Fails when
`gate/secrets`	gitleaks	any finding
`gate/tier-label`	script	PR lacks exactly one `tier:T*` label or label lower than path rules require
`gate/task-link`	script	PR title lacks `[T-nnnn]` or body lacks `Spec:` ids
`gate/traceability`	script	a MUST id listed in the task has no test/policy/scenario citing it (greps `SSP-\w+-\d+` in tests)
`gate/specs-sync`	script	`docs/specs` copy differs from `ssp-doc@main`

`ssp-cp`

Check	Tool	Fails when
`gate/go-lint`	golangci-lint, govulncheck	any error / known vuln
`gate/generate-diff`	`make generate` + git diff	generated code stale
`gate/unit`	`go test ./...` with race	failure; coverage of changed packages drops
`gate/replay`	Temporal replayer over `testdata/histories`	non-deterministic change without `GetVersion`
`gate/openapi`	spectral lint + oasdiff vs `main`	breaking change without version bump
`gate/schema-compat`	script	`service-spec.schema.json` or request schema change not additive
`gate/policy`	`opa test`, regal, coverage ≥ 90 %	failure
`gate/policy-impact`	`policyctl impact-replay` vs corpus	newly-allowed inputs without `tier:T3`
`gate/itest`	testcontainers	failure
`gate/image`	trivy image, cosign sign	critical CVE

`ssp-workers`

Check	Tool	Fails when
`gate/go-lint`, `gate/unit`, `gate/image`	as above	
`gate/runner-image`	hadolint, trivy, `terraform version`/`opa version` pin check, non-root check	drift from `30-infra/tf-worker/Dockerfile.runner`
`gate/reattach`	kind + Temporal dev server integration test	second Job created on retry, or orphan Job after cancel
`gate/redaction`	known-secret corpus	any secret survives
`gate/manifests`	kubeconform + conftest (RBAC verbs allowlist, NetworkPolicy present)	deviation from `30-infra/tf-worker/*.yaml`

`ssp-ui`, `ssp-control-room`, `ssp-migrate`

Check	Tool	Fails when
`gate/ts`	eslint, tsc --noEmit	errors
`gate/unit`	vitest	failure
`gate/unknown-service`	renderer test with `fixtures/catalog/unknown-service`	render error
`gate/e2e`	Playwright vs twin-backed sandbox API	failure
`gate/a11y`	Lighthouse CI (Home, Checkout, Order, Sidekick)	a11y < 95 or perf < 85
`gate/visual`	Playwright snapshots	unexpected pixel diff
`gate/tokens`	script	any color/radius literal outside `@ssp/ui`

`ssp-module-<service>`

Check	Tool	Fails when
`gate/tf-static`	fmt, validate, tflint, trivy config	any
`gate/tf-test`	`terraform test` (unit)	failure; MUST guardrail without assertion
`gate/catalog`	`catalogctl validate`	ServiceSpec/schema/ui/pricing inconsistent; variables mismatch
`gate/policy`	opa test + regal + coverage	failure
`gate/blast-radius`	plan new version against `ssp-corpus` sanitized states	any destroy/replace not declared in CHANGELOG; declared ones force `tier:T3`
`gate/import-mapping`	script	resources/variables changed without `import/mapping.yaml` update
`gate/apply-nightly`	`terraform test` apply in sandbox project (schedule)	failure opens an issue

`ssp-doc`

Check	Tool	Fails when
`gate/md-lint`, `gate/links`	markdownlint, lychee	broken links
`gate/ids`	script	duplicate or malformed `SSP-*`/`REQ-*` ids; requirement without Verify-by
`gate/decisions-locked`	script	change under `00-constitution/02-decisions-register.md` without `tier:T3` and a superseding ADR link
`gate/spec-critic`	Claude Code GitHub Action with `cursor-rules/review-prompt.md` (spec mode)	advisory

Evaluator (T1+; status `gate/evaluator`)

Runs from `ssp-acceptance` on a private runner against the sandbox deployment of the PR (ephemeral namespace): holdout end-to-end and conversational scenarios. Thresholds: T1 ≥ 0.9 satisfaction and 0

red-team successes; T2 ≥ 0.95; failures are posted as symptoms only.

60 — Architecture Decision Records

Five categories (D-16): **Global** (`G-`), **Business Process** (`B-`), **Endpoint** per service (`E-<SVC>-`), **Per-Cloud** (`C-<CLOUD>-`), **Automation** (`A-`). Each ADR has a named owner and a shared mailbox; the platform team are the sole PR approvers.

Folder layout

```
60-adr/
├── README.md              this file
├── global/                G-0001 …
├── business-process/      B-0001 …
├── endpoint/<service>/    E-SPAN-0001, E-GCS-0001 …
├── cloud/{gcp,aws,azure}/ C-GCP-0001 …
└── automation/            A-0001 …
```

Template

Frontmatter: `id, title, category, status (proposed | accepted | superseded), owner, mailbox, applies_to {clouds, services}, depends_on, supersedes, decided_in`. Sections: **Goal · Context · Decision · Requirements (table: ID | Requirement | Verify by) · Guardrails · Success criteria · Alternatives considered · Consequences · Open questions.**

Requirement IDs are `REQ-<ADR id>-<nn>`; every MUST row names a verification method (`tftest`, `rego:request`, `rego:plan`, `scenario`, `unit`, `manual`). The Spec Critic rejects rows without one. Endpoint ADR guardrails compile into the service's `ServiceSpec.guardrails`.

Backlog and status

The ADR IDs, titles and current status (decided in the design session vs. open) are listed in `00-constitution/02-decisions-register.md` and in the handoff document *02-dark-factory-plan.md* ("First ADR backlog"). Writing the decided ones down is the first Spec Scribe work: see `90-tasks/T-0001-write-phase0-adrs.md`. Open ones (G-0006 labels, G-0007 security baseline, G-0008 blast radius, C-AWS-*, C-AZ-*, A-0004 concurrency, A-0006 drift, B-0001 RBAC, B-0005 break-glass, B-0006 decommission) need the owners named in the register before they can be accepted.

70 — Acceptance (holdout)

Acceptance scenarios are **not in this repository** by design. They live in the private repo `ssp-acceptance`, which is never opened in a builder workspace and whose CI runner is the only thing that reads it (SSP-FAC-002). Builders receive symptoms, not scenarios.

What `ssp-acceptance` contains (human-written):

- `scenarios/e2e/*.yaml` — persona, destination, service, goal, expected approved params, expected state sequence and timings, expected post-build receipts.
- `scenarios/conversation/*.yaml` — persona, goal, messy opening line, golden `OrderInput`; run with an LLM-as-user harness.
- `scenarios/chaos/*.yaml` — kill worker mid-apply, lock contention, drift injection, Platform SA disable.
- `redteam/*.yaml` — prompt injection via resource names and ticket text, entitlement escalation, delete coercion.
- `evaluator/` — runner that deploys the PR to an ephemeral sandbox namespace, executes scenarios against twins and the sandbox project, scores satisfaction, posts `gate/evaluator`.

Thresholds are in `50-factory/54-gates.md`. Scenario authorship requires a human reviewer from outside the builder's task.

81 — Bootstrap Runbook: from an existing GCP project and GKE cluster to the first order

Target: a sandbox where one buyer orders a `gcp.gcs-bucket` or `gcp.spanner-instance` through the storefront and it reaches Observed. Every step names the spec it implements and the check that proves it.

#	Step	Spec	Command / action	Check
1	Enable APIs	31	`gcloud services enable container.googleapis.com sqladmin.googleapis.com secretmanager.googleapis.com cloudkms.googleapis.com cloudasset.googleapis.com artifactregistry.googleapis.com iamcredentials.googleapis.com pubsub.googleapis.com storage.googleapis.com --project=<CP_PROJECT>`	all listed as enabled
2	Node pool `ssp-runs`	31 §2	command in 31 §2	pool exists, 0 nodes idle
3	Namespaces + network policies	31 §1, 32 §2, 33 §2	`kubectl apply -f 30-infra/tf-worker/namespace-rbac.yaml`; create `temporal`, `ssp-cp`, `ssp-workers`, `ssp-ui` namespaces; apply network policies	`kubectl get netpol -A` shows default-deny in `ssp-runs`
4	Service accounts + Workload Identity	31 §3	script in 31 §3	`gcloud iam service-accounts list` shows 6 SAs; WI bindings present
5	Secret Manager secrets	31 §4	create `temporal-db-password`, `oidc-client`, (POC) `github-modules-ro`	versions exist
6	Cloud SQL	34	commands in 34 §1; create DBs `ssp`, `temporal`, `temporal_visibility`	`gcloud sql instances describe` → RUNNABLE, private IP
7	Buckets + KMS (sandbox LOB sbx in CP project)	35	commands in 35 §1 for `ssp-state-sbx`, plus `ssp-catalog-sandbox`, `ssp-modules-sandbox`, `ssp-corpus-sandbox`	buckets exist with CMEK, versioning, PAP
8	Temporal	32	`helm upgrade --install …` from 32 §3; post-install from §4	cluster health SERVING; namespace `ssp`; search attributes listed
9	Artifact Registry + first images	31 §5	`gcloud artifacts repositories create ssp --repository-format=docker --location=<REGION>`; build and push `ssp-api`, `ssp-control-worker`, `tf-worker`, `tf-runner`, `ssp-ui`	images listed
10	`ssp-cp` migrate + deploy	42, 34 §3	`helm upgrade --install ssp-cp deploy/chart -f deploy/envs/sandbox.yaml -n ssp-cp` (pre-upgrade hook runs migrations)	`/readyz` 200; tables exist

#	Step	Spec	Command / action	Check
11	Workers	33 §5	apply `tf-worker-deployment.yaml` (and later post-build/drift)	Temporal UI shows pollers on `tf-runs-default`
12	First tenant (sandbox project)	31 §3 tenant onboarding, 35 §1 per-tenant grant	create `ssp-tenant@<T>`; token-creator for Platform SA; prefix IAM on `ssp-state-sbx`; insert `lob` and `tenant` rows (`catalogctl tenant add` or SQL)	`GET /v1/destinations` lists it for a test principal
13	Entitlements + test principal	10 §1	`entitlements.yaml` with the test user → APM	`GET /v1/destinations` as that user returns the tenant
14	First module bundle	45, 15	tag `ssp-module-gcs-bucket` `v0.1.0` (or spanner); release workflow publishes to `ssp-catalog-sandbox`; `POST /v1/admin/catalog/reload`	`GET /v1/catalog/services` shows it
15	Policy baseline bundle	14, 42	`make bundle-policy` in ssp-cp	digest visible in `GET /v1/ops/policy/impact`
16	UI	44	`helm upgrade --install ssp-ui …`; OIDC client configured	Home renders SKUs and destination
17	Empty refresh run	33 §5 verification	`POST /v1/ops/tenants/<t>/drift-scan`	Job `tf-…` completes; `result.json` present
18	**First order**	10 §5	In the storefront: Configure & Buy → Place order	Order reaches `observed`; resource visible in `GET /v1/tenants/<t>/resources`; state object in bucket
19	Re-attach test	33 §5	delete `tf-worker` pod during an apply	same Job name, order still observed
20	Kill-switch test	31 §3 SSP-INF-010	`gcloud iam service-accounts disable ssp-tf-runner@…`; place an order	run fails at init with permission error; control room banner; re-enable

Record the date, operator and output of each check in `80-runbooks/bootstrap-log-<env>.md` (that file is the evidence that sandbox exists as specified).

Order of code work that unblocks the runbook

1. `ssp-cp`: migrations, catalog loader, `/catalog`, `/destinations`, `preflight`, `orders`, `OrderWorkflow` + `TenantReconciler` + render + `CreateRun/CaptureObserved` activities (tasks T-0002…T-0010).
2. `ssp-workers`: `tf-worker` + runner image + `gcsutil` (T-0011…T-0013).
3. `ssp-module-gcs-bucket` or `ssp-module-spanner`: module + catalog + release workflow (T-0014).
4. `ssp-ui`: shell, Home, Checkout with renderer, Order page (T-0015…T-0018).
5. Control room fleet page and kill switch (T-0019); Sidekick inquire/provision (T-0021).

90 — Tasks

One file per task (`T-nnnn-slug.md`), format in `50-factory/53-task-template.md`. Phase 0/1 tasks T-0001…T-0022 are ordered so that the bootstrap runbook (80-runbooks/81-bootstrap.md) can be completed: ssp-cp core (T-0002…T-0010) → workers (T-0011…T-0013) → first module (T-0014) → UI (T-0015…T-0018) → control room, migrate, Sidekick (T-0019…T-0021) → acceptance (T-0022, human-only).

ID	Repo	Tier	Title
T-0001	ssp-doc	T0	Write the decided ADRs (G-0001..G-0005, C-GCP-0001, C-GCP-0002, A-0001..A-0003, B-0007) from the decisions register using the ADR template
T-0002	ssp-cp	T2	Create Postgres migration 0001 from the domain model, sqlc queries and the store package
T-0003	ssp-cp	T2	Catalog bundle loader from GCS index.json, ServiceSpec/schema validation, derive and guardrail evaluation, pricing evaluator, code view renderer
T-0004	ssp-cp	T2	Embedded OPA request-stage evaluation, baseline platform rules SSP-POL-1xx with tests, bundle fetch by digest
T-0005	ssp-cp	T2	oapi-codegen strict server from 12-openapi.yaml, chi wiring, middleware (request id, OTel, JWT auth, persona, entitlement, idempotency, problem details), /healthz /readyz
T-0006	ssp-cp	T2	Implement /destinations and /destinations/{id}/context from entitlements and tenant/APM records
T-0007	ssp-cp	T2	Implement POST /orders/preflight via approval.Compute (shared with ComputeApproval)
T-0008	ssp-cp	T2	POST /orders with preview_hash check, order_event stream (SSE via LISTEN/NOTIFY), cancel, signoff, code endpoints
T-0009	ssp-cp	T2	OrderWorkflow with ComputeApproval, gate loop (CR via ServiceNow twin, owner sign-off), hand-off to reconciler, finalize
T-0010	ssp-cp	T2	TenantReconciler entity workflow: batching, AllocateGeneration, Render (main.tf.json), CreateRun, PlanGate, CaptureObserved, DriftScan schedule
T-0011	ssp-workers	T2	tf-worker: RunTerraform activity with deterministic Job names, re-attach, polling with heartbeats, result.json as source of truth, cancellation cleanup, metrics
T-0012	ssp-workers	T3	tf-runner image (Dockerfile.runner, runner.sh, provider mirror, gcsutil) exactly as 30-infra/tf-worker
T-0013	ssp-workers	T3	Helm chart and runs-namespace manifests mirroring 30-infra/tf-worker; NetworkPolicies; RBAC

ID	Repo	Tier	Title	
T-0014	ssp-module-gcs-bucket	T1	Create ssp-module-gcs-bucket (or spanner) from the module template: module, ServiceSpec, request schema, ui.yaml, pricing, policies, import mapping, tests, release workflow	
T-0015	ssp-ui	T2	App shell (top bar with destination selector, search ⌘K, Sidekick button, orders, theme), left nav with frequent SKUs, Storefront Home from catalog and destinations	
T-0016	ssp-ui	T2	@ssp/form-renderer: steps, widgets (text, select, radio-cards, number+unit, toggle, table, key-value, derived read-only), Ajv validation, receipt bindings, Storybook	
T-0017	ssp-ui	T2	Configure & Checkout page with stepper, numbered fields with mono keys, sticky Order Summary (Summary	Code), debounced preflight, Place order with preview_hash
T-0018	ssp-ui	T2	Order tracking page (SSE timeline, run panel, post-build checklist), Fulfilled Orders and Resources tables, resource detail actions	
T-0019	ssp-control-room	T2	Control room shell, Fleet page, Tenants, Runs with log viewer, pause/resume with blast-radius dialog, two-person kill switch	
T-0020	ssp-migrate	T2	Migration wizard steps 1-6 against ImportWorkflow: source, inventory, mapping editor (schema-aware), onboard, dry-run diff, reconcile differences	
T-0021	ssp-cp	T2	Assistant gateway: context assembly, router, knowledge path via Grounded interface (contextual/vertexrag), action path with preflight cards, SSE events; Sidekick panel wiring in ssp-ui	
T-0022	ssp-acceptance	T3	(human-only) Write Phase 1 holdout scenarios: e2e for first service, 10 conversational scenarios with golden OrderInput, 5 chaos, 10 red-team; evaluator posts gate/evaluator	